Imperial College London

Applied Project Report

Imperial College London
Imperial College Business School
MSc Risk Management & Financial Engineering

Statistical Arbitrage of Equity Index Volatility:

A Machine Learning Approach to Dispersion Trading

Nathan SEBBAG
CID: 06035175

July 30, 2026

Word count (main body, Sections 1–6): approximately 3,100 words

Applied Project submitted in partial fulfilment of the requirements for the MSc Risk Management & Financial Engineering.

Project specification

The following brief is written from the point of view of a fictitious client: the portfolio manager of a systematic volatility book at a multi-strategy hedge fund.

To: Quantitative Research
From: PM, Systematic Volatility
Re: Is the S&P 500 correlation risk premium still worth trading?

We run a short-volatility book and have historically sold index variance against single-name variance to harvest the correlation risk premium. The classic evidence for this trade is Driessen, Maenhout and Vilkov (2009), but their sample stops in the early 2000s. Spreads have compressed since, execution has moved electronic, and I want to know whether the trade still earns its keep on a modern sample before I allocate to it.

Concretely, I need answers to four questions.

1. **Does the premium still exist?** Measure the gap between the correlation implied by S&P 500 option prices and the correlation that subsequently realises, over as long a modern sample as the data allow, with proper overlapping-window statistics. Tell me whether it is still there, and whether it has changed over time.

2. **Can a systematic book capture it net of costs?** Build the vega-neutral dispersion trade, backtest it under realistic transaction costs, and report performance the way a desk would read it: Sharpe, but also skew and drawdown. I care about the tail as much as the mean.

3. **Does correlation-matrix cleaning help?** The realised correlation matrix is noisy. Test whether a Random Matrix Theory cleaning of it improves the trade, and be honest about where any improvement actually comes from.

4. **Can machine learning time the trade?** Test whether a supervised timing signal, in particular one built on spectral features of the correlation matrix, can beat a simple VIX-based rule at deciding when to put the trade on. A clean negative answer is as useful to me as a positive one.

The main output is this report. I want a clear, evidenced answer to each question, the honest limitations of the analysis, and a single recommended configuration I could actually run. Reproducible code supporting the results should accompany the report as an appendix.

Full implementation: `https://github.com/NSB22/Dispersion-Trading-Project` (source code in Appendix G)

Contents

Project specification **1**

1 Introduction **3**

2 Theory **4**

2.1 Implied correlation . . . 4
2.2 The premium as compensation for correlation risk . . . 4
2.3 The vega-neutral book . . . 4
2.4 Cleaning the correlation matrix . . . 5

3 Data and methodology **6**

3.1 Data sources . . . 6
3.2 Universe construction . . . 6
3.3 Realised measures and the two signal objects . . . 6
3.4 The backtest engine . . . 6
3.5 The machine-learning protocol . . . 7

4 Results **8**

4.1 The correlation risk premium exists, and has compressed . . . 8
4.2 The baseline trade: real, and nearly killed by frictions . . . 8
4.3 RMT helps, but the value is the estimation window, not the clipping . . . 10
4.4 The ML timing layer is a null; a regime gate trims the tail . . . 10

5 Limitations **14**

6 Conclusion **15**

References **16**

A Repository guide **18**

A.1 Structure . . . 18
A.2 Setup . . . 18
A.3 Running the pipeline . . . 19

B Mathematical derivations **20**

B.1 The equicorrelation inversion . . . 20
B.2 The index variance-risk-premium decomposition (sketch) . . . 20
B.3 Model-free implied variance (Carr–Madan) . . . 20
B.4 Relative greeks and the vega-neutral ratio . . . 21

C Additional robustness exhibits **22**

C.1 Meta-model predictions and feature importance . . . 22
C.2 The parsimonious leg . . . 22
C.3 Threshold and cost sensitivity . . . 24
C.4 Convexity of the frozen book and the return distribution . . . 24

D Data quality and known issues **26**

E Transaction-cost calibration **27**

F Statement on the use of generative AI **28**

G Source code **29**

G.1 Pipeline entry point . . . 29
G.2 Data acquisition (`src/dispersion/data/`) . . . 31
G.3 Signal construction (`src/dispersion/signal/`) . . . 45
G.4 Random matrix theory cleaning (`src/dispersion/rmt/`) . . . 48
G.5 Backtest engine (`src/dispersion/backtest/`) . . . 53
G.6 Machine learning layer (`src/dispersion/ml/`) . . . 63
G.7 Shared utilities (`src/dispersion/utils/`) . . . 75
G.8 Tests (`tests/`) . . . 78

1 Introduction

Investors buy index options as portfolio insurance, and that demand shows up in prices. Because index variance is a function of constituent variances and the correlations between them, an expensive index option, for given single-name option prices, mechanically implies an expensive correlation. Driessen, Maenhout and Vilkov (2009), hereafter DMV, showed that the correlation implied by S&P 500 option prices systematically exceeds the correlation that subsequently realises, and that this gap, the correlation risk premium (CRP), is priced. The gap is visible in raw data: on 31 March 2020, at the depth of the COVID sell-off, the mean 91-day implied volatility of our top-100 constituents was 46.3% against 37.6% for the index itself.

A dispersion trade harvests the premium directly. Sell the index straddle, buy a basket of constituent straddles, and size the legs so the book is vega-neutral: what remains is a short position in correlation, which pays off when realised correlation comes in below the level the market charged.

The client specification asks whether the premium still exists on a modern sample, whether a systematic book can capture it net of realistic costs, whether Random Matrix Theory (RMT) cleaning of the correlation matrix helps, and whether machine learning can time the trade. We answer on a point-in-time top-100 S&P 500 universe built from CRSP and OptionMetrics (1996 to 2024, 7,221 trading days), with a DMV-style vega-neutral backtest under a calibrated cost model and a purged walk-forward protocol for the ML layer. The main output of the project is this report; the code behind every number is listed in Appendix G.

The results are mixed in an instructive way. The premium exists, at $+0.079$ on average ($t = 7.4$), but it compressed to a statistically insignificant $+0.028$ after 2020, which fits a limits-to-arbitrage account. The unconditional trade replicates DMV almost exactly (gross Sharpe 0.77 against their 0.73) and, like theirs, is nearly killed by frictions. The RMT gate transforms the tail profile, though an ablation places the value in the 252-day estimation window rather than the eigenvalue clipping. The supervised ML timing layer is a clean null. We report these nulls plainly; they are findings, not failures.

2 Theory

2.1 Implied correlation

Under a one-factor approximation, in which a single average correlation $\bar{\rho}$ stands in for all pairs, index variance decomposes as

$$\sigma_I^2 \;=\; \sum_{i=1}^{N} w_i^2 \sigma_i^2 \;+\; \bar{\rho} \sum_{i \neq j} w_i w_j \sigma_i \sigma_j . \tag{1}$$

Solving for $\bar{\rho}$ and feeding implied volatilities gives the implied correlation:

$$\rho_{\text{implied}} \;=\; \frac{\sigma_I^2 - \sum_i w_i^2 \sigma_i^2}{\left(\sum_i w_i \sigma_i\right)^2 - \sum_i w_i^2 \sigma_i^2}, \tag{2}$$

where σ_I is the index implied volatility, σ_i the constituent implied volatilities and w_i the index weights. The denominator is strictly positive for $N \geq 2$, and the formula respects its natural bounds: $\rho_{\text{implied}} \leq 1$ exactly when the index vol sits below the weighted-average constituent vol (the diversification inequality), and $\rho_{\text{implied}} \geq 0$ whenever σ_I^2 exceeds the concentration term, which is tiny at $N = 100$. DMV build the same object from model-free implied variance (MFIV), the Carr and Madan (1998) integral of option prices across strikes, which captures the whole smile rather than a single point. We use at-the-money volatilities instead, a choice we defend in Section 3 and quantify in Section 4.

2.2 The premium as compensation for correlation risk

DMV's identification runs through the variance risk premium (VRP). Applying Itô's lemma to the index decomposition and changing measure, the index VRP splits into a weighted sum of individual VRPs plus terms in the difference between risk-neutral and physical correlation; the convexity corrections cancel in the $\mathbb{Q} - \mathbb{P}$ difference. Empirically, individual variance premia are close to zero (Bakshi & Kapadia, 2003; Bakshi et al., 2003), so the large negative index VRP must be carried by the correlation terms. Selling index variance and buying individual variance isolates precisely that exposure. Appendix B sketches the derivation.

2.3 The vega-neutral book

The trade is built from 91-day at-the-money straddles: short the index straddle, long the constituent straddles. The hedge is triangular. First the vega leg: sizes are set so that a proportional shock to all volatilities leaves the book flat. Since an ATM straddle's relative vega is approximately $1/\sigma$ (Appendix B), this convention pins the component leg near 100% of the short index notional, which is exactly DMV's +101.12%. The vol-of-vol cancels in the relative-vega ratio, so the hedge needs no vol-dynamics model. Second the delta leg: under the same one-factor structure, the residual delta of the vega-hedged book loads on the index alone while idiosyncratic deltas diversify away, so we hedge daily with a single index instrument. Daily delta-hedging a straddle is gamma scalping; to first order the hedge P&L accrues at

$$d\Pi \;\approx\; \tfrac{1}{2}\,\Gamma S^2 \left(\sigma^2_{\text{realised}} - \sigma^2_{\text{implied}}\right) dt, \tag{3}$$

so the hedged book pays off on realised-versus-implied variance, and through the index leg on realised-versus-implied correlation. That is the bet, cleanly stated.

2.4 Cleaning the correlation matrix

The one-factor inversion collapses a 100×100 matrix into one number, and the sample matrix it summarises is noisy. For N assets on T observations with $q = N/T$, a purely random matrix has eigenvalues confined to the Marchenko and Pastur (1967) bulk $\left[(1-\sqrt{q})^2, (1+\sqrt{q})^2\right]$. Financial spectra show a three-tier structure instead: one large market eigenvalue, a handful of sector modes, and a bulk statistically indistinguishable from noise (Laloux et al., 1999). Because the market mode absorbs a fraction λ_1/N of the trace, the noise band for the remainder shrinks; Laloux et al.'s corrected edge is $\lambda_+^{\text{eff}} = (1-\lambda_1/N)(1+\sqrt{q})^2$. Our cleaning pipeline diagonalises the de-volatilised matrix, clips bulk eigenvalues to a constant preserving the trace, and renormalises (Bun et al., 2017; Potters & Bouchaud, 2020). Whether this helps a dispersion trade is an empirical question, and Section 4 answers it more honestly than we expected.

3 Data and methodology

3.1 Data sources

All data come from WRDS. Implied volatilities are taken from the OptionMetrics IvyDB US standardised surface (OptionMetrics, 2025): the 91-day maturity, native to the surface grid, with the $+50\Delta$ call and -50Δ put averaged into an ATM-forward volatility; both legs are required, otherwise the observation is dropped. The index is SPX (`secid` 108105). Entry straddle prices come from the vendor's `impl_premium` field, which we verified to be forward-consistent: the dividend yield backed out of the two SPX legs is identical (1.04%), while a naive Black–Scholes reprice with $q = 0$ misses by $\pm 5\%$. Returns are CRSP total returns from the daily stock file, index membership is the point-in-time `dsp500list`, and zero rates come from `zerocd` (CRSP, 2025). OptionMetrics and CRSP identifiers are linked through the WRDS bridge, keeping exact (score-1) matches only; an ambiguous link is a silent source of bias, so we refuse it.

3.2 Universe construction

At each quarter-end we rank that day's actual S&P 500 members by market capitalisation and keep the top 100, with weights renormalised over the basket. $N = 100$ is a deliberate choice. It covers 63–70% of index capitalisation (against roughly 40% at $N = 30$), so the truncation error in the variance identity of eq. (1) stays small; it gives the RMT layer a well-populated spectrum (100 eigenvalues, 4,950 pairs); and coverage is excellent, with 97–100 of 100 names carrying a valid IV in every year since 1996. The sample runs 1996 to 2024, 116 quarterly rebalances, bounded below by the start of the vol surface and above by the end of our CRSP subscription. Turnover is 3–8 names per quarter. We use full market capitalisation rather than free float (the float-weight file sits behind an access wall); the difference is small for mega-caps and documented.

3.3 Realised measures and the two signal objects

Realised volatilities are rolling 63-day standard deviations of log returns, annualised by $\sqrt{252}$ to sit on the same footing as implied volatilities. The realised correlation matrix is estimated complete-case, which keeps it positive semi-definite, and collapsed to a weighted average using the same $w_i\sigma_i$ weights as eq. (2), so the implied and realised averages are the same kind of object. Timing matters here. The IV at date t prices the window $[t,\, t+63$ trading days$]$, while a trailing correlation looks backwards, so we build two series: the tradeable signal $S_t = \rho_{\text{implied},t} - \rho^{\text{trail}}_{\text{realised},t}$, which uses only information available at t, and the ex-post premium $\Pi_t = \rho_{\text{implied},t} - \rho^{\text{fwd}}_{\text{realised},t}$, measured over exactly the window the option priced, obtained by shifting the trailing series back 63 trading days. The premium establishes that the CRP exists; the signal, its ex-ante estimator through correlation persistence, is what the strategy trades. The forward series is never a trading input.

3.4 The backtest engine

At each rebalance the book is set vega-neutral to proportional volatility shocks and anchored at -100% of wealth in the index straddle, DMV's normalisation, which makes our returns directly comparable to their Table II. Positions are then frozen for the quarter: the vega drifts, and that

drift is the accepted convexity risk. Delta is hedged daily with the index alone, which is the hedge implied by the one-factor model itself and costs 1 bp of traded notional instead of a hundred single-name spread crossings. The book is marked to the surface daily (interpolation linear in total variance across maturity pillars) and settles at intrinsic value at the next rebalance, DMV's held-to-maturity convention. Costs follow their "spread paid once" rule: each entry leg pays half the relative bid–ask spread, calibrated on real quoted spreads from `opprcd` and interpolated linearly between 1996 and 2024 anchors (SPX 1.0% $\to$ 0.6%; large caps 7.0% $\to$ 1.2%; small lines 10.0% $\to$ 3.0%). Quoted spreads overstate effective ones, so net results are a conservative lower bound (calibration detail in Appendix E).

3.5 The machine-learning protocol

The timing layer is a meta-labelling veto (López de Prado, 2018): the RMT-gated strategy proposes each trade and the model may veto it. Features come in three nested sets, which form the central test of the project: A, VIX-family only (level, term slope, VIX regime); B, adding the RMT spectral features (λ_1/N, the absorption ratio of Kritzman et al. (2011), the count K of eigenvalues above the Laloux edge, $\Delta\lambda_1$, dominant-eigenvector rotation, and the Mahalanobis turbulence of Kritzman and Li (2010)); Full, adding signal-level variables (the variance risk premium, the 21-day change in implied correlation, the RMT-cleaned signal). An unsupervised regime detector, a GMM and a Gaussian HMM restricted to filtered, forward-only probabilities (Rabiner, 1989), is fitted without the label under the same walk-forward and feeds its regime probability into the candidate pool. Validation is a purged walk-forward with expanding windows, a 63-day purge and a 21-day embargo, and the full protocol (features, label, hyperparameter grids, the net-Sharpe 0.42 bar to beat) was pre-registered before any result was computed.

4 Results

4.1 The correlation risk premium exists, and has compressed

Over the 7,221 trading days from 1996 to 2024, implied correlation exceeded the correlation that subsequently realised over the priced window by +0.079 on average (Newey–West $t = 7.4$ with 63 lags), and the premium was positive on 75% of days (Figure 1). The tradeable signal, built from trailing correlation, averages +0.078 ($t = 10.6$): correlation is persistent enough that the ex-ante proxy tracks the ex-post premium closely.

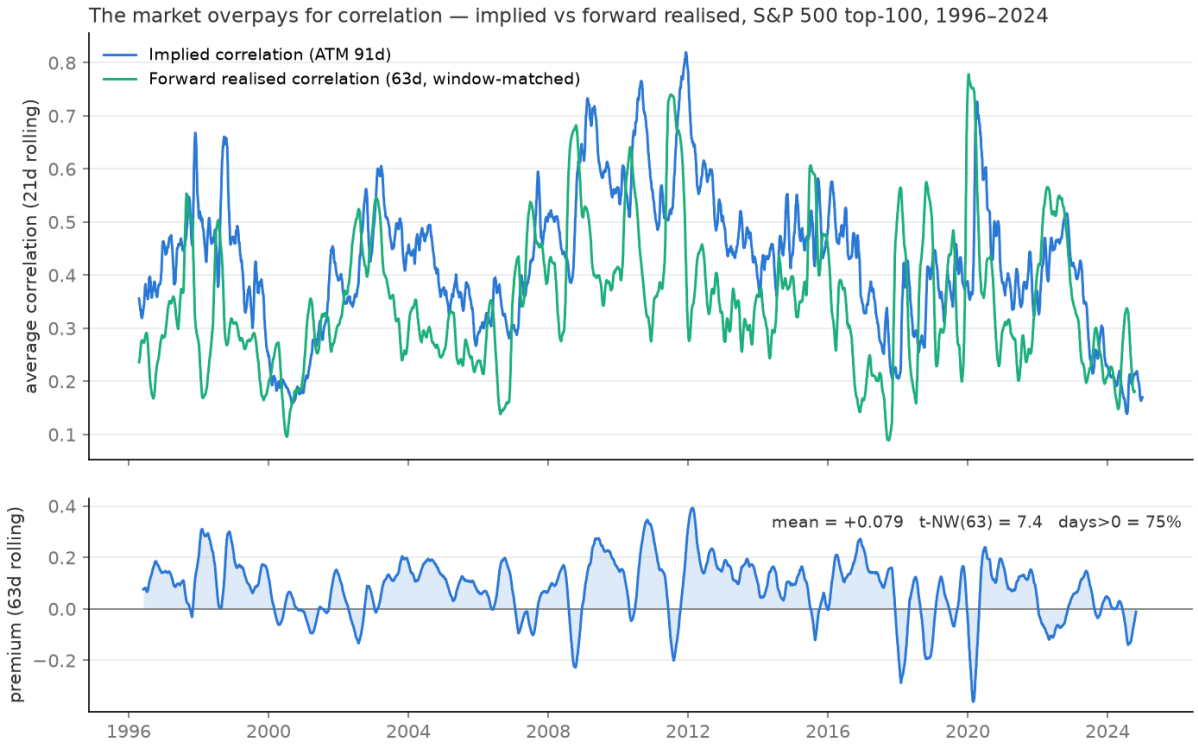

Figure 1: Implied versus forward realised correlation, S&P 500 top-100, 1996–2024, and the window-matched premium (mean +0.079, t-NW(63) = 7.4, positive on 75% of days).

The premium is not stable across time, though. Table 1 splits the sample: strong and significant through 2019, peaking at 0.135 in 2008–2012, then compressed to +0.028 ($t = 1.19$) in 2020–2024, indistinguishable from zero. We had registered this as a falsifiable limits-to-arbitrage prediction. DMV argue the premium persists because frictions stop outsiders from arbitraging it, so as spreads fell (our own cost anchors decline from 7.0% to 1.2% for large caps) the premium should shrink. It did.

4.2 The baseline trade: real, and nearly killed by frictions

The unconditional strategy v0 (trade every quarter, no signal) reproduces DMV's Table II out of the box. The vega-neutral sizing puts the component leg at $\Sigma y = +99.5\%$ of wealth against their +101.12%; the gross Sharpe is 0.77 against their 0.73; the mean gross return is +7.3% per quarter ($t = 4.11$), positive in 70% of quarters, with skew −1.29. Net of the cost grid, their Table V replicates on our 29 years: the Sharpe falls to 0.42 (theirs 0.41), the mean return to

Table 1: Window-matched correlation premium by subperiod.

Period	Mean premium	t-NW(63)	% days > 0
1996–2003	0.085	4.87	73%
2004–2007	0.078	4.06	81%
2008–2012	0.135	4.25	83%
2013–2019	0.069	3.29	75%
2020–2024	0.028	1.19	63%

+3.9% per quarter ($t = 2.24$), and the ×165 gross cumulative becomes ×2.5 (Figure 2). The premium survives costs, marginally.

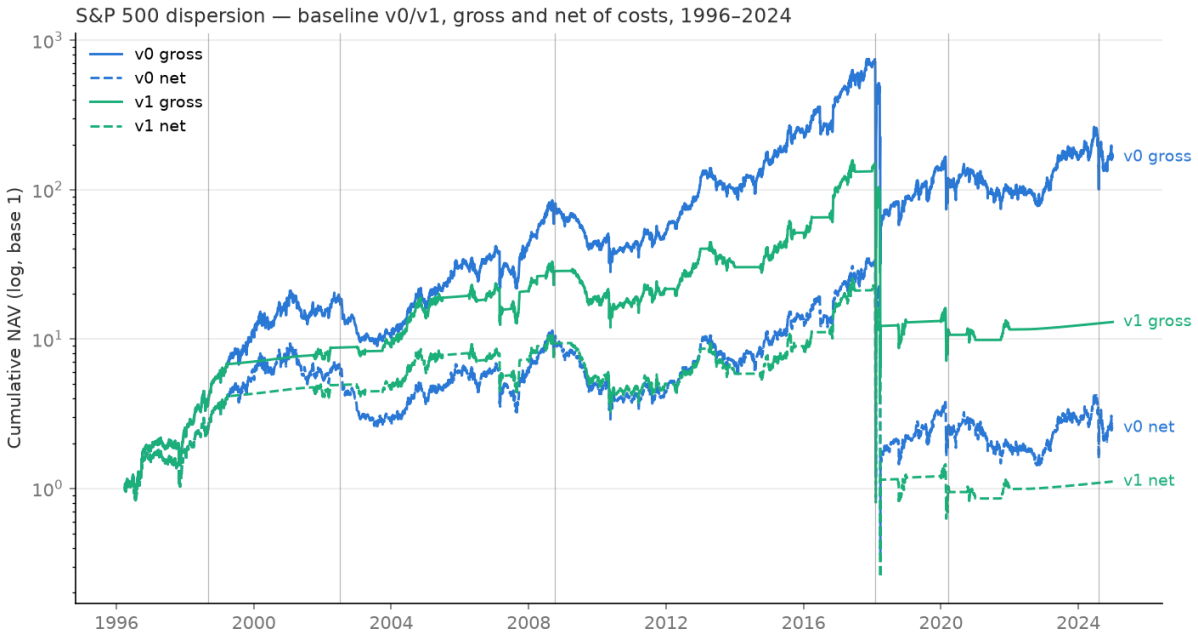

Figure 2: Cumulative NAV (log scale) of the unconditional v0 and signal-gated v1, gross and net of costs, 1996–2024.

The tail is the problem. Gross maximum drawdown is −95.8%, and one quarter, Q1-2018 (Volmageddon), loses 91% on its own. The daily ledger shows both option legs of that quarter ending positive; the loss came through the delta hedge, which bled −154% of wealth as realised correlation spiked. Attribution across the sample makes the same point: calm quarters earn on the short index leg (+10.9% on average), crisis quarters lose through the hedge (−23.0%), the realised-variance channel of eq. (3). A first conditioning attempt, v1, trades only when the signal is above its ex-ante median (58 of 115 quarters). It earns more per traded quarter (+7.9% against +7.3%) and skips 2002, Lehman and 2024, but it trades Q1-2018, because the spread measures how rich the premium is, not how dangerous the quarter will be. Net, v1 barely survives (×1.1 cumulative): its traded quarters cluster in the wide-spread era.

4.3 RMT helps, but the value is the estimation window, not the clipping

Gating on the RMT-cleaned 252-day signal (v1_rmt) transforms the risk profile: net Sharpe 0.56, skew +0.80 (from −1.33), maximum drawdown −58.5% (from −99.0%), cumulative ×12.4, and all three crashes avoided, including Volmageddon. Figure 3 shows the spectral geometry: with the Laloux-corrected edge, $K = 7$ eigenvalues escape the noise bulk (the market mode plus sector factors) where the naive edge finds 4.

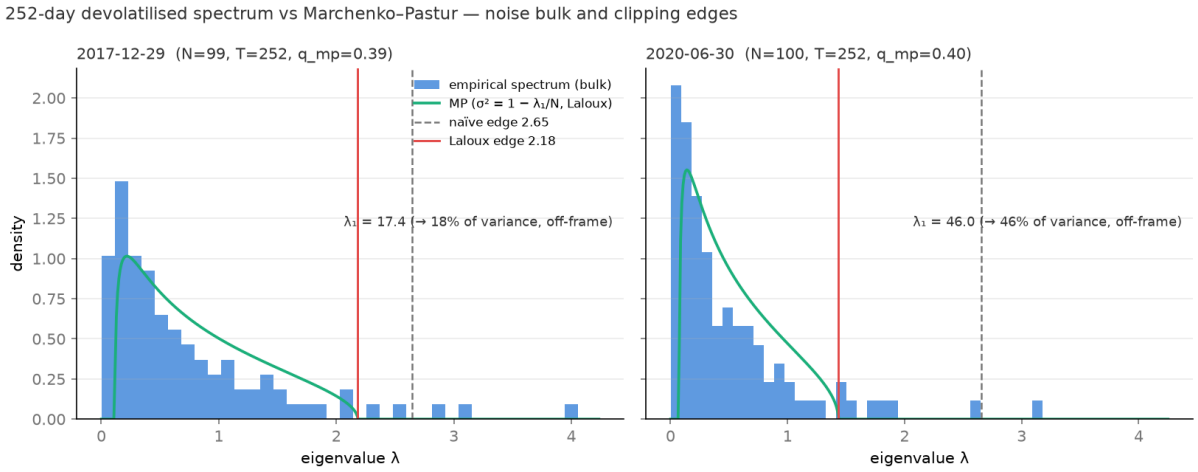

Figure 3: 252-day de-volatilised eigenvalue spectra against the Marchenko–Pastur density, with the naive and Laloux-corrected clipping edges, on a calm (2017) and a stressed (2020) date.

An ablation forces honesty here. Re-running the gate on the raw, un-clipped 252-day correlation gives virtually the same strategy: gross Sharpe 0.78, with the two gates disagreeing on 2 of 115 quarters; the clipping itself adds roughly +0.02 of Sharpe. The value comes from the slow 252-day window, which does not mistake a stressed quarter for an opportunity the way the reactive 63-day series does. We state this plainly because the fashionable claim would be the opposite.

The subperiod split (Figure 4) adds a second nuance: the gate's advantage is regime-dependent, not uniform. It shines in 2004–2007 (net 1.01 against v0's 0.67), but v0 wins in 2008–2012 (0.39 against 0.03) and in 2013–2019 (0.62 against 0.48): in the premium-rich crisis era, always trading did fine on Sharpe and gating merely cut trades. The gate's genuine, sample-wide benefit is tail shape, earned by dodging specific crashes. The spectral structure also suggests an answer to frictions: trading only the top-20 names cuts the cost from 3.68% to 3.36% per traded quarter and beats the full basket net (Sharpe 0.595 against 0.571, skew +1.14) while losing slightly gross (0.75 against 0.78; `fig_parsimonious.png`). The effect is non-monotonic ($n = 30$ and $n = 50$ fall below the full basket), so the defensible claim is that 15–20 names do at least as well net at lower cost, not that 20 is optimal.

4.4 The ML timing layer is a null; a regime gate trims the tail

The supervised meta-model cannot predict the trade's quarterly return. Out of sample, the correlation between prediction and outcome is +0.02 for the VIX-only feature set, −0.14 with

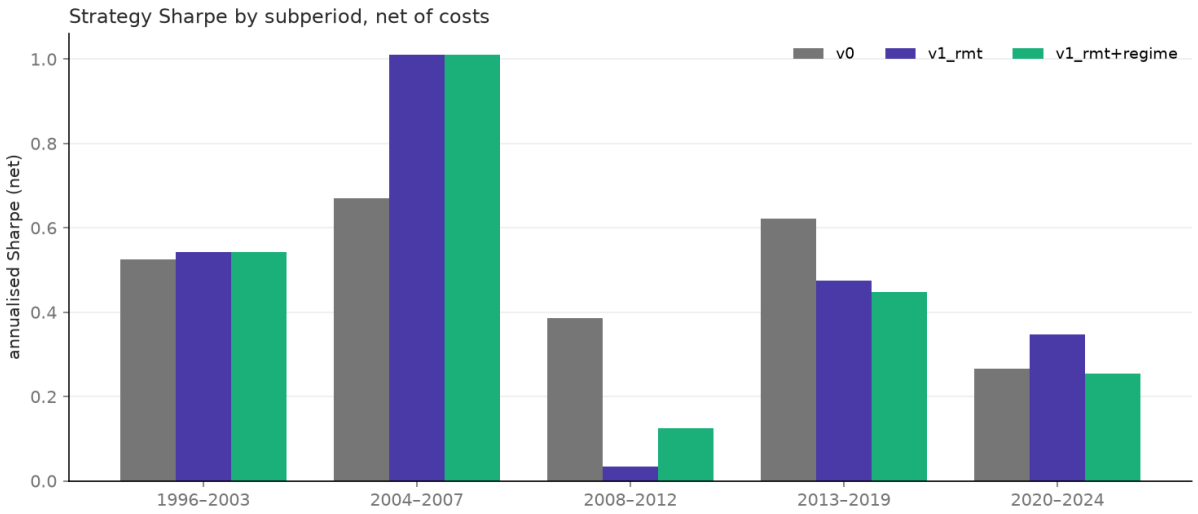

Figure 4: Net Sharpe by subperiod: v0, v1_rmt and v1_rmt+regime.

spectral features added and −0.20 for the full set (`fig_pred_scatter.png`). The central test of the project is therefore answered: spectral features do not beat the VIX for timing this trade, because neither predicts it. Crisis recall is stark: the VIX model catches 3 of 8 crisis quarters, the spectral model none (Figure 5).

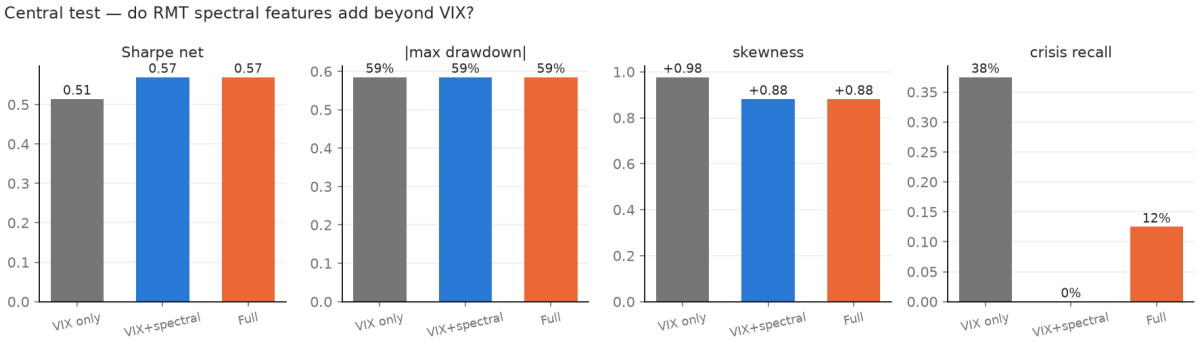

Figure 5: Central test: net Sharpe, drawdown, skew and crisis recall by feature set.

The veto built on these predictions changes nothing. It moves v1_rmt's net Sharpe from 0.56 to 0.57, which is noise, and a VIX-only veto slightly hurts (0.51); Figure 6 shows the vetoed NAV overlapping v1_rmt almost everywhere.

The null is informative rather than embarrassing. The unsupervised regime detector fires in every crisis of the sample (Figure 7), so stress detection works. A follow-up experiment shows the label was the obstacle, not the features: switching the target to the daily forward correlation spike ($\bar{\rho}_{t+63} - \bar{\rho}_t$, roughly 7,000 observations instead of 90) lifts out-of-sample correlation to +0.40. Yet vetoing on that predictable spike still lowers the Sharpe (0.53 against 0.56). Detecting stress is not predicting the loss, because stress is often exactly when the premium is richest; the danger appears to be priced.

The one overlay that earns its place is the simplest: veto the trade when the causal regime probability sits in its high tail. It leaves the net Sharpe at 0.57 but lifts net skew to +1.00 and cuts the drawdown to −52.4% (Table 2). The improvement survives every threshold choice: net

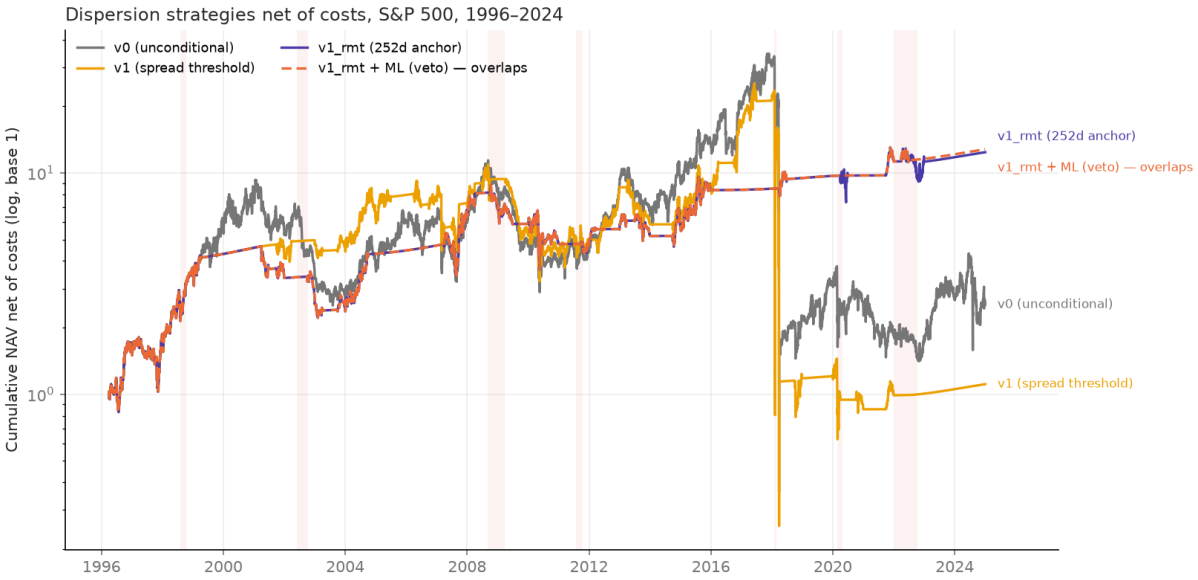

Figure 6: Net NAV of v0, v1, v1_rmt and v1_rmt+ML: the ML veto overlaps v1_rmt almost everywhere.

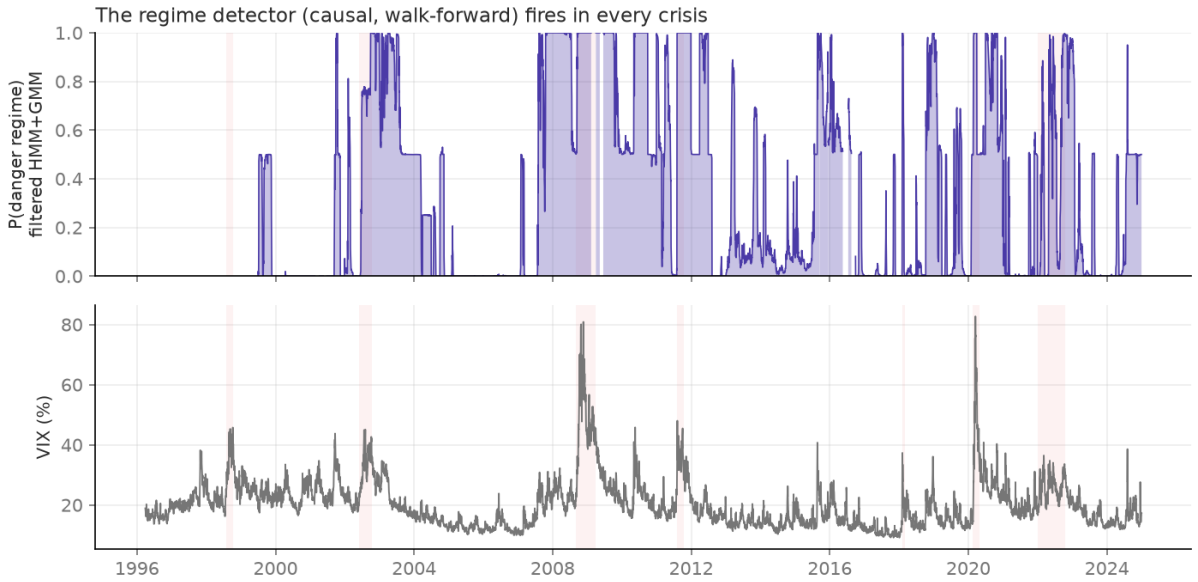

Figure 7: The causal (filtered, walk-forward) regime detector against the VIX: the danger probability rises in every crisis of the sample.

Sharpe stays within 0.46–0.67 across cost ±50% (above v0's 0.42 even at ×1.5), within 0.57–0.58 across the regime-veto quantile and 0.54–0.56 across the gate quantile, with a tighter veto lifting skew to +1.31 (Appendix C).

Last, the ATM proxy is quantifiably conservative: rebuilding the index leg's model-free implied variance from the full smile raises the index vol by 1.2 points on average (19.7% against 18.5%), on all 116 rebalances, which lifts implied correlation by +0.066 (from 0.43 to 0.50; Figure 8). Measured with MFIV, the premium would be larger; our headline number is a lower bound.

Table 2: Strategy summary, 1996–2024, 115 quarters.

Strategy	Net Sharpe	Net skew	Net maxDD	Gross Sharpe	Gross skew
v0 (unconditional)	0.42	−1.33	−99.0%	0.77	−1.29
v1_rmt	0.56	+0.80	−58.5%	0.80	+1.28
v1_rmt + regime	**0.57**	**+1.00**	**−52.4%**	0.78	+1.51

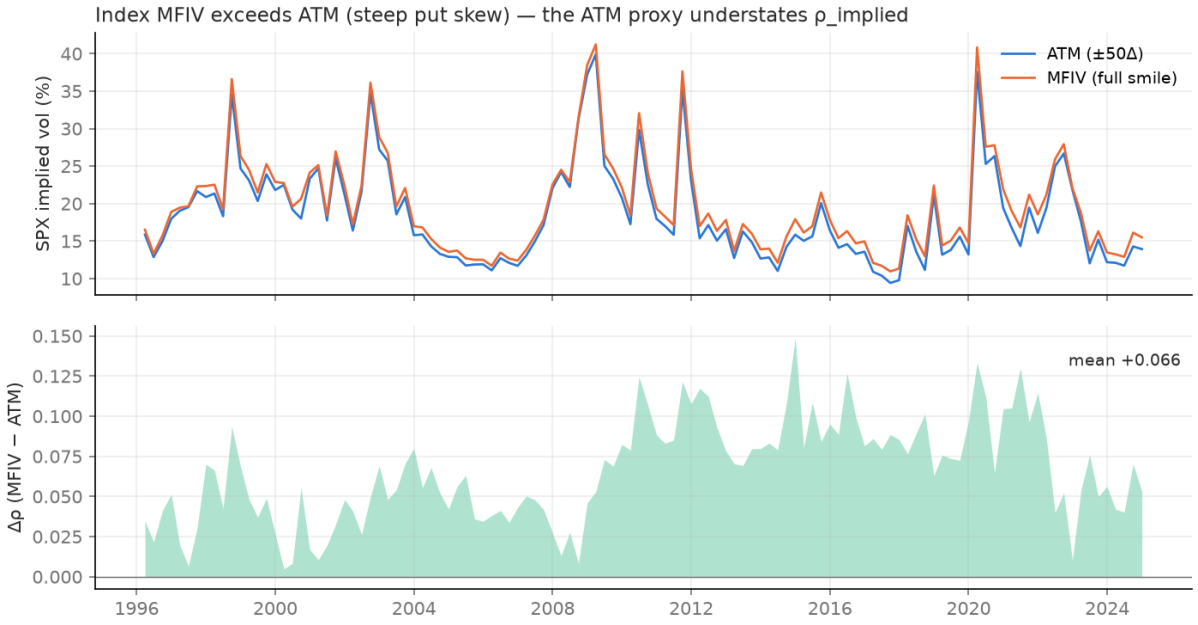

Figure 8: Index MFIV exceeds the ATM vol on every rebalance (mean gap 1.2 vol points); propagated through eq. (2), the ATM proxy understates implied correlation by 0.066 on average.

5 Limitations

Several approximations deserve plain statement. Weights use full market capitalisation rather than free float (the float-weight file sits outside our subscription) and are renormalised over the top-100 basket, so the implied-correlation measure embeds an index-versus-basket basis; it is, at least, the same basis the book itself trades. The ATM proxy for implied volatility understates the premium, and Section 4.4 quantifies the bias at +0.066 of implied correlation and signs it in our favour, so the headline number is a lower bound. Costs are a calibrated parametric grid fitted to quoted spreads, not per-trade quotes; quoted spreads overstate effective ones, which is again conservative, but the grid remains a model. The engine holds every position to maturity, with no intra-quarter unwind or stop-loss; an early-exit variant could shape the drawdowns differently, and straddles are path-dependent where variance swaps are not. Both are design choices we accept and document rather than test. Vega neutrality is exact only at inception: as spot and volatility drift, Volga and Vanna grow and the neutrality erodes (the v0 book's total vega drifted by roughly 3,600 over Q1-2018; `fig_vanna_volga.png`), part of why crisis quarters bleed. The ML null is honest but low-powered: about 90 quarterly predictions containing 12 crises cannot support strong claims, and the daily-label experiment suggests the target, not the features, was the binding constraint. The RMT scalar effect is small by construction, since clipping preserves the market mode that dominates $\bar{\rho}$. Finally, a 17-day OptionMetrics surface outage (August 2020) is accepted as missing data rather than repaired (Appendix D).

6 Conclusion

The correlation risk premium is real. Over 29 years, S&P 500 option prices charged +0.079 more correlation than subsequently realised, and a DMV-style vega-neutral dispersion book converted it into a gross Sharpe of 0.77. Almost everything else in this project qualifies that sentence. Frictions take the net Sharpe to 0.42; the premium compressed to statistical insignificance after 2020, exactly as a limits-to-arbitrage account predicts; and the fashionable layers helped less, and differently, than advertised. RMT improved the strategy through the humble 252-day estimation window, not the eigenvalue clipping. Supervised ML could not predict the trade's return at all, and even a predictable correlation spike did not help, because the danger is priced. What survives is deliberately simple: gate on the slow cleaned signal, veto in detected stress regimes, trade fewer names. Net Sharpe 0.57, positive skew, and drawdowns cut roughly in half: modest, honest and defensible.

References

Anthropic. (2026) *Claude* [Large language model]. Available from: https://claude.ai/ [Accessed on various dates between 5 May and 25 July 2026].

Bakshi, G. & Kapadia, N. (2003) Delta-hedged gains and the negative market volatility risk premium. *The Review of Financial Studies*. 16 (2), 527–566.

Bakshi, G., Kapadia, N. & Madan, D. (2003) Stock return characteristics, skew laws, and the differential pricing of individual equity options. *The Review of Financial Studies*. 16 (1), 101–143.

Bun, J., Bouchaud, J.-P. & Potters, M. (2017) Cleaning large correlation matrices: tools from random matrix theory. *Physics Reports*. 666, 1–109.

Carr, P. & Madan, D. (1998) Towards a theory of volatility trading. In: Jarrow, R. (ed.) *Volatility: new estimation techniques for pricing derivatives*. London, Risk Books. pp. 417–427.

Center for Research in Security Prices (CRSP). (2025) *CRSP US stock and index databases*. Accessed via Wharton Research Data Services, https://wrds-www.wharton.upenn.edu/ [Accessed: 20 June 2026].

Driessen, J., Maenhout, P. J. & Vilkov, G. (2009) The price of correlation risk: evidence from equity options. *The Journal of Finance*. 64 (3), 1377–1406.

Google. (2026) *Gemini* [Large language model]. Available from: https://gemini.google.com/ [Accessed on various dates between 12 May and 3 July 2026].

Kritzman, M. & Li, Y. (2010) Skulls, financial turbulence, and risk management. *Financial Analysts Journal*. 66 (5), 30–41.

Kritzman, M., Li, Y., Page, S. & Rigobon, R. (2011) Principal components as a measure of systemic risk. *The Journal of Portfolio Management*. 37 (4), 112–126.

Laloux, L., Cizeau, P., Bouchaud, J.-P. & Potters, M. (1999) Noise dressing of financial correlation matrices. *Physical Review Letters*. 83 (7), 1467–1470.

López de Prado, M. (2018) *Advances in financial machine learning*. Hoboken, NJ, John Wiley & Sons.

Marchenko, V. A. & Pastur, L. A. (1967) Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*. 1 (4), 457–483.

OpenAI. (2026) *ChatGPT* [Large language model]. Available from: https://chatgpt.com/ [Accessed on various dates between 8 May and 21 July 2026].

OptionMetrics. (2025) *IvyDB US options database*. Accessed via Wharton Research Data Services, https://wrds-www.wharton.upenn.edu/ [Accessed: 20 June 2026].

Perplexity AI. (2026) *Perplexity* [AI search engine]. Available from: https://www.perplexity.ai/ [Accessed on various dates between 19 May and 14 July 2026].

Potters, M. & Bouchaud, J.-P. (2020) *A first course in random matrix theory: for physicists, engineers and data scientists*. Cambridge, Cambridge University Press.

Rabiner, L. R. (1989) A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*. 77 (2), 257–286.

Sebbag, N. (2026) *Dispersion-Trading-Project* [source code repository]. GitHub. Available from: https://github.com/NSB22/Dispersion-Trading-Project [Accessed: 25 July 2026].

A Repository guide

The full implementation (code, notebooks, figures and tables) lives in a single repository, https://github.com/NSB22/Dispersion-Trading-Project (Sebbag, 2026). This appendix explains how it is organised and how to reproduce every number in the paper; Appendix G then lists the source code in full.

A.1 Structure

The layout separates the reproducible pipeline (`main.py`, `src/`, `tests/`) from the exploratory work (`exploration_notebooks/`) and the generated outputs (`data/`, `results/`):

```
main.py                       end-to-end pipeline: WRDS -> parquets -> backtests
pyproject.toml, uv.lock       locked dependency stack (uv, Python 3.12)
plan.md                       pre-registered ML protocol (written before results)
README.md                     full project documentation
src/dispersion/
  data/                       WRDS access, universe, IV, returns, spots, assembly
    wrds_client.py universe.py iv.py returns.py spots.py assemble.py
  signal/
    implied_corr.py           implied correlation, premium and signal
  rmt/
    cleaning.py daily.py      Marchenko-Pastur + Laloux cleaning; daily spectra
  backtest/
    marking.py engine.py      daily mark-to-surface; the dispersion engine
  ml/
    features.py regime.py metamodel.py experiments.py
  utils/
    greeks.py                 BS/Black-76 pricing, greeks, relative conversions
tests/                        pytest suite (37 tests)
exploration_notebooks/        01-10: exploration later promoted into src/
data/{raw,processed}          parquet deliverables (git-ignored, WRDS licensing)
results/{figures,tables}      every figure and table in this paper
```

The working rule throughout the project was to explore in notebooks (surface conventions in `01`, CRSP membership in `02`, the signal in `03-04`, costs in `05`, backtests in `06`, the spectrum in `07`, ML in `08-09`, robustness in `10`), then promote validated logic into `src/`. The final backtest runs from the `.py` modules only, so no result depends on notebook state.

A.2 Setup

The project uses `uv` as package manager, with Python 3.12 and every dependency pinned in `pyproject.toml`/`uv.lock`. Raw data is never committed (WRDS licensing), so reproduction requires a WRDS account with CRSP and OptionMetrics access. Setup is three steps:

```
uv sync                       # installs Python 3.12 + the locked stack
```

```
cp .env.example .env        # then set WRDS_USERNAME and PYTHONPATH=src
uv run --env-file .env python -m dispersion.data.wrds_client  # test access
```

`PYTHONPATH` must point to `src/` so the `dispersion` package is importable; the WRDS password is prompted once and can be cached in a `.pgpass` outside the repository.

A.3 Running the pipeline

A single entry point reproduces the paper end to end:

```
uv run --env-file .env python main.py            # full run (WRDS pull included)
uv run --env-file .env python main.py --no-data  # reuse the base parquets
uv run pytest                                    # test suite, 37 tests
```

The full run pulls WRDS, writes the base parquet deliverables to `data/processed/` (implied volatilities, weights, realised series, marking surface, spots, rates, returns), builds the signal and the ML features, then backtests v0, v1, v1_rmt and v1_rmt+regime, gross and net of costs. Figures and tables land in `results/`, under the exact filenames referenced in this paper, so each exhibit can be traced back to the code that produced it.

B Mathematical derivations

B.1 The equicorrelation inversion

Write $v_i = w_i\sigma_i$. Portfolio variance is $\sigma_I^2 = w^\top \Sigma w = \sum_i v_i^2 + \sum_{i\neq j} v_i v_j \rho_{ij}$. Under the equicorrelation assumption $\rho_{ij} = \bar\rho$ for all $i \neq j$, and using $\sum_{i\neq j} v_i v_j = \big(\sum_i v_i\big)^2 - \sum_i v_i^2$,

$$\sigma_I^2 \;=\; \sum_i v_i^2 \;+\; \bar\rho\Big[\Big(\sum_i v_i\Big)^2 - \sum_i v_i^2\Big],$$

which solves for $\bar\rho$ as eq. (2). The bracket is $\sum_{i\neq j} v_i v_j > 0$ automatically for $N \geq 2$ with positive weights and volatilities, so the inversion is always well defined.

B.2 The index variance-risk-premium decomposition (sketch)

Let each constituent follow $dS_i/S_i = \mu_i\,dt + \sigma_{i,t}\,dB_i$ with stochastic variance $d\sigma_{i,t}^2 = a_{i,t}\,dt + \xi_{i,t}\,dZ_i$ and stochastic pairwise correlations $\rho_{ij,t}$; the B_i drive returns and the Z_i drive variance shocks. Applying the variance identity of eq. (1) pathwise and integrating over the option's life, the index variance risk premium is the $\mathbb{Q} - \mathbb{P}$ difference of the same integrated expression. Girsanov shifts the drifts of the variance and correlation processes by their respective prices of risk; the convexity corrections (terms in $\mathbb{E}[\sigma_i\sigma_j\rho_{ij}] - \mathbb{E}[\sigma_i]\mathbb{E}[\sigma_j]\mathbb{E}[\rho_{ij}]$) take the same functional form under both measures and cancel to first order in the difference, leaving

$$\mathrm{VRP}_I \;\approx\; \sum_i w_i^2\,\mathrm{VRP}_i \;+\; \sum_{i\neq j} w_i w_j\,\sigma_i\sigma_j\big(\mathbb{E}^{\mathbb{Q}}[\bar\rho] - \mathbb{E}^{\mathbb{P}}[\bar\rho]\big).$$

Since individual variance premia are empirically close to zero (Bakshi & Kapadia, 2003), the large negative index VRP must be carried by the second term: it is compensation for correlation risk. This is a sketch; the full treatment is in DMV's appendix.

B.3 Model-free implied variance (Carr–Madan)

Any twice-differentiable payoff $H(S_T)$ admits the static replication

$$H(S_T) = H(\kappa) + H'(\kappa)(S_T - \kappa) + \int_0^\kappa H''(K)(K - S_T)^+\,dK + \int_\kappa^\infty H''(K)(S_T - K)^+\,dK.$$

Choosing the log contract $H(S) = \ln(S/S_0)$, so $H''(K) = -1/K^2$, taking $\mathbb{Q}$-expectations and using Itô's lemma on $\ln S_t$ ($d\ln S_t = \ldots - \tfrac{1}{2}\sigma_t^2\,dt$) links the expected integrated variance to a strike integral over option prices. With zero rates and $\kappa = S_0$ this collapses to the form quoted in Section 2:

$$\sigma^2_{\mathrm{MF}} \;=\; \frac{2}{T}\int_0^\infty \frac{C(K) - \max(S_0 - K,\,0)}{K^2}\,dK,$$

the $1/T$ annualising $\mathbb{E}^{\mathbb{Q}}\int_0^T \sigma_t^2\,dt$. The full version discounts and splits the integral at the forward. This is the object behind the MFIV robustness check of Section 4.4.

B.4 Relative greeks and the vega-neutral ratio

For an ATM-forward straddle, Black–Scholes gives price $V \approx 0.8\, S\sigma\sqrt{\tau}$ and vega $\nu = 2S\varphi(d_1)\sqrt{\tau} \approx 0.8\, S\sqrt{\tau}$, so the *relative* vega is

$$\frac{\nu}{V} \approx \frac{1}{\sigma}.$$

Under a proportional shock $\delta\sigma = \varepsilon\sigma$, the relative price move is $(\nu/V)\,\delta\sigma = \varepsilon$ for *every* ATM straddle regardless of the name's volatility level. Vega neutrality in wealth terms therefore sizes the component leg at (approximately) 100% of the short index notional, which is DMV's +101.12% and our +99.5%. The trap the unit tests guard against: hedging with the ratio of *per-contract* vegas instead leaves a residual exposure equal to the price ratio of the two straddles (a hedge that is silently wrong).

C Additional robustness exhibits

C.1 Meta-model predictions and feature importance

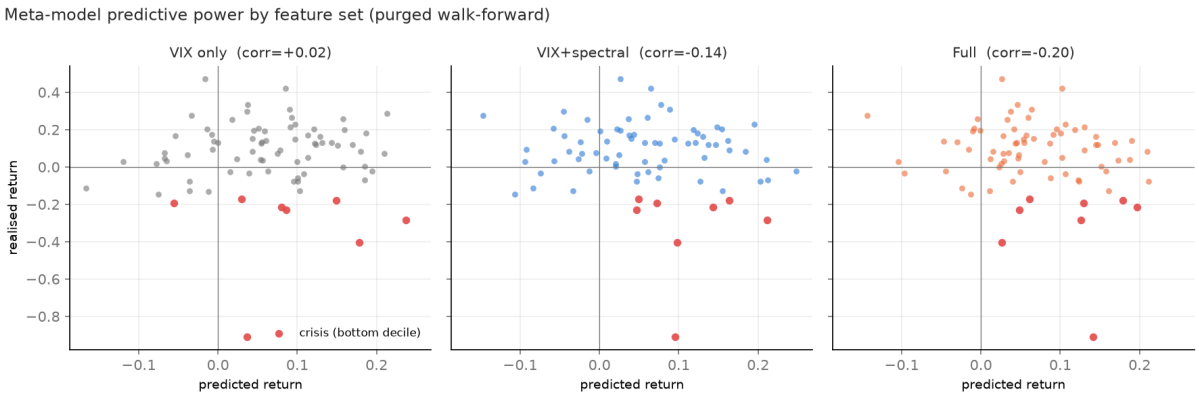

Figure 9: Out-of-sample meta-model predictions against realised quarterly trade returns, by feature set. No set produces positive predictive correlation; crisis quarters (red) are not separated.

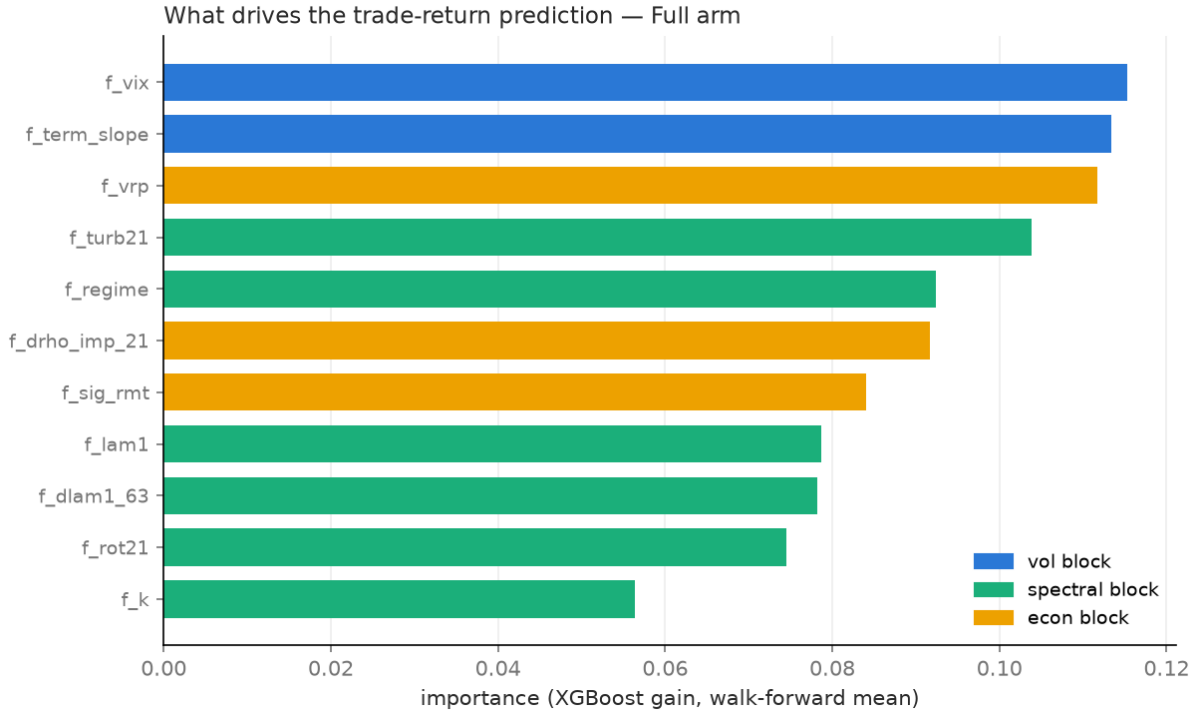

Figure 10: Meta-model feature importance. No feature family dominates, consistent with the absence of predictive power.

C.2 The parsimonious leg

The effect is non-monotonic ($n = 30$ and $n = 50$ fall below the full basket), so the defensible claim is that 15–20 names do at least as well as 100 net of costs, not that 20 is uniquely optimal.

Table 3: The central test in numbers: predictive correlation, net Sharpe with the veto applied, and crisis recall by feature set.

Feature set	corr($\hat{y}$, y)	Net Sharpe (veto)	Crisis recall
v1_rmt, no veto	—	0.56	—
A — VIX only	+0.02	0.51	3 of 8
B — VIX + spectral	−0.14	0.57	0 of 8
Full	−0.20	0.57	1 of 8

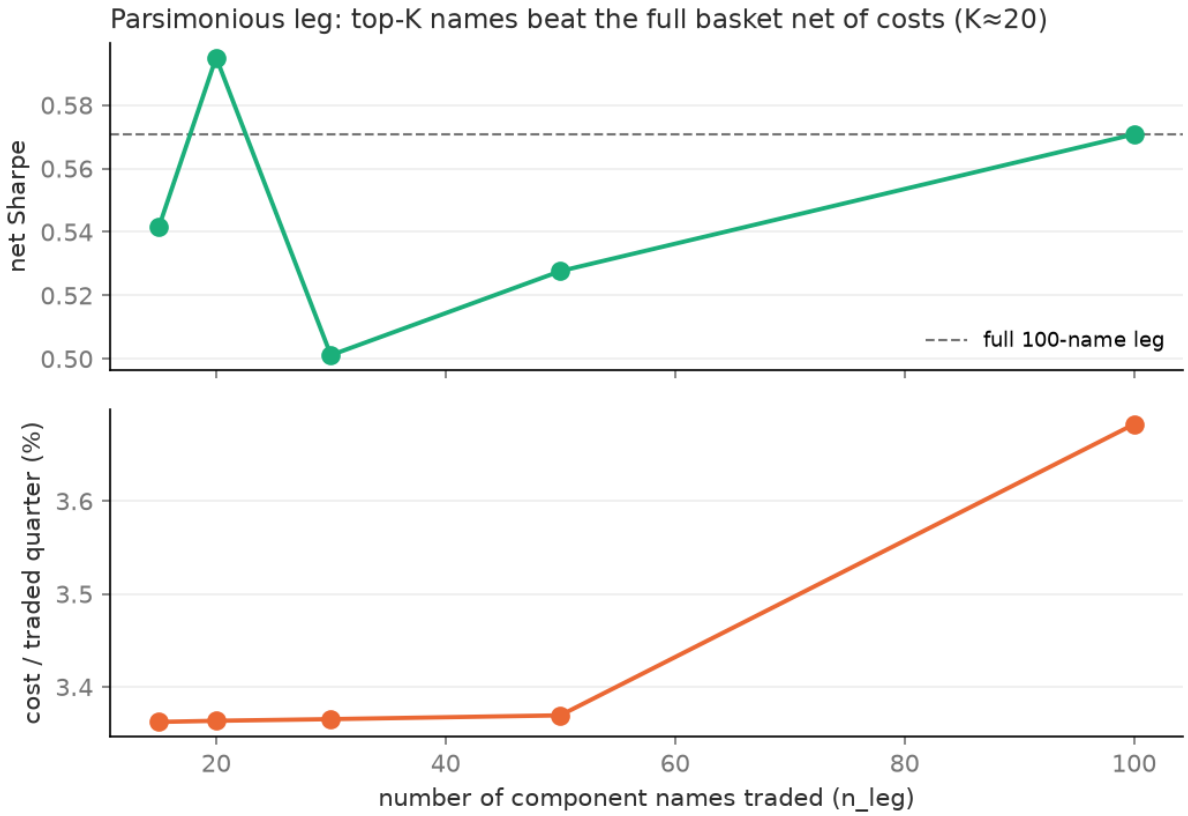

Figure 11: Net Sharpe of the dispersion book as a function of the number of constituent names traded.

Table 4: Parsimonious baskets, net of costs. The top-20 basket pays 3.36% per traded quarter against 3.68% for the full basket.

Names traded n	15	20	30	50	100
Net Sharpe	0.54	**0.60**	0.50	0.53	0.57

C.3 Threshold and cost sensitivity

Table 5: Sensitivity of v1_rmt+regime (net Sharpe) to the cost scale and the two gating thresholds. The base case is in bold.

Dimension	Setting	Net Sharpe
Cost scale	×0.5	0.67
	×1.0 (base)	**0.57**
	×1.5	0.46
Regime-veto quantile	0.50 (tighter; skew +1.31)	0.58
	0.67 (base)	**0.57**
	0.80 (looser)	0.57
Gate quantile	0.40	0.54
	0.50 (base)	**0.57**
	0.60	0.56

Net Sharpe stays above the unconditional baseline's 0.42 even at ×1.5 costs, and the tail improvement survives every threshold choice.

C.4 Convexity of the frozen book and the return distribution

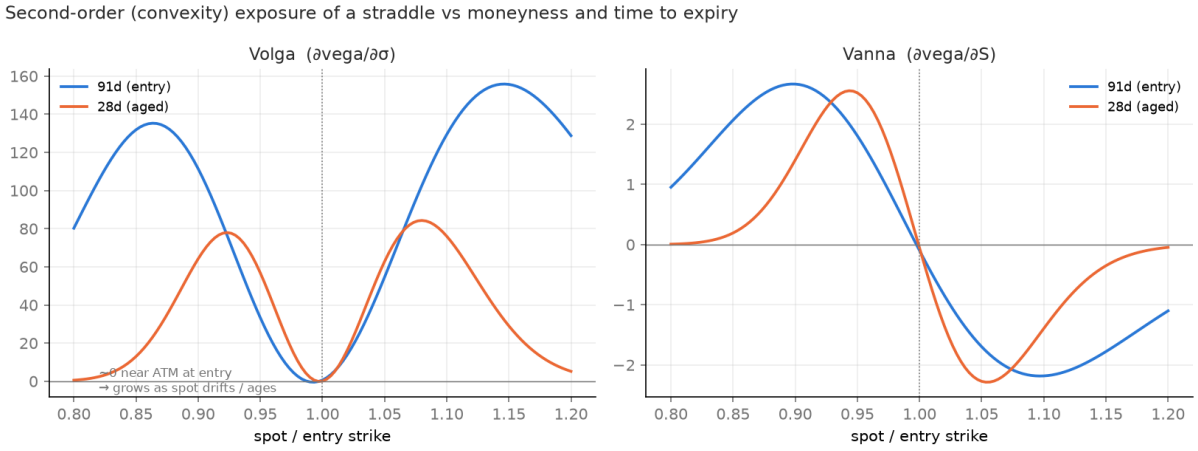

Figure 12: Volga and Vanna of the ageing straddle book. Both vanish at the second-order-neutral point at entry ($d_2 = 0$) and grow as positions age and spot drifts: the vega neutrality is a first-order property that erodes over the quarter (the v0 book's total vega drifted by roughly 3,600 over Q1-2018).

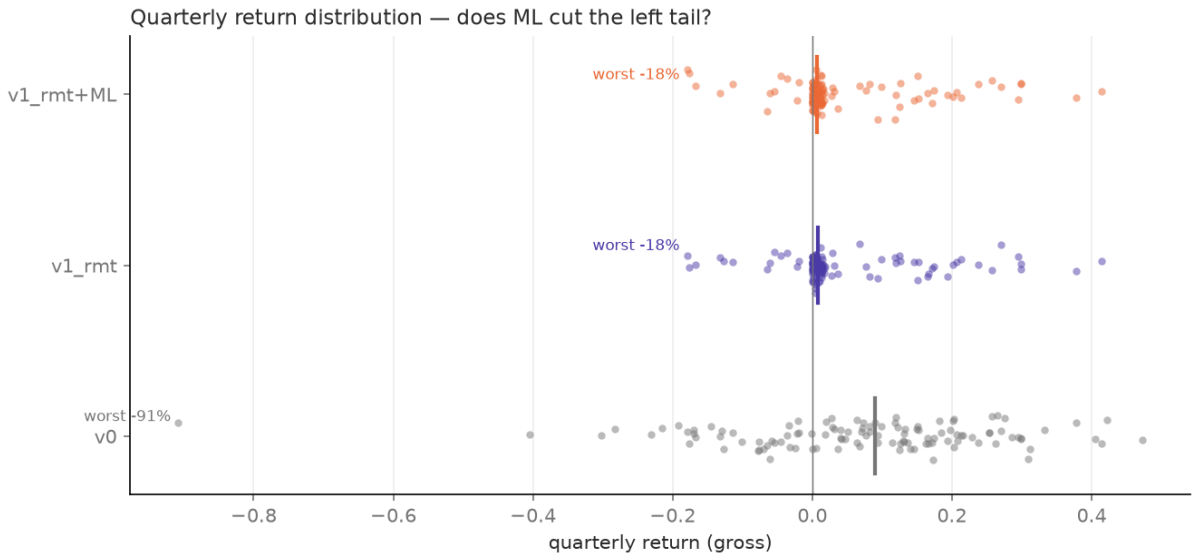

Figure 13: Quarterly gross return distributions. The gates do not raise the mean; they remove the −91% and −41% tail quarters (worst traded quarter −18%).

D Data quality and known issues

The assembled dataset passed a six-dimension quality audit (weights integrity, point-in-time discipline, coverage, value plausibility, cross-file consistency, cleaning correctness), with every flagged finding independently re-verified. The findings below are documented; none is a pipeline bug.

- **OptionMetrics surface outage, 27 July – 18 August 2020 (17 trading days).** The vendor never populated the interpolated surface for any security over this window. The dates are absent from every daily series (the master calendar is a strict intersection of OptionMetrics and CRSP days); the gap falls inside a held-position interval, and is accepted as missing rather than repaired. The only other multi-day break is the post-9/11 NYSE closure.

- **Listwise rank deficiency at 6 of 116 rebalances.** A newly-listed constituent without 63 days of history is dropped by the complete-case rule, so the correlation matrix is 99×99 on those dates while the weights assume 100 names.

- **Three names without an option surface in some quarters** (43 name-quarters of 11,600, 0.37%): the implied-correlation aggregation runs on fewer than 100 names there, with weights renormalised over the available set (coverage floor 90/100; observed minimum 91).

- **Realised-vol tails before delistings** ($\sim$0.2% of observations): a name's returns dry up a few days before it leaves the universe, so it keeps its frozen weight without a realised vol for those last days.

- **Settlement staleness for rotating names**: roughly five positions per quarter settle on the previous day's close because a name leaving the top-100 has no settlement-day price row; a one-day staleness on about 1% of the book.

E Transaction-cost calibration

The cost grid was calibrated on real quoted spreads from OptionMetrics option-price files: median relative bid–ask spread (full bid to ask, as a percentage of premium) of near-ATM options ($|\Delta| \in [0.35, 0.65]$), 60–120 days to expiry, positive open interest, pooled over three quarter-ends per era.

Table 6: Median quoted relative spreads by era (% of premium). *flagged cells discussed below.

Era	SPX	Mega-caps (rank 1–10)	Small lines (rank 90–100)
1998	0.9	6.8	9.8
2005	1.6	6.1	5.4
2012	7.6*	1.1	2.6
2020	0.7	3.0	7.9*
2024	0.6	1.2	3.1

Two cells motivate a parametric grid rather than raw per-trade quotes. The 2012 SPX cell is an end-of-day quote artefact of the pit era: quoted EOD spreads are not effective spreads (the microstructure literature puts effective at roughly 30–50% of quoted for options), so a raw extraction would inherit era-inconsistent measurement rather than economics. The 2020 small-cap cell includes the COVID quarter-end: spreads do widen in stress, a real property, but one to stress-test as a scenario rather than to hard-code from a single quarter. The frozen grid interpolates linearly between 1996 and 2024 anchors (SPX 1.0% $\to$ 0.6%; large caps 7.0% $\to$ 1.2%; small lines 10.0% $\to$ 3.0%), each entry leg pays half the relative spread, the daily index hedge pays 1 bp of traded notional, and intrinsic settlement is free. The declining anchors embed the friction-compression story of Section 4 in the cost model itself; the grid is stress-tested at $\pm 50\%$ in Appendix C.

F Statement on the use of generative AI

Generative AI tools were used during this project and are cited in the reference list (Anthropic's Claude, OpenAI's ChatGPT, Google's Gemini, and Perplexity). Their use fell into three areas: pair-programming and code review in the development environment (drafting module skeletons, reviewing implementations, writing unit tests, debugging); working through and checking the mathematical derivations of Section 2 and Appendix B against the source papers; and assisting with the proofreading, grammatical refinement, and structural clarity of the report, strictly under the author's direction.

All research design decisions, data-handling choices, empirical results and their interpretation are the author's own. AI-assisted output was reviewed, tested (the 39-test pytest suite of Appendix G) and verified against the underlying data before use, and the author takes full responsibility for the content of this report.

G Source code

This appendix lists the full implementation. The project is organised as a Python package (`src/dispersion`) with a top-level pipeline script (`main.py`) and a pytest suite (`tests/`). Code is grouped below by pipeline stage: data acquisition, signal construction, RMT cleaning, the backtest engine, the ML layer, shared utilities, then tests.

G.1 Pipeline entry point

Listing 1: main.py – end-to-end pipeline

```
"""
Run the whole project end to end: pull WRDS, build the base parquets, fit the
signals and ML features, then backtest v0 / v1 / v1_rmt / v1_rmt+regime. This is
the reproducible path from raw WRDS to the strategy P&L; notebooks only read the
parquets it writes.

    uv run --env-file .env python main.py            # full run
    uv run --env-file .env python main.py --no-data  # skip the WRDS rebuild, reuse
    base parquets
"""
import argparse
import time

import numpy as np
import pandas as pd

from dispersion.backtest.engine import exante_quantile_threshold, run_backtest
from dispersion.data.assemble import build_dataset
from dispersion.data.wrds_client import get_connection
from dispersion.ml.experiments import _high_veto, _stats
from dispersion.ml.features import build_ml_dataset
from dispersion.ml.regime import build_regime_feature
from dispersion.rmt.daily import build_rmt_daily, build_signal_rmt
from dispersion.signal.implied_corr import build_signal

PROC = "data/processed"


def stage_data():
    db = get_connection()
    out = build_dataset(db)
    db.close()
    return {k: v.shape for k, v in out.items()}


def stage_models():
    build_signal()          # signal.parquet
    build_rmt_daily()       # rmt_daily.parquet: 252d cleaned rho-bar + spectral
    features
```

```
    build_signal_rmt()      # signal_rmt.parquet: the RMT-cleaned signal
    build_ml_dataset()      # ml_dataset.parquet: features + spike label
    build_regime_feature()  # adds f_reg_gmm / f_reg_hmm / f_regime to ml_dataset


def stage_backtests():
    signal = pd.read_parquet(f"{PROC}/signal.parquet")
    signal_rmt = pd.read_parquet(f"{PROC}/signal_rmt.parquet")
    ml = pd.read_parquet(f"{PROC}/ml_dataset.parquet")
    weights = pd.read_parquet(f"{PROC}/weights.parquet")
    for d in (signal, signal_rmt, ml):
        d["date"] = pd.to_datetime(d["date"])
    weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
    rebals = sorted(weights["rebalance_date"].unique())
    base = {k: pd.read_parquet(f"{PROC}/{k}.parquet") for k in ("surface", "spots", "
    rates", "weights")}

    def bt(sig, thr, net, tag):
        r = run_backtest(data={**{k: v.copy() for k, v in base.items()}, "signal": sig.
    copy()},
                         threshold=thr, costs=({} if net else None))
        suffix = "net" if net else "gross"
        r["quarterly"].to_parquet(f"{PROC}/mainrun_{tag}_{suffix}_quarterly.parquet",
    index=False)
        return _stats(r["quarterly"], r["daily"])

    # gates: v1 on the base signal, v1_rmt on the RMT-cleaned signal, both ex-ante
    median
    thr_v1 = exante_quantile_threshold(signal, rebals, q=0.5, warmup=12)
    thr_v1rmt = exante_quantile_threshold(signal_rmt, rebals, q=0.5, warmup=12)
    reg_at = {pd.Timestamp(r): float(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].
    iloc[0])
              for r in rebals if (ml["date"] == pd.Timestamp(r)).any()
              and np.isfinite(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].iloc
    [0])}
    thr_reg = thr_v1rmt.copy()
    for d in _high_veto(reg_at, q=0.67):
        thr_reg[pd.Timestamp(d)] = 1e9

    res = {}
    res["v0_gross"] = bt(signal, None, False, "v0")
    res["v0_net"] = bt(signal, None, True, "v0")
    res["v1_net"] = bt(signal, thr_v1, True, "v1")
    res["v1rmt_gross"] = bt(signal_rmt, thr_v1rmt, False, "v1rmt")
    res["v1rmt_net"] = bt(signal_rmt, thr_v1rmt, True, "v1rmt")
    res["v1rmt_regime_gross"] = bt(signal_rmt, thr_reg, False, "v1rmt_regime")
    res["v1rmt_regime_net"] = bt(signal_rmt, thr_reg, True, "v1rmt_regime")
    return res


def main():
```

```
    ap = argparse.ArgumentParser()
    ap.add_argument("--no-data", action="store_true", help="skip the WRDS rebuild")
    args = ap.parse_args()

    t0 = time.time()
    if not args.no_data:
        print("[1/3] data - building base parquets from WRDS ...", flush=True)
        shapes = stage_data()
        print("      " + " | ".join(f"{k} {v}" for k, v in shapes.items()), flush=True
    )
    else:
        print("[1/3] data - skipped (reusing base parquets)", flush=True)

    print("[2/3] models - signal, RMT, ML features, regime ...", flush=True)
    stage_models()

    print("[3/3] backtests - v0 / v1 / v1_rmt / v1_rmt+regime ...", flush=True)
    res = stage_backtests()
    print(f"\n{'strategy':22s} {'Sharpe':>7s} {'skew':>6s} {'maxDD':>7s} {'cumul':>7s}
    ", flush=True)
    for k, s in res.items():
        print(f"{k:22s} {s['sharpe']:+7.2f} {s['skew']:+6.2f} {s['maxDD']:>7.1%} {s['
    cumul']:>6.1f}x", flush=True)
    print(f"\ndone in {time.time() - t0:.0f}s", flush=True)


if __name__ == "__main__":
    main()
```

G.2 Data acquisition (src/dispersion/data/)

Listing 2: src/dispersion/data/wrds_client.py

```
"""
WRDS connection plumbing.

Usage:
    from dispersion.data.wrds_client import get_connection
    db = get_connection()
"""
import os
from dotenv import load_dotenv
import wrds

load_dotenv()


def get_connection() -> wrds.Connection:
    """Open a WRDS connection (username read from .env)."""
    username = os.getenv("WRDS_USERNAME")
    if not username:
```

```
        raise RuntimeError("WRDS_USERNAME missing. Add it to the .env file.")
    return wrds.Connection(wrds_username=username)


def test_connection() -> None:
    """Smoke test: check access to crsp, optionm, comp libraries."""
    db = get_connection()
    print("WRDS connection established.")
    libs = db.list_libraries()
    for lib in ["crsp", "optionm", "comp"]:
        print(f"  - {lib}: {'OK' if lib in libs else 'ABSENT'}")
    db.close()


if __name__ == "__main__":
    test_connection()
```

Listing 3: src/dispersion/data/universe.py

```
"""
Point-in-time S&P 500 universe construction.

Usage:
    from dispersion.data.universe import get_universe
    df = get_universe(db, "2020-03-31")
"""
import pandas as pd
import wrds


def get_universe(db: wrds.Connection, date: str, n: int = 100) -> pd.DataFrame:
    """Top-N S&P 500 constituents by market cap on `date`, point-in-time.

    Only names with a score-1 OptionMetrics link are kept, so each permno maps
    to exactly one secid.

    Columns: permno, secid, market_cap (USD on `date`), weight (cap weight
    renormalised within the top-N), rnk (rank by cap, 1 = largest).
    """
    query = f"""
    WITH members AS (
        -- point-in-time S&P 500 membership
        SELECT permno
        FROM crsp.dsp500list
        WHERE start <= '{date}' AND ending >= '{date}'
    ),
    capi AS (
        -- market cap = abs(prc) * shrout * 1000
        -- CRSP prc can be negative (it's a bid/ask midpoint)
        SELECT d.permno,
               ABS(d.prc) * d.shrout * 1000 AS market_cap
```

```
        FROM crsp.dsf d
        JOIN members m ON d.permno = m.permno
        WHERE d.date = '{date}'
          AND d.prc IS NOT NULL
          AND d.shrout IS NOT NULL
    ),
    ranked AS (
        -- ROW_NUMBER not RANK: with RANK a cap tie at rank N would let in >N rows.
        -- permno breaks ties deterministically.
        SELECT permno, market_cap,
               ROW_NUMBER() OVER (ORDER BY market_cap DESC, permno) AS rnk
        FROM capi
    ),
    top_n AS (
        SELECT * FROM ranked WHERE rnk <= {n}
    ),
    link AS (
        -- score=1 only. A permno can have several overlapping score-1 links;
        -- keep the latest edate so secid is unique per permno.
        SELECT DISTINCT ON (permno) permno, secid
        FROM wrdsapps_link_crsp_optionm.opcrsphist
        WHERE score = 1 AND sdate <= '{date}' AND edate >= '{date}'
        ORDER BY permno, edate DESC
    )
    SELECT t.permno,
           l.secid,
           t.market_cap,
           t.market_cap / SUM(t.market_cap) OVER () AS weight,
           t.rnk
    FROM top_n t
    LEFT JOIN link l ON t.permno = l.permno
    ORDER BY t.rnk
    """
    return db.raw_sql(query)
```

Listing 4: src/dispersion/data/iv.py

```
"""
Implied-volatility extraction from OptionMetrics `vsurfd` (volatility surface).

Usage:
    from dispersion.data.iv import get_iv, get_surface
    df = get_iv(db, secids=[108105, 101594], date_start="2020-01-01", date_end="
    2020-12-31")
"""
import numpy as np
import pandas as pd
import wrds


def get_iv(
```

```python
    db: wrds.Connection,
    secids: list[int],
    date_start: str,
    date_end: str,
    days: int = 91,
) -> pd.DataFrame:
    """ATM (delta ±50) implied vol at a fixed maturity for the given securities.

    Pure extraction -- no rebalancing or membership logic. Works the same for the
    index (secid 108105) and its constituents.

    One row per (secid, date): secid, date, iv_call_50 (delta +50), iv_put_50
    (delta -50), iv_atm (mean of the two legs, NaN if either is missing).
    """
    secid_list = ",".join(str(int(s)) for s in secids)
    y0, y1 = int(date_start[:4]), int(date_end[:4])

    frames = []
    for year in range(y0, y1 + 1):
        # clip the window to this year's slice of vsurfd
        lo = max(date_start, f"{year}-01-01")
        hi = min(date_end, f"{year}-12-31")
        q = f"""
        SELECT secid, date, delta, impl_volatility
        FROM optionm.vsurfd{year}
        WHERE secid IN ({secid_list})
          AND days = {days}
          AND delta IN (50, -50)
          AND date BETWEEN '{lo}' AND '{hi}'
          AND impl_volatility IS NOT NULL
          AND impl_volatility > 0
        """
        frames.append(db.raw_sql(q))

    long = pd.concat(frames, ignore_index=True)
    if long.empty:
        return pd.DataFrame(columns=["secid", "date", "iv_call_50", "iv_put_50", "
    iv_atm"])

    # pivot the two legs (delta +50 -> call, -50 -> put) into columns
    wide = (
        long.pivot_table(index=["secid", "date"], columns="delta", values="
    impl_volatility")
        .rename(columns={50.0: "iv_call_50", -50.0: "iv_put_50"})
        .reset_index()
    )
    wide.columns.name = None
    # make sure both leg columns exist even if a delta never showed up.
    # use np.nan, not pd.NA -- an object column makes mean(skipna=False) raise
    for col in ("iv_call_50", "iv_put_50"):
        if col not in wide.columns:
```

```
            wide[col] = np.nan

    # need both legs or it's NaN
    wide["iv_atm"] = wide[["iv_call_50", "iv_put_50"]].mean(axis=1, skipna=False)

    return wide[["secid", "date", "iv_call_50", "iv_put_50", "iv_atm"]].sort_values(
        ["secid", "date"]
    ).reset_index(drop=True)


def get_surface(
    db: wrds.Connection,
    secids: list[int],
    date_start: str,
    date_end: str,
    days: tuple[int, ...] = (10, 30, 60, 91),
) -> pd.DataFrame:
    """Standardised surface (both ±50Δ legs) at several maturity pillars.

    Marking-grade data: a 91d position ages into shorter residual maturities, so
    we pull the sub-91 pillars too. Raw vendor values, no cleaning here.

    One row per (secid, date, days, cp_flag): secid, date, days, cp_flag
    ('C'/'P'), iv, premium, strike.
    """
    secid_list = ",".join(str(int(s)) for s in secids)
    days_list = ",".join(str(int(d)) for d in days)
    y0, y1 = int(date_start[:4]), int(date_end[:4])

    frames = []
    for year in range(y0, y1 + 1):
        lo = max(date_start, f"{year}-01-01")
        hi = min(date_end, f"{year}-12-31")
        q = f"""
        SELECT secid, date, days, cp_flag,
               impl_volatility AS iv, impl_premium AS premium, impl_strike AS strike
        FROM optionm.vsurfd{year}
        WHERE secid IN ({secid_list})
          AND days IN ({days_list})
          AND delta IN (50, -50)
          AND date BETWEEN '{lo}' AND '{hi}'
          AND impl_volatility IS NOT NULL
          AND impl_volatility > 0
        """
        frames.append(db.raw_sql(q))

    surf = pd.concat(frames, ignore_index=True)
    if surf.empty:
        return pd.DataFrame(columns=["secid", "date", "days", "cp_flag", "iv", "
    premium", "strike"])

```

```
    surf["cp_flag"] = surf["cp_flag"].str.strip()
    surf["days"] = surf["days"].astype(int)
    return surf.sort_values(["secid", "date", "days", "cp_flag"]).reset_index(drop=
    True)
```

Listing 5: src/dispersion/data/returns.py

```
"""
Returns extraction from CRSP `dsf` and realized-volatility computation.

Usage:
    from dispersion.data.returns import get_returns, realized_vol
    px = get_returns(db, permnos=[14593, 10107], date_start="2019-01-01", date_end="
    2020-12-31")
    vol = realized_vol(px)          # {21: df, 63: df}, annualised
"""
import numpy as np
import pandas as pd
import wrds

TRADING_DAYS = 252


def get_returns(
    db: wrds.Connection,
    permnos: list[int],
    date_start: str,
    date_end: str,
) -> pd.DataFrame:
    """Daily total returns (dividend- and split-adjusted) for the given permnos.

    CRSP `ret` is a holding-period return, so dividends and splits are already
    baked in -- no manual price adjustment.

    Wide DataFrame: index = date, columns = permno, values = daily return; NaN
    where a security did not trade.
    """
    permno_list = ",".join(str(int(p)) for p in permnos)
    q = f"""
    SELECT permno, date, ret
    FROM crsp.dsf
    WHERE permno IN ({permno_list})
      AND date BETWEEN '{date_start}' AND '{date_end}'
      AND ret IS NOT NULL
    """
    long = db.raw_sql(q)
    if long.empty:
        return pd.DataFrame()

    wide = long.pivot(index="date", columns="permno", values="ret").sort_index()
    wide.columns.name = None
```

```
    return wide


def realized_vol(
    returns: pd.DataFrame,
    windows: tuple[tuple[int, int], ...] = ((21, 17), (63, 50)),
) -> dict[int, pd.DataFrame]:
    """Rolling annualised realized vol per security, one DataFrame per window.

    Uses log-returns; a window is NaN until it has at least `min_obs`
    observations. `windows` is a tuple of (length, min_obs) pairs in trading days.
    Returns dict {window_length: wide DataFrame (date x permno)}.
    """
    log_ret = np.log1p(returns)
    out = {}
    for window, min_obs in windows:
        vol = log_ret.rolling(window=window, min_periods=min_obs).std(ddof=1)
        out[window] = vol * np.sqrt(TRADING_DAYS)
    return out


def realized_corr_matrix(
    returns: pd.DataFrame,
    end_date: str,
    window: int = 63,
    min_obs: int = 50,
    method: str = "listwise",
) -> pd.DataFrame | None:
    """Realized correlation matrix over the `window` days ending at `end_date`.

    `method` is 'listwise' (complete-case rows -> PSD) or 'pairwise'. Returns an
    N x N DataFrame (index/columns = permno), or None if there isn't enough data.
    """
    log_ret = np.log1p(returns)
    win = log_ret.loc[:end_date].tail(window)
    # drop names with too few obs in the window
    win = win.loc[:, win.notna().sum() >= min_obs]
    if win.shape[1] < 2:
        return None

    if method == "listwise":
        win = win.dropna(axis=0, how="any")  # keep only complete rows so it stays PSD
        if len(win) < min_obs:
            return None
        corr = win.corr()
    elif method == "pairwise":
        corr = win.corr(min_periods=min_obs)
    else:
        raise ValueError(f"method must be 'listwise' or 'pairwise', got {method!r}")
    return corr

```

```python

def average_correlation(
    corr: pd.DataFrame,
    weights: pd.Series,
    vols: pd.Series,
    method: str = "weighted",
) -> float:
    """Collapse a correlation matrix into a single average correlation rho-bar.

    'weighted' (default) is the formula-consistent average from the index
    variance decomposition (matches the implied-correlation definition):
        rho_bar = (v' R v - sum v_i^2) / ((sum v_i)^2 - sum v_i^2),  v_i = w_i * sigma_i
    'equal' is the plain mean of off-diagonal pairwise correlations.

    `weights` are the frozen rebalance cap weights; `vols` are realized sigma_i on
    the window's end date. Returns NaN if not computable.
    """
    permnos = corr.index
    w = weights.reindex(permnos)
    s = vols.reindex(permnos)
    valid = w.notna() & s.notna()
    permnos = permnos[valid]
    if len(permnos) < 2:
        return float("nan")

    R = corr.loc[permnos, permnos].to_numpy()

    if method == "weighted":
        v = (w[permnos] * s[permnos]).to_numpy()
    elif method == "equal":
        v = np.ones(len(permnos))
    else:
        raise ValueError(f"method must be 'weighted' or 'equal', got {method!r}")

    sum_sq = np.sum(v**2)
    num = v @ R @ v - sum_sq          # off-diagonal weighted sum of rho_ij
    den = v.sum() ** 2 - sum_sq       # off-diagonal weighted sum (rho_ij = 1)
    return float(num / den) if den != 0 else float("nan")
```

Listing 6: src/dispersion/data/spots.py

```python
"""Daily spot prices from OptionMetrics `secprd`.

`secprd` is the price series OptionMetrics builds the surface from, so its closes
share the split convention of `impl_strike`. Marking and intrinsic settlement
need S and K on the same convention (|S - K|); mid-quarter splits are handled
with `cfadj`.

Usage:
    from dispersion.data.spots import get_spots
```

```
    df = get_spots(db, [108105], "2020-01-01", "2020-12-31")
"""
import pandas as pd
import wrds


def get_spots(
    db: wrds.Connection,
    secids: list[int],
    date_start: str,
    date_end: str,
) -> pd.DataFrame:
    """Close price + cumulative split-adjustment factor per (secid, date)."""
    secid_list = ",".join(str(int(s)) for s in secids)
    y0, y1 = int(date_start[:4]), int(date_end[:4])

    frames = []
    for year in range(y0, y1 + 1):
        lo = max(date_start, f"{year}-01-01")
        hi = min(date_end, f"{year}-12-31")
        q = f"""
        SELECT secid, date, close, cfadj
        FROM optionm.secprd{year}
        WHERE secid IN ({secid_list})
          AND date BETWEEN '{lo}' AND '{hi}'
          AND close IS NOT NULL
        """
        frames.append(db.raw_sql(q))

    spots = pd.concat(frames, ignore_index=True)
    if spots.empty:
        return pd.DataFrame(columns=["secid", "date", "close", "cfadj"])

    spots["close"] = spots["close"].abs()  # close can be signed (midpoint convention);
     take abs
    return spots.sort_values(["secid", "date"]).reset_index(drop=True)
```

Listing 7: src/dispersion/data/assemble.py

```
"""Build the clean backtest dataset: pull each piece, align to one calendar,
apply the cleaning rules, and write the parquets to data/processed/.

Usage:
    from dispersion.data.assemble import build_dataset
    out = build_dataset(db, "1996-01-01", "2024-12-31")
"""
import os
import numpy as np
import pandas as pd
import wrds

```

```python
from .universe import get_universe
from .iv import get_iv, get_surface
from .spots import get_spots
from .returns import (
    get_returns,
    realized_vol,
    realized_corr_matrix,
    average_correlation,
)

SPX_SECID = 108105


# ---------------------------------------------------------------------------- #
# Calendar helpers
# ---------------------------------------------------------------------------- #
def _master_calendar(db, iv_index: pd.DataFrame, date_start: str, date_end: str) -> pd.DatetimeIndex:
    """Strict intersection of OptionMetrics (SPX IV) and CRSP (dsf) trading days."""
    iv_dates = pd.to_datetime(iv_index["date"]).unique()
    dsf = db.raw_sql(
        f"SELECT DISTINCT date FROM crsp.dsf WHERE date BETWEEN '{date_start}' AND '{date_end}'"
    )
    dsf_dates = pd.to_datetime(dsf["date"]).unique()
    common = pd.DatetimeIndex(sorted(set(iv_dates) & set(dsf_dates)))
    return common


def _rebalance_dates(calendar: pd.DatetimeIndex) -> list[pd.Timestamp]:
    """Last trading day of each quarter present in the master calendar."""
    s = pd.Series(calendar, index=calendar)
    grp = s.groupby([calendar.year, calendar.quarter]).max()
    return [pd.Timestamp(x) for x in grp.values]


# ---------------------------------------------------------------------------- #
# Main
# ---------------------------------------------------------------------------- #
def build_dataset(
    db: wrds.Connection,
    date_start: str = "1996-01-01",
    date_end: str = "2024-12-31",
    n: int = 100,
    iv_days: int = 91,
    corr_window: int = 63,
    corr_min_obs: int = 50,
    corr_method: str = "listwise",
    rho_method: str = "weighted",
    ffill_limit: int = 3,
    iv_pct: tuple[float, float] = (0.001, 0.999),
```

```
    surface_days: tuple[int, ...] = (10, 30, 60, 91),
    returns_start: str = "1995-01-01",
    out_dir: str | None = "data/processed",
) -> dict[str, pd.DataFrame]:
    """Build and optionally save the clean backtest dataset; returns a dict of DataFrames.

    Writes iv_index, iv_components, weights, realized_vol, realized_corr,
    corr_matrices (cleaned, signal-grade) plus surface, rates, returns (marking/
    RMT grade, stored raw). `returns_start` predates `date_start` so the 252-day
    RMT windows have warm history from the first rebalance.
    """

    # --- 1. SPX IV (the index leg) + master calendar + rebalance schedule -----
    iv_index = get_iv(db, [SPX_SECID], date_start, date_end, days=iv_days)
    iv_index["date"] = pd.to_datetime(iv_index["date"])
    calendar = _master_calendar(db, iv_index, date_start, date_end)
    rebals = _rebalance_dates(calendar)

    iv_index = iv_index[iv_index["date"].isin(calendar)].drop(columns="secid")
    # bounded ffill on the index series; no outlier clipping on the index
    iv_index = (
        iv_index.set_index("date").reindex(calendar).ffill(limit=ffill_limit).reset_index(names="date")
    )

    # --- 2. Per-quarter loop ------------------------------------------------- #
    weights_rows, iv_comp_parts, vol_parts, rho_rows, corr_mat_parts = [], [], [], [], []
    surface_parts, spot_parts = [], []

    # active window per rebalance; the cleaning step reuses this too
    active_by_reb: dict[pd.Timestamp, pd.DatetimeIndex] = {}
    for i, reb in enumerate(rebals):
        nxt = rebals[i + 1] if i + 1 < len(rebals) else calendar[-1] + pd.Timedelta(days=1)
        active_by_reb[reb] = calendar[(calendar >= reb) & (calendar < nxt)]

    for reb in rebals:
        active = active_by_reb[reb]
        if len(active) == 0:
            continue

        univ = get_universe(db, str(reb.date()), n=n)
        univ["permno"] = univ["permno"].astype(int)
        univ["secid"] = univ["secid"].astype("Int64")
        w = univ.set_index("permno")["weight"]
        permnos = univ["permno"].tolist()
        secids = univ["secid"].dropna().astype(int).tolist()
        sec2permno = dict(zip(univ["secid"].astype("Int64"), univ["permno"]))

```

```
        # weights (frozen) at this rebalance
        wr = univ[["permno", "secid", "weight", "market_cap", "rnk"]].copy()
        wr.insert(0, "rebalance_date", reb)
        weights_rows.append(wr)

        # component IV over the active window (point-in-time secids)
        ivc = get_iv(db, secids, str(active[0].date()), str(active[-1].date()), days=
    iv_days)
        if not ivc.empty:
            ivc["date"] = pd.to_datetime(ivc["date"])
            ivc["permno"] = ivc["secid"].astype("Int64").map(sec2permno).astype("Int64
    ")
            ivc["rebalance_date"] = reb
            iv_comp_parts.append(ivc[["rebalance_date", "date", "permno", "secid", "
    iv_atm"]])

        # multi-pillar surface (marking-grade, stored raw) for this quarter's names
        surf = get_surface(db, secids, str(active[0].date()), str(active[-1].date()),
    days=surface_days)
        if not surf.empty:
            surf["date"] = pd.to_datetime(surf["date"])
            surf = surf[surf["date"].isin(active)]
            surf["permno"] = surf["secid"].astype("Int64").map(sec2permno).astype("
    Int64")
            surf.insert(0, "rebalance_date", reb)
            surface_parts.append(surf)

        # daily spots (same split convention as the strikes)
        sp = get_spots(db, secids, str(active[0].date()), str(active[-1].date()))
        if not sp.empty:
            sp["date"] = pd.to_datetime(sp["date"])
            sp = sp[sp["date"].isin(active)]
            sp["permno"] = sp["secid"].astype("Int64").map(sec2permno).astype("Int64")
            sp.insert(0, "rebalance_date", reb)
            spot_parts.append(sp)

        # returns with a trailing buffer for the rolling windows
        buf_start = (active[0] - pd.Timedelta(days=corr_window * 2 + 20)).date()
        ret = get_returns(db, permnos, str(buf_start), str(active[-1].date()))
        if ret.empty:
            continue
        ret.index = pd.to_datetime(ret.index)
        vols = realized_vol(ret)

        # realized vol (long) on active dates
        v21 = vols[21].reindex(active)
        v63 = vols[63].reindex(active)
        vlong = (
            v21.reset_index(names="date").melt("date", var_name="permno", value_name="
    vol_21")
            .merge(
```

```
            v63.reset_index(names="date").melt("date", var_name="permno",
value_name="vol_63"),
            on=["date", "permno"],
        )
    )
    vol_parts.append(vlong)

    # daily rho-bar + full matrix at the rebalance date
    for d in active:
        C = realized_corr_matrix(ret, str(d.date()), corr_window, corr_min_obs,
corr_method)
        if C is None:
            rho_rows.append((d, np.nan, np.nan))
            continue
        if d not in vols[63].index:  # skip rather than fall back to a stale vol
row
            rho_rows.append((d, np.nan, np.nan))
            continue
        vols_d = vols[63].loc[d]
        rho_rows.append(
            (
                d,
                average_correlation(C, w, vols_d, rho_method),
                average_correlation(C, w, vols_d, "equal"),
            )
        )
        if d == reb:  # store the full matrix only at rebalances
            m = C.stack().rename("corr").reset_index()
            m.columns = ["permno_i", "permno_j", "corr"]
            m.insert(0, "rebalance_date", reb)
            corr_mat_parts.append(m)

# --- 2bis. Marking/RMT-grade extras: SPX surface, zero curve, returns panel  #
surf_spx = get_surface(db, [SPX_SECID], date_start, date_end, days=surface_days)
surf_spx["date"] = pd.to_datetime(surf_spx["date"])
surf_spx = surf_spx[surf_spx["date"].isin(calendar) & (surf_spx["date"] >= rebals
[0])].copy()
reb_idx = pd.DatetimeIndex(rebals)
surf_spx.insert(
    0, "rebalance_date",
    reb_idx[reb_idx.searchsorted(surf_spx["date"].values, side="right") - 1],
)
surf_spx["permno"] = pd.Series(pd.NA, index=surf_spx.index, dtype="Int64")

surface = pd.concat(surface_parts + [surf_spx], ignore_index=True)
surface["secid"] = surface["secid"].astype("Int64")
surface = surface[
    ["rebalance_date", "date", "permno", "secid", "days", "cp_flag", "iv", "
premium", "strike"]
].sort_values(["rebalance_date", "date", "secid", "days", "cp_flag"]).reset_index(
drop=True)
```

```
199
200     spx_spots = get_spots(db, [SPX_SECID], date_start, date_end)
201     spx_spots["date"] = pd.to_datetime(spx_spots["date"])
202     spx_spots = spx_spots[spx_spots["date"].isin(calendar) & (spx_spots["date"] >=
        rebals[0])].copy()
203     spx_spots.insert(
204         0, "rebalance_date",
205         reb_idx[reb_idx.searchsorted(spx_spots["date"].values, side="right") - 1],
206     )
207     spx_spots["permno"] = pd.Series(pd.NA, index=spx_spots.index, dtype="Int64")
208
209     spots = pd.concat(spot_parts + [spx_spots], ignore_index=True)
210     spots["secid"] = spots["secid"].astype("Int64")
211     spots = spots[
212         ["rebalance_date", "date", "permno", "secid", "close", "cfadj"]
213     ].sort_values(["rebalance_date", "date", "secid"]).reset_index(drop=True)
214
215     rates = db.raw_sql(
216         f"SELECT date, days, rate FROM optionm.zerocd WHERE date BETWEEN '{date_start}'
         AND '{date_end}'"
217     )
218     rates["date"] = pd.to_datetime(rates["date"])
219     rates = rates[rates["date"].isin(calendar)].sort_values(["date", "days"]).
        reset_index(drop=True)
220
221     # union of all constituents ever held; history from returns_start for 252d RMT
        windows
222     all_permnos = sorted({int(p) for wr in weights_rows for p in wr["permno"]})
223     ret_all = get_returns(db, all_permnos, returns_start, date_end)
224     ret_all.index = pd.to_datetime(ret_all.index)
225     returns_long = (
226         ret_all.reset_index(names="date")
227         .melt("date", var_name="permno", value_name="ret")
228         .dropna(subset=["ret"])
229         .sort_values(["date", "permno"])
230         .reset_index(drop=True)
231     )
232     returns_long["permno"] = returns_long["permno"].astype(int)
233
234     # --- 3. Cleaning of component IV (global percentile outliers, then ffill) -- #
235     iv_components = pd.concat(iv_comp_parts, ignore_index=True)
236     lo, hi = iv_components["iv_atm"].quantile(list(iv_pct))
237     iv_components.loc[(iv_components["iv_atm"] < lo) | (iv_components["iv_atm"] > hi),
         "iv_atm"] = np.nan
238     # ffill per (quarter, permno) over the full active window, not just up to the
239     # name's last observation -- so trailing gaps stay NaN, extended at most
        ffill_limit days
240     cleaned = []
241     for (reb, _), g in iv_components.groupby(["rebalance_date", "permno"]):
242         active = active_by_reb[reb]
243         s = g.set_index("date")["iv_atm"].reindex(active).ffill(limit=ffill_limit)
```

```
        cleaned.append(
            pd.DataFrame(
                {"date": s.index, "permno": g["permno"].iloc[0], "secid": g["secid"].iloc[0],
                 "iv_atm": s.values, "rebalance_date": reb}
            )
        )
    iv_components = pd.concat(cleaned, ignore_index=True)[
        ["rebalance_date", "date", "permno", "secid", "iv_atm"]
    ]

    # --- 4. Collect ------------------------------------------------------------ #
    out = {
        "iv_index": iv_index,
        "iv_components": iv_components,
        "weights": pd.concat(weights_rows, ignore_index=True),
        "realized_vol": pd.concat(vol_parts, ignore_index=True),
        "realized_corr": pd.DataFrame(rho_rows, columns=["date", "rho_bar", "rho_bar_equal"]),
        "corr_matrices": pd.concat(corr_mat_parts, ignore_index=True),
        "surface": surface,
        "spots": spots,
        "rates": rates,
        "returns": returns_long,
    }

    if out_dir:
        os.makedirs(out_dir, exist_ok=True)
        for name, df in out.items():
            df.to_parquet(os.path.join(out_dir, f"{name}.parquet"), index=False)

    return out
```

G.3 Signal construction (src/dispersion/signal/)

Listing 8: src/dispersion/signal/implied_corr.py

```
"""
Implied correlation, the correlation-risk premium, and the dispersion signal.

Inverts the one-factor index-variance identity on ATM 91-day IVs to get an
implied rho, then builds two series against the realised rho-bar:

    premium(t) = rho_implied(t) - rho_trailing(t + horizon)    # ex-post check
    signal(t)  = rho_implied(t) - rho_trailing(t)              # ex-ante, tradeable

Usage:
    from dispersion.signal.implied_corr import build_signal
    sig = build_signal()          # reads data/processed/, writes signal.parquet
"""
import os
```

```python

import numpy as np
import pandas as pd


def implied_correlation(
    iv_components: pd.DataFrame,
    weights: pd.DataFrame,
    iv_index: pd.DataFrame,
    n_min: int = 90,
) -> pd.DataFrame:
    """
    Daily implied correlation from the one-factor inversion.

    rho_implied = (sigma_I^2 - S2) / (S1^2 - S2), with S1 = sum(w_hat * sigma)
    and S2 = sum(w_hat^2 * sigma^2) over the names with a valid IV that day;
    w_hat renormalises the frozen rebalance weights over that set.

    n_min sets a coverage floor: days with fewer valid names are dropped.
    Returns a frame indexed by date with [rho_implied, n_names].
    """
    m = iv_components.merge(
        weights[["rebalance_date", "permno", "weight"]],
        on=["rebalance_date", "permno"],
        how="left",
        validate="m:1",
    )
    if m["weight"].isna().any():
        raise ValueError("component IV rows without a matching universe weight")

    valid = m.dropna(subset=["iv_atm"]).copy()
    valid["w_sig"] = valid["weight"] * valid["iv_atm"]
    valid["w2_sig2"] = valid["weight"] ** 2 * valid["iv_atm"] ** 2

    g = valid.groupby("date").agg(
        w_sum=("weight", "sum"),
        s1_raw=("w_sig", "sum"),
        s2_raw=("w2_sig2", "sum"),
        n_names=("weight", "size"),
    )
    s1 = g["s1_raw"] / g["w_sum"]  # renormalised weighted-average component IV
    s2 = g["s2_raw"] / g["w_sum"] ** 2  # renormalised diagonal term

    sig_i = iv_index.set_index("date")["iv_atm"].reindex(g.index)
    n_missing = int(sig_i.isna().sum())
    if n_missing:  # a NaN here would look like a coverage-floor day, not an input
    mismatch
        raise ValueError(f"{n_missing} component-IV dates have no index IV --
    inconsistent inputs")
    rho = (sig_i ** 2 - s2) / (s1 ** 2 - s2)
    rho[g["n_names"] < n_min] = np.nan  # coverage floor
```

```
    return pd.DataFrame(
        {"rho_implied": rho.astype("float64"), "n_names": g["n_names"].astype("Int64")}
    )


def build_signal(
    processed_dir: str = "data/processed",
    horizon: int = 63,
    n_min: int = 90,
    out_file: str | None = "signal.parquet",
) -> pd.DataFrame:
    """
    Build the full signal table and optionally write it to parquet.

    rho_forward(t) is rho_trailing shifted back `horizon` rows so it lines up
    with the implied window. Columns: date, rho_implied, rho_trailing,
    rho_forward, premium, signal, n_names.
    """
    iv_index = pd.read_parquet(os.path.join(processed_dir, "iv_index.parquet"))
    iv_comp = pd.read_parquet(os.path.join(processed_dir, "iv_components.parquet"))
    weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
    rcorr = pd.read_parquet(os.path.join(processed_dir, "realized_corr.parquet"))

    iv_comp["secid"] = iv_comp["secid"].astype("Int64")
    iv_comp["permno"] = iv_comp["permno"].astype("Int64")
    weights["permno"] = weights["permno"].astype("Int64")
    for df in (iv_index, iv_comp, weights, rcorr):
        for c in ("date", "rebalance_date"):
            if c in df.columns:
                df[c] = pd.to_datetime(df[c])

    rho = implied_correlation(iv_comp, weights, iv_index, n_min=n_min)

    spine = rcorr.set_index("date").sort_index()  # master calendar
    lost = rho.index.difference(spine.index)
    extra = spine.index.difference(rho.index)
    if len(lost) or len(extra):  # the right-join must be lossless
        raise ValueError(
            f"calendar mismatch: {len(lost)} rho dates off-spine, {len(extra)} spine
    dates without rho"
        )
    sig = rho.join(spine[["rho_bar"]], how="right").sort_index()
    sig = sig.rename(columns={"rho_bar": "rho_trailing"})
    sig["rho_forward"] = sig["rho_trailing"].shift(-horizon)
    sig["premium"] = sig["rho_implied"] - sig["rho_forward"]
    sig["signal"] = sig["rho_implied"] - sig["rho_trailing"]

    cols = ["rho_implied", "rho_trailing", "rho_forward", "premium", "signal"]
    sig[cols] = sig[cols].astype("float64")
```

```
    sig["n_names"] = sig["n_names"].astype("Int64")
    sig = sig.reset_index(names="date")[["date"] + cols + ["n_names"]]

    if out_file:
        sig.to_parquet(os.path.join(processed_dir, out_file), index=False)
    return sig
```

G.4 Random matrix theory cleaning (src/dispersion/rmt/)

Listing 9: src/dispersion/rmt/cleaning.py

```
"""
RMT cleaning of realised correlation matrices: EWMA de-vol, a 252-day
correlation of the standardised residuals, then Marchenko-Pastur clipping of
the noise bulk (Laloux edge).

Here ‘q_mp‘ = N/T is the MP aspect ratio, not the dividend yield ‘q‘ used in
the pricing modules.

Usage:
    from dispersion.rmt.cleaning import devolatilise, corr_window, laloux_clip
    z = devolatilise(returns_wide)
    C = corr_window(z, date, permnos)
    C_clean, diag = laloux_clip(C)
"""
import numpy as np
import pandas as pd

T_WIN = 252
MIN_OBS = 200
EWMA_LAMBDA = 0.94


def devolatilise(returns_wide: pd.DataFrame, lam: float = EWMA_LAMBDA,
                 min_periods: int = 20) -> pd.DataFrame:
    """
    Standardised residuals z_t = log-return_t / sigma_{t-1}, sigma from an EWMA.
    The one-day shift keeps sigma predictable so correlation windows don't peek
    ahead.
    """
    logret = np.log1p(returns_wide.astype("float64"))
    sigma = logret.ewm(alpha=1.0 - lam, min_periods=min_periods).std().shift(1)
    return logret / sigma


def corr_window(z: pd.DataFrame, date, permnos, t_win: int = T_WIN,
                min_obs: int = MIN_OBS, min_names: int = 60):
    """
    Complete-case correlation of the last ‘t_win‘ rows of z up to ‘date‘ for the
    given universe. Names with fewer than ‘min_obs‘ observations are dropped
    first. Returns None if the panel is too thin, else (C, t_eff).
```

```
41
42      t_eff is the number of complete-case rows actually used; the MP edge needs
43      q_mp = N/t_eff, not N/t_win, or iid noise slips through as signal.
44      """
45      cols = [p for p in permnos if p in z.columns]
46      win = z.loc[:pd.Timestamp(date), cols].tail(t_win)
47      win = win.dropna(axis=1, thresh=min_obs).dropna(axis=0)
48      if win.shape[1] < min_names or win.shape[0] < min_obs:
49          return None
50      return win.corr(), int(win.shape[0])
51
52
53  def laloux_clip(C: pd.DataFrame, t_win: int = T_WIN):
54      """
55      Clip eigenvalues below the Laloux edge to the bulk mean, then renormalise.
56      Returns (C_clean, diagnostics); diagnostics holds n, t, q_mp, lam1_share,
57      edge_naive, edge_laloux, k_signal and trace_prenorm.
58      """
59      is_df = isinstance(C, pd.DataFrame)
60      A = C.to_numpy(dtype="float64") if is_df else np.asarray(C, dtype="float64")
61      n = A.shape[0]
62      ev, V = np.linalg.eigh(A)                       # ascending
63      q_mp = n / t_win
64      lam1_share = ev[-1] / n
65      edge_naive = (1.0 + np.sqrt(q_mp)) ** 2
66      edge = (1.0 - lam1_share) * edge_naive          # Laloux correction
67      bulk = ev < edge
68      ev2 = ev.copy()
69      if bulk.any():
70          ev2[bulk] = ev[bulk].mean()                 # trace-preserving on the bulk
71      A2 = (V * ev2) @ V.T
72      d = np.sqrt(np.diag(A2))
73      A2 = A2 / np.outer(d, d)
74      np.fill_diagonal(A2, 1.0)
75
76      out = pd.DataFrame(A2, index=C.index, columns=C.columns) if is_df else A2
77      diagnostics = {
78          "n": n, "t": t_win, "q_mp": q_mp, "lam1_share": float(lam1_share),
79          "edge_naive": float(edge_naive), "edge_laloux": float(edge),
80          "k_signal": int((~bulk).sum()), "trace_prenorm": float(ev2.sum()),
81      }
82      return out, diagnostics
83
84
85  def spectral_features(C: pd.DataFrame, t_win: int = T_WIN, top: int = 5):
86      """
87      Regime features of a correlation matrix. Returns (features dict, dominant
88      eigenvector); the caller uses v1 for the day-to-day rotation.
89
90      Features: lam1_share (market mode), k_signal (factors above the Laloux
91      edge), absorption_top (top-‘top‘ absorption ratio), lam2_share, spec_entropy
```

```
    (spectral entropy, risk concentration), pr_v1 (how spread the market mode is).
    """
    A = C.to_numpy(dtype="float64")
    n = A.shape[0]
    ev, V = np.linalg.eigh(A)
    q_mp = n / t_win
    edge = (1.0 - ev[-1] / n) * (1.0 + np.sqrt(q_mp)) ** 2
    p = np.clip(ev, 1e-12, None) / n
    v1 = pd.Series(V[:, -1], index=C.index)
    if v1.sum() < 0:                                # sign convention: market mode
    positive
        v1 = -v1
    feats = {
        "lam1_share": float(ev[-1] / n),
        "k_signal": int((ev >= edge).sum()),
        "absorption_top": float(ev[-top:].sum() / n),
        "lam2_share": float(ev[-2] / n) if n >= 2 else np.nan,
        "spec_entropy": float(-(p * np.log(p)).sum()),
        "pr_v1": float(1.0 / (v1.to_numpy() ** 4).sum() / n),
    }
    return feats, v1
```

Listing 10: src/dispersion/rmt/daily.py

```
"""
Daily RMT series: a cleaned rho-bar variant of the signal plus spectral regime
features for the ML layer. Builds rmt_daily.parquet and signal_rmt.parquet.

signal_rmt keeps a ‘premium‘ column only to match signal.parquet’s schema --
don’t use it. A 252d trailing leg shifted by 63 rows overlaps itself ~75%, so
that premium is mechanically contaminated; only ‘signal‘ is safe to trade on.

Usage:
    from dispersion.rmt.daily import build_rmt_daily, build_signal_rmt
    build_rmt_daily()      # ~7,200 window eigendecompositions, a few minutes
    build_signal_rmt()
"""
import os

import numpy as np
import pandas as pd

from ..data.returns import average_correlation
from .cleaning import (EWMA_LAMBDA, MIN_OBS, T_WIN, corr_window, devolatilise,
                       laloux_clip, spectral_features)


def _rotation(v1: pd.Series, v1_prev: pd.Series | None) -> float:
    """
    Day-to-day rotation of the dominant eigenvector: 1 - |<v1_t, v1_{t-1}>| over
    the common names (each subvector renormalised). NaN with no previous vector
```

```
    or no overlap.
    """
    if v1_prev is None:
        return np.nan
    common = v1.index.intersection(v1_prev.index)
    if len(common) < 10:
        return np.nan
    a = v1.loc[common].to_numpy(dtype="float64")
    b = v1_prev.loc[common].to_numpy(dtype="float64")
    na, nb = np.linalg.norm(a), np.linalg.norm(b)
    if na == 0 or nb == 0:
        return np.nan
    return float(1.0 - abs(a @ b) / (na * nb))


def build_rmt_daily(
    processed_dir: str = "data/processed",
    t_win: int = T_WIN,
    min_obs: int = MIN_OBS,
    lam: float = EWMA_LAMBDA,
    out_file: str | None = "rmt_daily.parquet",
) -> pd.DataFrame:
    """Daily cleaned rho-bar + spectral features, frozen universe per quarter."""
    returns = pd.read_parquet(os.path.join(processed_dir, "returns.parquet"))
    weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
    signal = pd.read_parquet(os.path.join(processed_dir, "signal.parquet"))

    returns["date"] = pd.to_datetime(returns["date"])
    weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
    spine = pd.DatetimeIndex(pd.to_datetime(signal["date"])).sort_values()

    ret = returns.pivot(index="date", columns="permno", values="ret").sort_index().
    astype("float64")
    z = devolatilise(ret, lam=lam)
    sig252 = (np.log1p(ret)).rolling(t_win, min_periods=min_obs).std() * np.sqrt
    (252.0)

    rebals = sorted(weights["rebalance_date"].unique())
    rows = []
    for k, reb in enumerate(rebals):
        reb = pd.Timestamp(reb)
        nxt = pd.Timestamp(rebals[k + 1]) if k + 1 < len(rebals) else spine[-1] + pd.
    Timedelta(days=1)
        active = spine[(spine >= reb) & (spine < nxt)]
        wq = weights[weights["rebalance_date"] == reb]
        wmap = wq.set_index("permno")["weight"].astype("float64")
        v1_prev, lam1_prev = None, np.nan

        for d in active:
            res = corr_window(z, d, wmap.index.tolist(), t_win=t_win, min_obs=min_obs)
            if res is None:
```

```
                rows.append((d,) + (np.nan,) * 12 + (0,))
                v1_prev, lam1_prev = None, np.nan
                continue
            C, t_eff = res                              # effective T for the MP edge
            C_clean, diag = laloux_clip(C, t_win=t_eff)
            feats, v1 = spectral_features(C, t_win=t_eff)

            kept = C.columns
            w_kept = wmap.reindex(kept)
            s_kept = sig252.loc[d, kept]                # KeyError if d is missing,
    rather than a stale row
            rho_raw = average_correlation(C, w_kept, s_kept, "weighted")
            rho_clean = average_correlation(C_clean, w_kept, s_kept, "weighted")

            # cross-name dispersion of average correlation
            A = C.to_numpy(dtype="float64")
            corr_xdisp = float(np.std(A.mean(axis=1)))
            # Mahalanobis turbulence on the cleaned inverse -- inverting a noisy C
            # is where cleaning actually matters
            zrow = z.loc[d, kept].to_numpy(dtype="float64")
            mask = np.isfinite(zrow)
            if mask.sum() >= 30:
                Ainv = np.linalg.pinv(C_clean.to_numpy(dtype="float64")[np.ix_(mask,
    mask)])
                zc = zrow[mask]
                turb = float(zc @ Ainv @ zc / mask.sum())
            else:
                turb = np.nan

            rows.append((d, rho_raw, rho_clean, feats["lam1_share"], feats["k_signal"],

                         feats["absorption_top"],
                         feats["lam1_share"] - lam1_prev if np.isfinite(lam1_prev)
    else np.nan,
                         _rotation(v1, v1_prev), feats["lam2_share"], feats["
    spec_entropy"],
                         feats["pr_v1"], corr_xdisp, turb, len(kept)))
            v1_prev, lam1_prev = v1, feats["lam1_share"]

    out = pd.DataFrame(rows, columns=["date", "rho_rmt_raw", "rho_rmt_clean", "
    lam1_share",
                                      "k_signal", "absorption_top", "d_lam1", "
    rotation",
                                      "lam2_share", "spec_entropy", "pr_v1", "
    corr_xdisp",
                                      "turb", "n_names_rmt"])
    if out_file:
        out.to_parquet(os.path.join(processed_dir, out_file), index=False)
    return out
```

```python
def build_signal_rmt(
    processed_dir: str = "data/processed",
    horizon: int = 63,
    out_file: str | None = "signal_rmt.parquet",
) -> pd.DataFrame:
    """
    Signal variant whose trailing realised leg is the RMT-cleaned 252d rho-bar.
    Same columns as signal.parquet so the engine gate works unchanged.
    """
    sig = pd.read_parquet(os.path.join(processed_dir, "signal.parquet"))
    rmt = pd.read_parquet(os.path.join(processed_dir, "rmt_daily.parquet"))
    sig["date"] = pd.to_datetime(sig["date"])
    rmt["date"] = pd.to_datetime(rmt["date"])

    m = sig[["date", "rho_implied", "n_names"]].merge(
        rmt[["date", "rho_rmt_clean", "n_names_rmt"]], on="date", how="left", validate
    ="1:1"
    ).sort_values("date").reset_index(drop=True)
    m["rho_trailing"] = m["rho_rmt_clean"].astype("float64")
    m["rho_forward"] = m["rho_trailing"].shift(-horizon)
    m["premium"] = m["rho_implied"] - m["rho_forward"]
    m["signal"] = m["rho_implied"] - m["rho_trailing"]
    m = m[["date", "rho_implied", "rho_trailing", "rho_forward", "premium", "signal",
           "n_names", "n_names_rmt"]]
    if out_file:
        m.to_parquet(os.path.join(processed_dir, out_file), index=False)
    return m
```

G.5 Backtest engine (src/dispersion/backtest/)

Listing 11: src/dispersion/backtest/marking.py

```python
"""
Mark aged option positions day by day: interpolate ATM sigma across pillars,
look up rates, and rebase strikes for splits.

    from dispersion.backtest.marking import interp_sigma, RateCurve, adjust_strike
"""
import numpy as np
import pandas as pd

PILLARS = (30, 60, 91)


def interp_sigma(sig30, sig60, sig91, tau_days):
    """
    ATM sigma at residual maturity `tau_days`, interpolated linearly in total
    variance w = sigma^2 * tau. Below 30 days we hold sigma(30) (short-end slope
    dropped, level still tracked). Scalars or aligned arrays; NaN pillars propagate.
    """
    sig30 = np.asarray(sig30, dtype=float)
```

```
    sig60 = np.asarray(sig60, dtype=float)
    sig91 = np.asarray(sig91, dtype=float)
    tau = np.asarray(tau_days, dtype=float)

    w30, w60, w91 = sig30**2 * 30.0, sig60**2 * 60.0, sig91**2 * 91.0

    w_lo = w30 + (w60 - w30) * (tau - 30.0) / 30.0        # [30, 60]
    w_hi = w60 + (w91 - w60) * (tau - 60.0) / 31.0        # [60, 91]

    with np.errstate(invalid="ignore", divide="ignore"):
        sig_lo = np.sqrt(w_lo / tau)
        sig_hi = np.sqrt(w_hi / tau)

    out = np.where(
        tau <= 30.0, sig30,
        np.where(tau <= 60.0, sig_lo, np.where(tau <= 91.0, sig_hi, sig91)),
    )
    return out if out.ndim else float(out)


class RateCurve:
    """
    Continuously-compounded zero rate r(date, tau) from the zerocd panel.
    Linear across the maturity grid, clamped at the ends. If `date` is missing
    from zerocd, fall back to the most recent curve within `max_stale_days`
    calendar days; beyond that, raise rather than use a stale rate. Parquet is in
    percent, returned as decimals.
    """

    def __init__(self, rates: pd.DataFrame, max_stale_days: int = 7):
        self._dates = pd.DatetimeIndex(np.sort(rates["date"].unique()))
        self._by_date = {
            d: (g["days"].to_numpy(dtype=float), g["rate"].to_numpy(dtype=float))
            for d, g in rates.sort_values("days").groupby("date")
        }
        self.max_stale_days = int(max_stale_days)

    def rate(self, date, tau_days: float) -> float:
        date = pd.Timestamp(date)
        if date in self._by_date:
            use = date
        else:
            pos = self._dates.searchsorted(date, side="right") - 1
            if pos < 0:
                raise KeyError(f"no zerocd curve on or before {date.date()}")
            use = self._dates[pos]
            if (date - use).days > self.max_stale_days:
                raise KeyError(f"zerocd gap too wide at {date.date()} (last curve {use.
    date()})")
        days, rate = self._by_date[use]
        return float(np.interp(float(tau_days), days, rate)) / 100.0
```

```

def adjust_strike(k_entry: float, cfadj_entry: float, cfadj_now: float) -> float:
    """
    Strike rebased to today's price scale after splits: K_t = K_entry *
    cfadj_entry / cfadj_t. Quantity scales by the inverse, so a split leaves the
    invested wealth unchanged.
    """
    return float(k_entry) * float(cfadj_entry) / float(cfadj_now)
```

Listing 12: src/dispersion/backtest/engine.py

```
"""
Dispersion backtest: short the SPX straddle, long a vega-matched basket of
component straddles, delta-hedge the index daily. v0 trades every quarter; v1
gates on the signal.

    from dispersion.backtest.engine import run_backtest
    res = run_backtest()                    # v0 (unconditional)
    res = run_backtest(threshold=0.05)      # v1 (trade only if signal > threshold)
    res["daily"], res["quarterly"]
"""
import os

import numpy as np
import pandas as pd
from scipy.optimize import brentq

from dispersion.backtest.marking import RateCurve, adjust_strike, interp_sigma
from dispersion.utils.greeks import bs_greeks, bs_price

SPX_SECID = 108105
PILLARS = (30, 60, 91)
ACT = 365.0

# Quoted relative bid-ask spreads (% of premium), linear between the 1996 and
# 2024 anchors, clamped outside. Entry pays half per leg (impl_premium is
# mid-like); the daily index hedge pays `hedge_bps` of notional; settlement is free.
COST_ANCHORS = {"spx": (0.010, 0.006),     # SPX options
                "large": (0.070, 0.012),   # constituents ranked 1-50
                "small": (0.100, 0.030)}   # constituents ranked 51-100
DEFAULT_COSTS = {"hedge_bps": 1e-4, "scale": 1.0}


def parametric_spread(year: int, group: str) -> float:
    """Quoted relative spread for `group` in `year` (linear between anchors)."""
    v0, v1 = COST_ANCHORS[group]
    t = min(max(year, 1996), 2024)
    return v0 + (v1 - v0) * (t - 1996) / (2024 - 1996)


```

```
# ---------------------------------------------------------------------------- #
# Entry helpers
# ---------------------------------------------------------------------------- #
def implied_q(S, k_call, k_put, sig_call, sig_put, T, r, c_mkt, p_mkt,
              lo=-0.10, hi=0.25) -> float:
    """
    Dividend yield backed out of the vendor premiums via C - P (monotone in q).
    Falls back to 0.0 if there's no root in [lo, hi].
    """
    def cp_diff(q):
        c = bs_price(S, k_call, sig_call, T, r, q, "C")
        p = bs_price(S, k_put, sig_put, T, r, q, "P")
        return (c - p) - (c_mkt - p_mkt)

    try:
        return float(brentq(cp_diff, lo, hi))
    except ValueError:
        return 0.0


def exante_quantile_threshold(signal: pd.DataFrame, rebalances, q: float = 0.5,
                              warmup: int = 12) -> pd.Series:
    """
    Per-rebalance ex-ante threshold for v1: quantile `q` of the daily signal
    history up to that date (expanding window, so no look-ahead). The first
    `warmup` rebalances get NaN, which the engine reads as "trade every quarter".
    """
    s = signal.copy()
    s["date"] = pd.to_datetime(s["date"])
    s = s.set_index("date")["signal"].astype(float).sort_index()
    out = {}
    for i, reb in enumerate(rebalances):
        reb = pd.Timestamp(reb)
        hist = s.loc[:reb].dropna()
        out[reb] = np.nan if (i < warmup or hist.empty) else float(hist.quantile(q))
    return pd.Series(out)


def _entry_leg(rows, cp):
    r = rows[rows["cp_flag"] == cp]
    if len(r) != 1:
        return None
    r = r.iloc[0]
    return float(r["iv"]), float(r["premium"]), float(r["strike"])


class _Position:
    """One straddle: fixed strikes, fixed entry q-hat, split-aware."""

    __slots__ = ("secid", "qty0", "cfadj0", "k_call", "k_put", "q_hat",
                 "last_mark", "last_greeks", "frozen_days")
```

```
91
92      def __init__(self, secid, qty0, cfadj0, k_call, k_put, q_hat, entry_price):
93          self.secid = secid
94          self.qty0 = qty0
95          self.cfadj0 = cfadj0
96          self.k_call = k_call
97          self.k_put = k_put
98          self.q_hat = q_hat
99          self.last_mark = entry_price          # per-unit straddle value
100         self.last_greeks = {"delta_S": 0.0, "vega_c": 0.0, "vega_p": 0.0,
101                             "gamma": 0.0, "theta": 0.0, "S": np.nan,
102                             "sig_c": np.nan, "sig_p": np.nan}
103         self.frozen_days = 0
104
105     def qty(self, cfadj_now):
106         return self.qty0 * cfadj_now / self.cfadj0
107
108     def strikes(self, cfadj_now):
109         return (adjust_strike(self.k_call, self.cfadj0, cfadj_now),
110                 adjust_strike(self.k_put, self.cfadj0, cfadj_now))
111
112
113 # ------------------------------------------------------------------------ #
114 # Main
115 # ------------------------------------------------------------------------ #
116 def run_backtest(
117     processed_dir: str = "data/processed",
118     data: dict | None = None,
119     threshold: float | None = None,
120     costs: dict | None = None,
121     cash_tenor: float = 10.0,
122     min_names: int = 80,
123     n_leg: int | None = None,
124 ) -> dict:
125     """
126     Run the backtest over every rebalance.
127
128     threshold=None -> v0 (always trade); a float or per-rebalance pd.Series gates
129     the quarters (v1). costs=None -> gross; costs={} or DEFAULT_COSTS -> net of the
130     parametric spreads (plus hedge_bps on hedge notional; optional 'scale'). `data`
131     injects the input frames directly for tests, otherwise they're read from parquet.
132     Returns dict(daily, quarterly, ledger).
133     """
134     if data is None:
135         data = {k: pd.read_parquet(os.path.join(processed_dir, f"{k}.parquet"))
136                 for k in ("surface", "spots", "rates", "weights", "signal")}
137     surface, spots, rates = data["surface"], data["spots"], data["rates"]
138     weights, signal = data["weights"], data["signal"]
139
140     for df in (surface, spots, signal):
141         df["date"] = pd.to_datetime(df["date"])
```

```
    for df in (surface, spots, weights):
        df["rebalance_date"] = pd.to_datetime(df["rebalance_date"])

    rc = RateCurve(rates)
    sig_at = signal.set_index("date")["signal"]
    rebals = sorted(weights["rebalance_date"].unique())

    use_costs = costs is not None
    cost_scale = (costs or {}).get("scale", DEFAULT_COSTS["scale"])
    hedge_bps = (costs or {}).get("hedge_bps", DEFAULT_COSTS["hedge_bps"]) if
    use_costs else 0.0

    W = 1.0
    prev_nav = W
    daily_rows, q_rows, ledger_rows = [], [], []

    for k, reb in enumerate(rebals[:-1]):
        reb = pd.Timestamp(reb)
        nxt = pd.Timestamp(rebals[k + 1])

        surf_q = surface[surface["rebalance_date"] == reb]
        spot_q = spots[spots["rebalance_date"] == reb]
        w_q = weights[weights["rebalance_date"] == reb]

        # quarter pivots: sigma pillars per (secid, cp, days); spot close/cfadj
        piv_iv = surf_q.pivot_table(index="date", columns=["secid", "cp_flag", "days"],

                                    values="iv", aggfunc="first").astype("float64")
        piv_close = spot_q.pivot_table(index="date", columns="secid", values="close",
                                       aggfunc="first").astype("float64")
        piv_cfadj = spot_q.pivot_table(index="date", columns="secid", values="cfadj",
                                       aggfunc="first").astype("float64")
        # force float64 -- nullable dtypes leak pd.NA and break np.isfinite
        dates = piv_close.index.sort_values()

        # gate: None -> v0; float -> fixed threshold; Series -> per-rebalance
        # threshold (NaN = warm-up -> trade)
        if threshold is None:
            traded = True
        else:
            thr = float(threshold.get(reb, np.nan)) if isinstance(threshold, pd.Series
    ) \
                else float(threshold)
            if np.isnan(thr):
                traded = True
            else:
                s_val = float(sig_at.loc[reb]) if reb in sig_at.index else np.nan
                traded = bool(np.isfinite(s_val) and s_val > thr)

        # ------------------------------------------------------------------ #
        # Entry at the rebalance close
```

```
        # ------------------------------------------------------------------ #
        book, cash, n_hedge = [], W, 0.0
        y_sum = lam = np.nan
        held_n = 0
        c_entry_q = c_hedge_q = 0.0
        if traded:
            T0 = (nxt - reb).days / ACT
            r0 = rc.rate(reb, (nxt - reb).days)
            e = surf_q[(surf_q["date"] == reb) & (surf_q["days"] == 91)]

            entries = {}
            for secid, rows in e.groupby("secid"):
                c = _entry_leg(rows, "C")
                p = _entry_leg(rows, "P")
                S0 = piv_close.at[reb, secid] if secid in piv_close.columns else np.
nan
                if c is None or p is None or not np.isfinite(S0):
                    continue
                sc, prem_c, kc = c
                sp_, prem_p, kp = p
                qh = implied_q(S0, kc, kp, sc, sp_, T0, r0, prem_c, prem_p)
                g_c = bs_greeks(S0, kc, sc, T0, r0, qh, "C")
                g_p = bs_greeks(S0, kp, sp_, T0, r0, qh, "P")
                price = prem_c + prem_p
                entries[secid] = dict(
                    price=price, kc=kc, kp=kp, qh=qh,
                    nu=(g_c["vega"] + g_p["vega"]) / price,   # relative vega
                    sig_bar=0.5 * (sc + sp_),
                    cfadj0=float(piv_cfadj.at[reb, secid]),
                )

            if SPX_SECID not in entries:
                raise ValueError(f"no SPX entry at {reb.date()}")
            idx_e = entries.pop(SPX_SECID)

            w_map = w_q.dropna(subset=["secid"]).set_index("secid")["weight"]
            held = [s for s in entries if s in w_map.index]
            # only trade the top n_leg names by cap weight; w_til renormalises over
            # them so the leg stays vega-neutral. Fewer, larger legs -> fewer
            # spreads, and tight large-cap spreads replace the wide small-cap tail.
            if n_leg is not None and len(held) > n_leg:
                held = w_map.loc[held].sort_values(ascending=False).head(n_leg).index.
tolist()
            held_n = len(held)
            floor = min(n_leg, 10) if n_leg is not None else min_names
            if held_n < floor:
                raise ValueError(f"only {held_n} names with entry premiums at {reb.
date()}")

            w_til = w_map.loc[held] / w_map.loc[held].sum()
            denom = sum(w_til[s] * entries[s]["nu"] * entries[s]["sig_bar"] for s in
```

```
    held)
238         lam = idx_e["nu"] * idx_e["sig_bar"] / denom      # vega-neutral to a
    proportional shock
239         y = {s: lam * w_til[s] for s in held}
240         y_sum = float(sum(y.values()))
241 
242         # index short: -100% of wealth; components: +y_i of wealth
243         book.append(_Position(SPX_SECID, -W / idx_e["price"], idx_e["cfadj0"],
244                              idx_e["kc"], idx_e["kp"], idx_e["qh"], idx_e["price"
    ]))
245         cash += W                                          # short proceeds
246         for s in held:
247             q_i = y[s] * W / entries[s]["price"]
248             book.append(_Position(s, q_i, entries[s]["cfadj0"], entries[s]["kc"],
249                                  entries[s]["kp"], entries[s]["qh"], entries[s]["
    price"]))
250             cash -= y[s] * W
251 
252         if use_costs:  # pay half the quoted spread on each entry leg
253             rnk_map = w_q.dropna(subset=["secid"]).set_index("secid")["rnk"] \
254                 if "rnk" in w_q.columns else pd.Series(dtype=float)
255             c_entry_q = 0.5 * cost_scale * parametric_spread(reb.year, "spx") * W
256             for s in held:
257                 grp = "large" if float(rnk_map.get(s, 1)) <= 50 else "small"
258                 c_entry_q += 0.5 * cost_scale * parametric_spread(reb.year, grp) *
     y[s] * W
259             cash -= c_entry_q
260 
261     # ------------------------------------------------------------------ #
262     # Daily loop: mark, delta-hedge with the index, accrue cash
263     # ------------------------------------------------------------------ #
264     def _mark(pos, t, tau_days):
265         """Per-unit straddle mark + greeks; freezes on missing data."""
266         secid = pos.secid
267         S = piv_close.at[t, secid] if (secid in piv_close.columns
268                                        and t in piv_close.index) else np.nan
269         cf = piv_cfadj.at[t, secid] if np.isfinite(S) else np.nan
270         sigs = {}
271         for cp in ("C", "P"):
272             try:
273                 s30 = piv_iv.at[t, (secid, cp, 30)]
274                 s60 = piv_iv.at[t, (secid, cp, 60)]
275                 s91 = piv_iv.at[t, (secid, cp, 91)]
276             except KeyError:
277                 s30 = s60 = s91 = np.nan
278             sigs[cp] = interp_sigma(s30, s60, s91, tau_days)
279         if not (np.isfinite(S) and np.isfinite(sigs["C"]) and np.isfinite(sigs["P"
    ])
280                 and np.isfinite(cf)):
281             pos.frozen_days += 1
282             return pos.last_mark, pos.last_greeks, None
```

```
            kc, kp = pos.strikes(cf)
            T = max(tau_days, 0.5) / ACT
            r = rc.rate(t, max(tau_days, 1.0))
            g_c = bs_greeks(S, kc, sigs["C"], T, r, pos.q_hat, "C")
            g_p = bs_greeks(S, kp, sigs["P"], T, r, pos.q_hat, "P")
            mark = g_c["price"] + g_p["price"]
            greeks = {"delta_S": (g_c["delta"] + g_p["delta"]) * S,
                      "vega_c": g_c["vega"], "vega_p": g_p["vega"],
                      "gamma": g_c["gamma"] + g_p["gamma"],
                      "theta": g_c["theta"] + g_p["theta"],
                      "S": S, "sig_c": sigs["C"], "sig_p": sigs["P"]}
            pos.last_mark, pos.last_greeks = mark, greeks
            return mark, greeks, cf

        def _book_state(t, tau_days):
            st = {"nav_pos": 0.0, "delta_dollar": 0.0, "nav_idx": 0.0, "nav_comp": 0.0,
                  "theta": 0.0, "vega": 0.0, "gamma_S2": 0.0}
            for pos in book:
                mark, greeks, cf = _mark(pos, t, tau_days)
                qty = pos.qty(cf) if cf is not None else pos.qty(pos.cfadj0)
                v = qty * mark
                st["nav_pos"] += v
                st["delta_dollar"] += qty * greeks["delta_S"]
                st["theta"] += qty * greeks["theta"]
                st["vega"] += qty * (greeks["vega_c"] + greeks["vega_p"])
                if np.isfinite(greeks["S"]):
                    st["gamma_S2"] += qty * greeks["gamma"] * greeks["S"] ** 2
                if pos.secid == SPX_SECID:
                    st["nav_idx"] += v
                else:
                    st["nav_comp"] += v
            return st

        # entry-day state (hedge set at the entry close)
        spx_S = lambda t: piv_close.at[t, SPX_SECID]
        if traded:
            st0 = _book_state(reb, (nxt - reb).days)
            n_hedge = -st0["delta_dollar"] / spx_S(reb)
            cash -= n_hedge * spx_S(reb)
            c = hedge_bps * abs(n_hedge) * spx_S(reb)
            cash -= c
            c_hedge_q += c

        prev_t = reb
        for t in dates[dates > reb]:
            dt_cal = (t - prev_t).days
            cash *= float(np.exp(rc.rate(prev_t, cash_tenor) * dt_cal / ACT))
            tau = (nxt - t).days
            if traded:
                st = _book_state(t, tau)
```

```
                nav = cash + st["nav_pos"] + n_hedge * spx_S(t)
                ledger_rows.append((t, reb, nav, cash, st["nav_idx"], st["nav_comp"],
                                    n_hedge, float(spx_S(t)), st["theta"], st["vega"],
                                    st["gamma_S2"], st["delta_dollar"]))
                # re-hedge at the close
                n_new = -st["delta_dollar"] / spx_S(t)
                cash -= (n_new - n_hedge) * spx_S(t)
                c = hedge_bps * abs(n_new - n_hedge) * spx_S(t)
                cash -= c
                c_hedge_q += c
                n_hedge = n_new
            else:
                nav = cash
            daily_rows.append((t, reb, nav / prev_nav - 1.0, nav, held_n, traded))
            prev_nav = nav
            prev_t = t

        # ------------------------------------------------------------------ #
        # Settlement at the next rebalance (intrinsic payoff)
        # ------------------------------------------------------------------ #
        dt_cal = (nxt - prev_t).days
        cash *= float(np.exp(rc.rate(prev_t, cash_tenor) * dt_cal / ACT))
        n_frozen_settle = 0
        if traded:
            spot_n = spots[(spots["rebalance_date"] == nxt) & (spots["date"] == nxt)]
            s_map = spot_n.set_index("secid")["close"]
            cf_map = spot_n.set_index("secid")["cfadj"]
            for pos in book:
                if pos.secid in s_map.index:
                    S, cf = float(s_map.loc[pos.secid]), float(cf_map.loc[pos.secid])
                else:  # left the universe / delisted: last frozen mark's spot
                    S, cf = pos.last_greeks["S"], pos.cfadj0
                    n_frozen_settle += 1
                    if not np.isfinite(S):
                        S, cf = 0.0, pos.cfadj0   # worthless fallback, counted
                kc, kp = pos.strikes(cf)
                payoff = max(S - kc, 0.0) + max(kp - S, 0.0)
                cash += pos.qty(cf) * payoff
            s_settle = float(s_map.loc[SPX_SECID])
            cash += n_hedge * s_settle
            c = hedge_bps * abs(n_hedge) * s_settle
            cash -= c
            c_hedge_q += c
            n_hedge = 0.0

        nav = cash
        daily_rows.append((nxt, reb, nav / prev_nav - 1.0, nav, held_n, traded))
        prev_nav = nav

        q_rows.append((reb, nxt, traded, held_n, nav / W - 1.0, y_sum, lam,
                       float(np.mean([p.frozen_days for p in book])) if book else 0.0,
```

```
                        n_frozen_settle, c_entry_q / W, c_hedge_q / W))
        W = nav

    daily = pd.DataFrame(daily_rows,
                         columns=["date", "rebalance_date", "ret", "nav",
                                  "n_names", "traded"])
    quarterly = pd.DataFrame(q_rows,
                             columns=["rebalance_date", "settle_date", "traded",
                                      "n_names", "ret_q", "y_sum", "lam",
                                      "avg_frozen_days", "n_frozen_settle",
                                      "cost_entry", "cost_hedge"])
    ledger = pd.DataFrame(ledger_rows,
                          columns=["date", "rebalance_date", "nav", "cash", "nav_idx",
                                   "nav_comp", "hedge_shares", "spx_close", "theta",
                                   "vega", "gamma_S2", "delta_dollar"])
    return {"daily": daily, "quarterly": quarterly, "ledger": ledger}
```

G.6 Machine learning layer (src/dispersion/ml/)

Listing 13: src/dispersion/ml/features.py

```
"""
Build the ML feature table from the existing parquets (no new WRDS pull):
spectral features from the cleaned correlation matrix, SPX vol structure,
realised vol / VRP, the component-IV cross-section, and correlation levels. The
label y_spike is future minus current realised correlation, used in training only.

    from dispersion.ml.features import build_ml_dataset
    df = build_ml_dataset()        # writes data/processed/ml_dataset.parquet
"""
import os

import numpy as np
import pandas as pd

from ..backtest.engine import SPX_SECID, parametric_spread

HORIZON = 63          # trading days ~ the 91-calendar-day decision horizon
D21 = 21              # short dynamics window

# core set for the Gaussian models -- a full-cov fit in 32 dims wouldn't survive
# ~2,000 walk-forward observations
CORE_SET = ["f_lam1", "f_dlam1_21", "f_turb21", "f_vrp", "f_term_slope",
            "f_anchor_gap", "f_iv_spx", "f_rot21"]


def build_ml_dataset(
    processed_dir: str = "data/processed",
    out_file: str | None = "ml_dataset.parquet",
) -> pd.DataFrame:
    sig = pd.read_parquet(os.path.join(processed_dir, "signal.parquet"))
```

```
    sigr = pd.read_parquet(os.path.join(processed_dir, "signal_rmt.parquet"))
    rmt = pd.read_parquet(os.path.join(processed_dir, "rmt_daily.parquet"))
    ivx = pd.read_parquet(os.path.join(processed_dir, "iv_index.parquet"))
    surface = pd.read_parquet(os.path.join(processed_dir, "surface.parquet"))
    spots = pd.read_parquet(os.path.join(processed_dir, "spots.parquet"))
    ivc = pd.read_parquet(os.path.join(processed_dir, "iv_components.parquet"))
    wts = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
    vix = pd.read_parquet(os.path.join(processed_dir, "vix.parquet"))  # CBOE (cboe.
cboe)
    for df in (sig, sigr, rmt, ivx, surface, spots, ivc, vix):
        df["date"] = pd.to_datetime(df["date"])

    # SPX term slope (30d -> 91d ATM pillars, C/P averaged)
    spx_surf = surface[(surface["secid"] == SPX_SECID) & (surface["days"].isin([30,
91]))]
    piv = (spx_surf.groupby(["date", "days"])["iv"].mean().unstack("days")
           .astype("float64"))
    term = ((piv[91] - piv[30]) / piv[30]).rename("f_term_slope")

    # SPX realised vol block (vendor closes, same source as the strikes)
    spx_close = (spots[spots["secid"] == SPX_SECID]
                 .drop_duplicates("date").set_index("date")["close"]
                 .astype("float64").sort_index())
    lr = np.log(spx_close).diff()
    rv21 = (lr.rolling(D21, min_periods=15).std() * np.sqrt(252.0)).rename("f_rv21")
    rv63 = (lr.rolling(63, min_periods=50).std() * np.sqrt(252.0)).rename("f_rv63")
    dd252 = (spx_close / spx_close.rolling(252, min_periods=100).max() - 1.0).rename("
f_dd252")
    mom63 = (spx_close.pct_change(63)).rename("f_mom63")

    # component-IV cross-section (weighted, renormalised)
    ivw = ivc.merge(wts[["rebalance_date", "permno", "weight"]],
                    on=["rebalance_date", "permno"], how="left")
    ivw = ivw.dropna(subset=["iv_atm"])
    g = ivw.assign(ws=ivw["weight"] * ivw["iv_atm"]).groupby("date")
    iv_comp = (g["ws"].sum() / g["weight"].sum()).astype("float64").rename("f_iv_comp"
)
    iv_xdisp = g["iv_atm"].std().astype("float64").rename("f_iv_xdisp")

    # assemble everything on the date spine
    m = (
        sig[["date", "rho_implied", "rho_trailing", "signal"]]
        .rename(columns={"rho_trailing": "rho_trail63", "signal": "f_sig_base"})
        .merge(sigr[["date", "signal"]].rename(columns={"signal": "f_sig_rmt"}),
               on="date", validate="1:1")
        .merge(rmt[["date", "rho_rmt_clean", "lam1_share", "k_signal", "absorption_top
",
                    "rotation", "lam2_share", "spec_entropy", "pr_v1", "corr_xdisp",
                    "turb"]], on="date", validate="1:1")
        .merge(ivx[["date", "iv_atm", "iv_call_50", "iv_put_50"]], on="date",
               how="left", validate="1:1")
```

```
        .sort_values("date").reset_index(drop=True)
    )
    for s in (term, rv21, rv63, dd252, mom63, iv_comp, iv_xdisp):
        m = m.merge(s.reset_index(), on="date", how="left")
    m = m.merge(vix, on="date", how="left", validate="1:1")

    out = pd.DataFrame({"date": m["date"]})
    # A. spectral
    out["f_lam1"] = m["lam1_share"]
    out["f_lam2"] = m["lam2_share"]
    out["f_k"] = m["k_signal"]
    out["f_abs"] = m["absorption_top"]
    out["f_entropy"] = m["spec_entropy"]
    out["f_prv1"] = m["pr_v1"]
    out["f_xdisp"] = m["corr_xdisp"]
    out["f_dlam1_21"] = m["lam1_share"].diff(D21)
    out["f_dlam1_63"] = m["lam1_share"].diff(63)
    out["f_rot21"] = m["rotation"].rolling(D21, min_periods=10).mean()
    out["f_turb"] = m["turb"]
    out["f_turb21"] = m["turb"].rolling(D21, min_periods=10).mean()
    # B. vol structure
    out["f_iv_spx"] = m["iv_atm"]
    out["f_div_21"] = m["iv_atm"].diff(D21)
    out["f_vix"] = m["vix"] / 100.0                  # VIX in decimal units
    out["f_term_slope"] = m["f_term_slope"]
    out["f_skew50"] = m["iv_put_50"] - m["iv_call_50"]
    out["f_volofvol"] = m["iv_atm"].rolling(D21, min_periods=15).std()
    # C. realised & VRP
    out["f_rv21"] = m["f_rv21"]
    out["f_rv63"] = m["f_rv63"]
    out["f_vrp"] = m["iv_atm"] - m["f_rv63"]
    out["f_dd252"] = m["f_dd252"]
    out["f_mom63"] = m["f_mom63"]
    # D. IV cross-section
    out["f_iv_comp"] = m["f_iv_comp"]
    out["f_iv_xdisp"] = m["f_iv_xdisp"]
    # E. correlation levels / signal / cost
    out["f_rho_imp"] = m["rho_implied"]
    out["f_drho_imp_21"] = m["rho_implied"].diff(D21)   # the market starting to price
     stress
    out["f_rho_trail63"] = m["rho_trail63"]
    out["f_rho_clean252"] = m["rho_rmt_clean"]
    out["f_drho_21"] = m["rho_trail63"].diff(D21)
    out["f_sig_base"] = m["f_sig_base"]
    out["f_sig_rmt"] = m["f_sig_rmt"]
    out["f_anchor_gap"] = m["rho_trail63"] - m["rho_rmt_clean"]
    out["f_sig_rmt_pct"] = (m["f_sig_rmt"].expanding(min_periods=252)
                            .rank(pct=True))          # expanding percentile, no look-
    ahead
    year = m["date"].dt.year
    out["f_cost_era"] = [
```

```
        0.5 * (parametric_spread(y, "spx")
               + 0.7 * parametric_spread(y, "large") + 0.3 * parametric_spread(y, "
    small"))
        for y in year
    ]
    # label uses the future, so it's training-only and purged in every fit
    out["y_spike"] = m["rho_trail63"].shift(-HORIZON) - m["rho_trail63"]

    # force float64 everywhere -- nullable dtypes break numpy/sklearn
    num_cols = [c for c in out.columns if c != "date"]
    out[num_cols] = out[num_cols].astype("float64")

    if out_file:
        out.to_parquet(os.path.join(processed_dir, out_file), index=False)
    return out
```

Listing 14: src/dispersion/ml/regime.py

```
"""
Daily danger-regime probability from a GMM and a filtered HMM on the core stress
features, both refit walk-forward at each rebalance. The danger state is named
from the training window only, and the HMM uses the filtered (forward-pass)
posterior rather than the smoothed one, so nothing looks ahead. The feature is
the mean of the two probabilities. Causality is checked in tests/test_regime.py.

    from dispersion.ml.regime import build_regime_feature
    build_regime_feature()      # adds regime probs to ml_dataset.parquet
"""
import os
import warnings

import numpy as np
import pandas as pd
from hmmlearn.hmm import GaussianHMM
from scipy.special import logsumexp
from sklearn.mixture import GaussianMixture

CORE = ["f_lam1", "f_dlam1_63", "f_turb21", "f_vix"]
# every core feature is oriented "higher = more stress", so we name the danger
# state by the highest composite centroid instead of any single axis (turbulence
# alone doesn't work -- EWMA devol flattens it)
WARMUP = 756                        # ~3y of complete core rows before the first fit
N_STATES = 3


def hmm_filtered_posterior(model: GaussianHMM, X: np.ndarray) -> np.ndarray:
    """
    Filtered posterior P(state_t | obs_1..t) via a forward pass -- at each t it
    only uses observations up to t, never later ones. Returns (T, K).
    """
    logB = model._compute_log_likelihood(X)          # (T, K) log emission probs
```

```
    log_pi = np.log(model.startprob_ + 1e-300)
    log_A = np.log(model.transmat_ + 1e-300)
    T, K = logB.shape
    log_alpha = np.empty((T, K))
    log_alpha[0] = log_pi + logB[0]
    log_alpha[0] -= logsumexp(log_alpha[0])          # normalise (filtered)
    for t in range(1, T):
        log_alpha[t] = logsumexp(log_alpha[t - 1][:, None] + log_A, axis=0) + logB[t]
        log_alpha[t] -= logsumexp(log_alpha[t])
    return np.exp(log_alpha)


def _danger_index(centroids: np.ndarray) -> int:
    """
    Danger = the component/state whose centroid has the highest mean across the
    core features. Centroids are already in training z-score units, so a plain
    mean is comparable across features.
    """
    return int(np.argmax(centroids.mean(axis=1)))


def regime_probs(feat: pd.DataFrame, rebals, core=CORE, warmup: int = WARMUP,
                 seed: int = 0):
    """
    Walk-forward danger probabilities for a core feature subset. `feat` is
    date-indexed with at least the `core` columns. Returns (p_gmm, p_hmm) aligned
    to feat.index -- used for both the full-core feature and the VIX-only variant.
    """
    feat = feat[core]
    p_gmm = pd.Series(np.nan, index=feat.index)
    p_hmm = pd.Series(np.nan, index=feat.index)

    for k, reb in enumerate(rebals):
        reb = pd.Timestamp(reb)
        nxt = pd.Timestamp(rebals[k + 1]) if k + 1 < len(rebals) \
            else feat.index[-1] + pd.Timedelta(days=1)

        train = feat.loc[:reb].dropna()
        if len(train) < warmup:
            continue

        mu, sd = train.mean(), train.std().replace(0, 1.0)
        Ztr = ((train - mu) / sd).to_numpy(dtype="float64")

        gmm = GaussianMixture(N_STATES, covariance_type="full", random_state=seed,
                              reg_covar=1e-4).fit(Ztr)
        gd = _danger_index(gmm.means_)
        hmm = GaussianHMM(N_STATES, covariance_type="full", random_state=seed,
                          n_iter=100, tol=1e-3)
        hmm.fit(Ztr)
        hd = _danger_index(hmm.means_)
```

```
85
86          seg = feat.loc[reb:nxt].iloc[:-1] if nxt in feat.index else feat.loc[reb:nxt]
87          seg = seg.dropna()
88          if seg.empty:
89              continue
90          Zseg = ((seg - mu) / sd).to_numpy(dtype="float64")
91          p_gmm.loc[seg.index] = gmm.predict_proba(Zseg)[:, gd]
92
93          hist = feat.loc[:reb].dropna()
94          combo = pd.concat([hist, seg[~seg.index.isin(hist.index)]])
95          Zc = ((combo - mu) / sd).to_numpy(dtype="float64")
96          post = hmm_filtered_posterior(hmm, Zc)
97          p_hmm.loc[seg.index] = pd.Series(post[:, hd], index=combo.index).loc[seg.index
        ].to_numpy()
98
99      return p_gmm, p_hmm
100
101
102 def build_regime_feature(
103     processed_dir: str = "data/processed",
104     core=CORE,
105     warmup: int = WARMUP,
106     out_file: str | None = "ml_dataset.parquet",
107     seed: int = 0,
108 ) -> pd.DataFrame:
109     """Walk-forward causal danger-regime probabilities, merged into ml_dataset."""
110     ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
111     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
112     ml["date"] = pd.to_datetime(ml["date"])
113     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
114
115     feat = ml[["date"] + core].set_index("date")
116     rebals = sorted(weights["rebalance_date"].unique())
117     p_gmm, p_hmm = regime_probs(feat, rebals, core=core, warmup=warmup, seed=seed)
118
119     out = ml.copy()
120     out["f_reg_gmm"] = p_gmm.to_numpy()
121     out["f_reg_hmm"] = p_hmm.to_numpy()
122     with warnings.catch_warnings():
123         # warm-up rows are NaN in both models -> nanmean warns on those; ignore it
124         warnings.simplefilter("ignore", RuntimeWarning)
125         out["f_regime"] = np.nanmean(np.c_[p_gmm.to_numpy(), p_hmm.to_numpy()], axis
        =1)
126     num = [c for c in out.columns if c != "date"]
127     out[num] = out[num].astype("float64")
128
129     if out_file:
130         out.to_parquet(os.path.join(processed_dir, out_file), index=False)
131     return out
```

Listing 15: src/dispersion/ml/metamodel.py

```
"""
Predict each quarter's trade return, walk-forward and purged, then veto the
trades in the bad tail. Sits on top of the v1_rmt gate. Sample is small
(~90 quarters), so read the results with that in mind.

Usage:
    from dispersion.ml.metamodel import run_central_test
    res = run_central_test()
"""
import os

import numpy as np
import pandas as pd
from xgboost import XGBRegressor

from ..backtest.engine import exante_quantile_threshold, run_backtest
from .regime import regime_probs

HORIZON, EMBARGO = 63, 21          # trading days
MIN_TRAIN = 24                     # quarters before the first prediction (~6y)
VETO_PCTILE = 0.33                 # veto the bottom third of predicted returns

XGB_PARAMS = dict(objective="reg:squarederror", max_depth=2, n_estimators=100,
                  learning_rate=0.05, subsample=0.8, colsample_bytree=0.8,
                  min_child_weight=5, reg_lambda=1.0, random_state=0)

FEATURE_SETS = {
    "A_vix":       ["f_vix", "f_term_slope", "f_regime_vix"],
    "B_spectral":  ["f_vix", "f_term_slope", "f_regime_vix",
                    "f_lam1", "f_dlam1_63", "f_turb21", "f_k", "f_rot21", "f_regime"],
    "Full":        ["f_vix", "f_term_slope", "f_lam1", "f_dlam1_63", "f_turb21",
                    "f_k", "f_rot21", "f_vrp", "f_drho_imp_21", "f_sig_rmt", "f_regime
    "],
}


def _snapshots(processed_dir):
    """Features at each rebalance close, joined to the quarter's v0 return as the
    label."""
    ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
    ml["date"] = pd.to_datetime(ml["date"])
    q0 = pd.read_parquet(os.path.join(processed_dir, "backtest_v0_quarterly.parquet"))
    q0["rebalance_date"] = pd.to_datetime(q0["rebalance_date"])

    # VIX-only regime variant, fit on vol features only so the A feature set stays
    pure
    weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
    weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
    rebals = sorted(weights["rebalance_date"].unique())
```

```
    feat_vix = ml.set_index("date")[["f_vix", "f_term_slope"]]
    pg, ph = regime_probs(feat_vix, rebals, core=["f_vix", "f_term_slope"])
    ml["f_regime_vix"] = np.nanmean(np.c_[pg.to_numpy(), ph.to_numpy()], axis=1)

    snap = ml.merge(q0[["rebalance_date", "ret_q"]].rename(
        columns={"rebalance_date": "date", "ret_q": "y"}), on="date", how="inner")
    return snap.sort_values("date").reset_index(drop=True)


def walk_forward_predict(snap: pd.DataFrame, features: list) -> pd.Series:
    """
    Walk-forward prediction of y at each rebalance. Only trains on quarters whose
    label horizon plus embargo ends before the test date, so no future leaks in.
    """
    dates = snap["date"].to_numpy()
    yhat = pd.Series(np.nan, index=snap.index)
    for i in range(len(snap)):
        test_date = snap["date"].iloc[i]
        cutoff = test_date - pd.Timedelta(days=int((HORIZON + EMBARGO) * 7 / 5))  #
    trading days -> calendar
        train = snap.iloc[:i][snap["date"].iloc[:i] <= cutoff]
        train = train.dropna(subset=features + ["y"])
        if len(train) < MIN_TRAIN:
            continue
        Xtr, ytr = train[features].to_numpy("float64"), train["y"].to_numpy("float64")
        xte = snap[features].iloc[[i]]
        if xte.isna().any(axis=None):
            continue
        model = XGBRegressor(**XGB_PARAMS)
        model.fit(Xtr, ytr)
        yhat.iloc[i] = float(model.predict(xte.to_numpy("float64"))[0])
    return yhat


def _veto_mask(yhat: pd.Series) -> pd.Series:
    """Veto quarter i if its prediction is below the expanding VETO_PCTILE of past
    predictions."""
    veto = pd.Series(False, index=yhat.index)
    hist = []
    for i in range(len(yhat)):
        yi = yhat.iloc[i]
        if np.isfinite(yi) and len(hist) >= MIN_TRAIN:
            veto.iloc[i] = yi < np.quantile(hist, VETO_PCTILE)
        if np.isfinite(yi):
            hist.append(yi)
    return veto


def predictive_metrics(snap, yhat, crisis_q=0.10):
    """Directional accuracy, plus precision and recall of the veto on crisis quarters.
    """
```

```
    m = snap.assign(yhat=yhat.to_numpy()).dropna(subset=["yhat", "y"])
    if len(m) < MIN_TRAIN:
        return {}
    crisis = m["y"] <= m["y"].quantile(crisis_q)          # worst quarters ex-post,
    used only to score the veto
    veto = _veto_mask(pd.Series(m["yhat"].to_numpy(), index=m.index)).to_numpy()
    tp = int((veto & crisis.to_numpy()).sum())
    fp = int((veto & ~crisis.to_numpy()).sum())
    fn = int((~veto & crisis.to_numpy()).sum())
    return {
        "n_pred": int(len(m)),
        "corr_yhat_y": float(np.corrcoef(m["yhat"], m["y"])[0, 1]),
        "dir_acc": float((np.sign(m["yhat"]) == np.sign(m["y"])).mean()),
        "n_crisis": int(crisis.sum()),
        "recall_crisis": tp / (tp + fn) if tp + fn else np.nan,
        "precision_veto": tp / (tp + fp) if tp + fp else np.nan,
    }


def feature_importance(snap, features):
    """Mean XGBoost gain importance across a few expanding folds."""
    imp = np.zeros(len(features))
    n = 0
    for cut in (0.5, 0.7, 0.9):
        k = int(len(snap) * cut)
        tr = snap.iloc[:k].dropna(subset=features + ["y"])
        if len(tr) < MIN_TRAIN:
            continue
        m = XGBRegressor(**XGB_PARAMS).fit(tr[features].to_numpy("float64"),
                                           tr["y"].to_numpy("float64"))
        imp += m.feature_importances_
        n += 1
    imp = imp / max(n, 1)
    return dict(sorted(zip(features, imp.tolist()), key=lambda kv: -kv[1]))


def run_central_test(processed_dir: str = "data/processed"):
    """Run predictions, veto, and gross+net backtests for each of the A/B/Full feature
     sets."""
    snap = _snapshots(processed_dir)
    signal_rmt = pd.read_parquet(os.path.join(processed_dir, "signal_rmt.parquet"))
    weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
    signal_rmt["date"] = pd.to_datetime(signal_rmt["date"])
    weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
    rebals = sorted(weights["rebalance_date"].unique())
    base = {k: pd.read_parquet(os.path.join(processed_dir, f"{k}.parquet"))
            for k in ("surface", "spots", "rates", "weights")}
    v1rmt_thr = exante_quantile_threshold(signal_rmt, rebals, q=0.5, warmup=12)

    out = {"snap_dates": snap["date"], "y": snap["y"], "arms": {}}
    for name, feats in FEATURE_SETS.items():
```

```
        yhat = walk_forward_predict(snap, feats)
        veto = _veto_mask(yhat)
        veto_dates = set(snap.loc[veto.to_numpy(), "date"])
        thr = v1rmt_thr.copy()
        for d in veto_dates:
            thr[pd.Timestamp(d)] = 1e9                    # unreachable threshold ->
    skip the quarter
        res = {}
        for tag, kw in [("gross", {}), ("net", dict(costs={}))]:
            bt = run_backtest(data={**{k: v.copy() for k, v in base.items()},
                                    "signal": signal_rmt.copy()}, threshold=thr, **kw)
            bt["quarterly"].to_parquet(
                os.path.join(processed_dir, f"backtest_ml_{name}_{tag}_quarterly.
    parquet"), index=False)
            bt["daily"].to_parquet(
                os.path.join(processed_dir, f"backtest_ml_{name}_{tag}_daily.parquet"),
     index=False)
            res[tag] = bt["quarterly"]
        out["arms"][name] = {
            "yhat": yhat, "veto": veto, "n_veto": int(veto.sum()),
            "pred": predictive_metrics(snap, yhat),
            "importance": feature_importance(snap, feats),
        }
    return out
```

Listing 16: src/dispersion/ml/experiments.py

```
"""
Two experimental vetoes that sit alongside the main ML, neither overwrites it.

Lever 1 predicts the forward correlation spike on daily data and vetoes a trade
when the spike predicted at the rebalance is in the high tail. Lever 3 skips the
model entirely and vetoes on the regime probability instead. Both layer on top
of the v1_rmt gate.

Usage:
    from dispersion.ml.experiments import run_levers
    run_levers()      # writes backtest_regonly_* and backtest_spike_* + a summary
"""
import json
import os

import numpy as np
import pandas as pd
from xgboost import XGBRegressor

from ..backtest.engine import exante_quantile_threshold, run_backtest

HORIZON, EMBARGO = 63, 21
MIN_TRAIN_DAYS = 756
VETO_Q = 0.67                       # veto the top third of predicted danger
```

```
25
26 SPIKE_FEATS = ["f_vix", "f_term_slope", "f_lam1", "f_dlam1_63", "f_turb21", "f_k",
27                "f_rot21", "f_vrp", "f_drho_imp_21", "f_sig_rmt", "f_regime"]
28 XGB = dict(objective="reg:squarederror", max_depth=3, n_estimators=200,
29            learning_rate=0.05, subsample=0.8, colsample_bytree=0.8,
30            min_child_weight=20, reg_lambda=1.0, random_state=0)
31
32
33 def predict_spike_daily(processed_dir="data/processed", features=SPIKE_FEATS):
34     """
35     Walk-forward daily spike prediction. Refit at each rebalance on the purged
36     expanding history, then predict that quarter's days. Returns the prediction at
37     each rebalance and oos_corr, the correlation between predicted and realised
38     spike over all out-of-sample days (tells you whether the spike is predictable).
39     """
40     ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
41     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
42     ml["date"] = pd.to_datetime(ml["date"])
43     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
44     rebals = sorted(weights["rebalance_date"].unique())
45     idx = ml.set_index("date")
46
47     purge_cal = int((HORIZON + EMBARGO) * 7 / 5)
48     yhat_reb = {}
49     oos_pred, oos_true = [], []
50     for k, reb in enumerate(rebals):
51         reb = pd.Timestamp(reb)
52         nxt = pd.Timestamp(rebals[k + 1]) if k + 1 < len(rebals) else idx.index[-1] +
    pd.Timedelta(days=1)
53         cutoff = reb - pd.Timedelta(days=purge_cal)
54         tr = ml[ml["date"] <= cutoff].dropna(subset=features + ["y_spike"])
55         if len(tr) < MIN_TRAIN_DAYS:
56             continue
57         m = XGBRegressor(**XGB).fit(tr[features].to_numpy("float64"),
58                                     tr["y_spike"].to_numpy("float64"))
59         # value used by the gate at the rebalance
60         row = idx.loc[[reb], features] if reb in idx.index else None
61         if row is not None and row.notna().all(axis=1).iloc[0]:
62             yhat_reb[reb] = float(m.predict(row.to_numpy("float64"))[0])
63         # out-of-sample check on the quarter's days
64         seg = ml[(ml["date"] >= reb) & (ml["date"] < nxt)].dropna(subset=features + ["
    y_spike"])
65         if not seg.empty:
66             p = m.predict(seg[features].to_numpy("float64"))
67             oos_pred.extend(p.tolist())
68             oos_true.extend(seg["y_spike"].tolist())
69     oos_corr = float(np.corrcoef(oos_pred, oos_true)[0, 1]) if len(oos_pred) > 10 else
     np.nan
70     return yhat_reb, oos_corr
71
72
```

```python
def _high_veto(vals_at_reb: dict, q=VETO_Q, warmup=MIN_TRAIN_DAYS // 21) -> set:
    """Veto rebalances whose value is above the expanding q-quantile of past values.""
    "
    dates = sorted(vals_at_reb)
    veto, hist = set(), []
    for d in dates:
        v = vals_at_reb[d]
        if len(hist) >= warmup and np.isfinite(v) and v > np.quantile(hist, q):
            veto.add(pd.Timestamp(d))
        if np.isfinite(v):
            hist.append(v)
    return veto


def _stats(q, d):
    r, dr = q["ret_q"].astype(float), d["ret"].astype(float)
    cum = (1 + dr).cumprod()
    return dict(trades=int(q["traded"].sum()), n=len(q), sharpe=r.mean() / r.std() * 2,

                skew=float(r.skew()), maxDD=float((cum / cum.cummax() - 1).min()),
                cumul=float(cum.iloc[-1]), worst=float(r.min()))


def run_levers(processed_dir="data/processed"):
    """Backtest lever 1 (spike) and lever 3 (regime-only) against v1_rmt, and write
    the outputs."""
    ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
    signal_rmt = pd.read_parquet(os.path.join(processed_dir, "signal_rmt.parquet"))
    weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
    ml["date"] = pd.to_datetime(ml["date"])
    signal_rmt["date"] = pd.to_datetime(signal_rmt["date"])
    weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
    rebals = sorted(weights["rebalance_date"].unique())
    base = {kk: pd.read_parquet(os.path.join(processed_dir, f"{kk}.parquet"))
            for kk in ("surface", "spots", "rates", "weights")}
    v1rmt_thr = exante_quantile_threshold(signal_rmt, rebals, q=0.5, warmup=12)

    # lever 3: regime probability at each rebalance
    reg_at = {pd.Timestamp(r): float(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].
    iloc[0])
              for r in rebals if (ml["date"] == pd.Timestamp(r)).any()
              and np.isfinite(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].iloc
    [0])}
    veto_reg = _high_veto(reg_at)

    # lever 1: daily spike predictor
    yhat_reb, oos_corr = predict_spike_daily(processed_dir)
    veto_spike = _high_veto(yhat_reb)

    out = {"oos_corr_spike": oos_corr, "n_veto_regime": len(veto_reg),
           "n_veto_spike": len(veto_spike), "arms": {}}
```

```
    for name, veto in [("regonly", veto_reg), ("spike", veto_spike)]:
        thr = v1rmt_thr.copy()
        for d in veto:
            thr[pd.Timestamp(d)] = 1e9
        for tag, kw in [("gross", {}), ("net", dict(costs={}))]:
            bt = run_backtest(data={**{k: v.copy() for k, v in base.items()},
                                    "signal": signal_rmt.copy()}, threshold=thr, **kw)
            bt["quarterly"].to_parquet(
                os.path.join(processed_dir, f"backtest_{name}_{tag}_quarterly.parquet"
), index=False)
            bt["daily"].to_parquet(
                os.path.join(processed_dir, f"backtest_{name}_{tag}_daily.parquet"),
index=False)
            out["arms"].setdefault(name, {})[tag] = _stats(bt["quarterly"], bt["daily"
])
    json.dump(out, open(os.path.join(processed_dir, "ml_levers_summary.json"), "w"),
indent=2)
    return out
```

G.7 Shared utilities (src/dispersion/utils/)

Listing 17: src/dispersion/utils/greeks.py

```
"""
Black-Scholes / Black-76 prices and greeks, with per-contract <-> relative
conversions. sigma is annualised IV in decimals, T in years, r and q continuous.

Relative greeks are per dollar invested (greek / price) -- that's the ratio the
vega hedge uses; mixing per-contract greeks with wealth weights gives a wrong hedge.

Usage:
    from dispersion.utils.greeks import bs_greeks, relative_greeks, implied_forward
"""
import numpy as np
from scipy.optimize import brentq
from scipy.stats import norm

GREEK_KEYS = ("price", "delta", "gamma", "vega", "theta", "volga", "vanna")


# ---------------------------------------------------------------------------- #
# Pricing
# ---------------------------------------------------------------------------- #
def _d1_d2(S, K, sigma, T, r, q):
    st = sigma * np.sqrt(T)
    d1 = (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / st
    return d1, d1 - st


def bs_price(S, K, sigma, T, r, q, cp: str) -> float:
    """European option price, spot form with continuous dividend yield q."""
```

```
    d1, d2 = _d1_d2(S, K, sigma, T, r, q)
    if cp == "C":
        return S * np.exp(-q * T) * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    if cp == "P":
        return K * np.exp(-r * T) * norm.cdf(-d2) - S * np.exp(-q * T) * norm.cdf(-d1)
    raise ValueError(f"cp must be 'C' or 'P', got {cp!r}")


def black76_price(F, K, sigma, T, r, cp: str) -> float:
    """European option price, forward form (Black-76): discounts the forward payoff.""
    "
    # same as the spot form with S = F*exp(-(r-q)*T)
    st = sigma * np.sqrt(T)
    d1 = (np.log(F / K) + 0.5 * sigma**2 * T) / st
    d2 = d1 - st
    df = np.exp(-r * T)
    if cp == "C":
        return df * (F * norm.cdf(d1) - K * norm.cdf(d2))
    if cp == "P":
        return df * (K * norm.cdf(-d2) - F * norm.cdf(-d1))
    raise ValueError(f"cp must be 'C' or 'P', got {cp!r}")


# ---------------------------------------------------------------------------- #
# Greeks (per contract)
# ---------------------------------------------------------------------------- #
def bs_greeks(S, K, sigma, T, r, q, cp: str) -> dict:
    """
    Price plus first- and second-order greeks per contract, spot form.
    Returns dict(price, delta, gamma, vega, theta, volga, vanna).

    volga and vanna carry the book's convexity risk:
      volga = d(vega)/d(sigma) = vega * d1 * d2 / sigma
      vanna = d(vega)/d(S)     = d(delta)/d(sigma)
    Both vanish at the ATM-forward and grow as a straddle drifts off ATM.
    """
    d1, d2 = _d1_d2(S, K, sigma, T, r, q)
    eq, er = np.exp(-q * T), np.exp(-r * T)
    pdf1 = norm.pdf(d1)

    gamma = eq * pdf1 / (S * sigma * np.sqrt(T))
    vega = S * eq * pdf1 * np.sqrt(T)
    volga = vega * d1 * d2 / sigma
    vanna = -eq * pdf1 * d2 / sigma
    common_theta = -S * eq * pdf1 * sigma / (2.0 * np.sqrt(T))

    if cp == "C":
        price = S * eq * norm.cdf(d1) - K * er * norm.cdf(d2)
        delta = eq * norm.cdf(d1)
        theta = common_theta - r * K * er * norm.cdf(d2) + q * S * eq * norm.cdf(d1)
    elif cp == "P":
```

```
        price = K * er * norm.cdf(-d2) - S * eq * norm.cdf(-d1)
        delta = -eq * norm.cdf(-d1)
        theta = common_theta + r * K * er * norm.cdf(-d2) - q * S * eq * norm.cdf(-d1)
    else:
        raise ValueError(f"cp must be 'C' or 'P', got {cp!r}")

    return {"price": float(price), "delta": float(delta), "gamma": float(gamma),
            "vega": float(vega), "theta": float(theta),
            "volga": float(volga), "vanna": float(vanna)}


def straddle_greeks(S, K_call, K_put, sigma_call, sigma_put, T, r, q) -> dict:
    """Greeks of a standardised straddle = call + put legs (strikes may differ slightly)."""
    c = bs_greeks(S, K_call, sigma_call, T, r, q, "C")
    p = bs_greeks(S, K_put, sigma_put, T, r, q, "P")
    return {k: c[k] + p[k] for k in GREEK_KEYS}


# ---------------------------------------------------------------------------- #
# Per-contract <-> relative (per dollar invested)
# ---------------------------------------------------------------------------- #
def relative_greeks(greeks: dict) -> dict:
    """
    Per-contract greeks -> per-dollar-invested greeks (greek / price).

    These are what go into the wealth-weighted hedge ratio
    y_i = vega_rel_index / vega_rel_component; per-contract vegas with wealth
    weights give the wrong hedge.
    """
    price = greeks["price"]
    if price <= 0:
        raise ValueError("relative greeks undefined for non-positive price")
    out = {k: greeks[k] / price for k in GREEK_KEYS if k != "price"}
    out["price"] = price
    return out


# ---------------------------------------------------------------------------- #
# Implied quantities
# ---------------------------------------------------------------------------- #
def implied_forward(call_price, put_price, K, T, r) -> float:
    """Model-free forward from put-call parity at a common strike: F = K + e^{rT}(C - P)."""
    return float(K + np.exp(r * T) * (call_price - put_price))


def implied_vol(price, S, K, T, r, q, cp: str, lo=1e-4, hi=5.0) -> float:
    """Invert bs_price for sigma (brentq)."""
    return float(brentq(lambda s: bs_price(S, K, s, T, r, q, cp) - price, lo, hi))
```

G.8 Tests (tests/)

Listing 18: tests/test_greeks.py

```python
"""
Tests for dispersion.utils.greeks: closed-form values and identities, greeks vs
finite differences, the relative-vs-per-contract vega hedge, and reproducing real
SPX premiums.
"""
import numpy as np
import pytest
from scipy.stats import norm

from dispersion.utils.greeks import (
    bs_greeks,
    bs_price,
    black76_price,
    implied_forward,
    implied_vol,
    relative_greeks,
    straddle_greeks,
)

# non-trivial base case: ITM-ish call, with dividends and rates
BASE = dict(S=100.0, K=95.0, sigma=0.25, T=0.25, r=0.04, q=0.015)


# ---------------------------------------------------------------------------- #
# 1. Closed form & identities
# ---------------------------------------------------------------------------- #
def test_atm_forward_closed_form():
    # BS price against its exact closed form when r = q = 0 and K = S
    S = K = 100.0
    sigma, T = 0.20, 91 / 365
    d = 0.5 * sigma * np.sqrt(T)
    expected_call = S * (norm.cdf(d) - norm.cdf(-d))
    assert bs_price(S, K, sigma, T, 0.0, 0.0, "C") == pytest.approx(expected_call, rel
    =1e-12)
    # ATM-forward, so call and put match
    assert bs_price(S, K, sigma, T, 0.0, 0.0, "P") == pytest.approx(expected_call, rel
    =1e-12)


def test_put_call_parity_and_symmetries():
    rng = np.random.default_rng(42)
    for _ in range(200):
        S = rng.uniform(20, 500)
        K = S * rng.uniform(0.7, 1.3)
        sigma = rng.uniform(0.05, 0.9)
        T = rng.uniform(0.02, 2.0)
        r = rng.uniform(0.0, 0.08)
```

```
        q = rng.uniform(0.0, 0.05)
        c = bs_greeks(S, K, sigma, T, r, q, "C")
        p = bs_greeks(S, K, sigma, T, r, q, "P")
        # put-call parity: C - P = S e^{-qT} - K e^{-rT}
        assert c["price"] - p["price"] == pytest.approx(
            S * np.exp(-q * T) - K * np.exp(-r * T), abs=1e-9
        )
        # delta_C - delta_P = e^{-qT}; gamma and vega equal across the two legs
        assert c["delta"] - p["delta"] == pytest.approx(np.exp(-q * T), abs=1e-12)
        assert c["gamma"] == pytest.approx(p["gamma"], rel=1e-12)
        assert c["vega"] == pytest.approx(p["vega"], rel=1e-12)


def test_black76_equals_spot_form():
    S, K, sigma, T, r, q = BASE.values()
    F = S * np.exp((r - q) * T)
    for cp in ("C", "P"):
        assert black76_price(F, K, sigma, T, r, cp) == pytest.approx(
            bs_price(S, K, sigma, T, r, q, cp), rel=1e-12
        )


def test_implied_forward_parity():
    S, K, sigma, T, r, q = BASE.values()
    c = bs_price(S, K, sigma, T, r, q, "C")
    p = bs_price(S, K, sigma, T, r, q, "P")
    assert implied_forward(c, p, K, T, r) == pytest.approx(S * np.exp((r - q) * T),
    rel=1e-12)


def test_implied_vol_round_trip():
    S, K, sigma, T, r, q = BASE.values()
    for cp in ("C", "P"):
        price = bs_price(S, K, sigma, T, r, q, cp)
        assert implied_vol(price, S, K, T, r, q, cp) == pytest.approx(sigma, abs=1e
    -10)


def test_straddle_is_sum_of_legs():
    S, K, sigma, T, r, q = BASE.values()
    st = straddle_greeks(S, K_call=K, K_put=K + 1, sigma_call=sigma, sigma_put=sigma +
     0.01,
                         T=T, r=r, q=q)
    c = bs_greeks(S, K, sigma, T, r, q, "C")
    p = bs_greeks(S, K + 1, sigma + 0.01, T, r, q, "P")
    for k in ("price", "delta", "gamma", "vega", "theta"):
        assert st[k] == pytest.approx(c[k] + p[k], rel=1e-12)


# ---------------------------------------------------------------------------- #
# 2. Greeks vs bump-and-reprice
```

```python
# ---------------------------------------------------------------------------- #
@pytest.mark.parametrize("cp", ["C", "P"])
def test_greeks_match_finite_differences(cp):
    S, K, sigma, T, r, q = BASE.values()
    g = bs_greeks(S, K, sigma, T, r, q, cp)
    hs, h = 1e-3, 1e-5  # bigger S bump for the second derivative (cancellation noise ~1/h^2)

    delta_fd = (bs_price(S + hs, K, sigma, T, r, q, cp) - bs_price(S - hs, K, sigma, T, r, q, cp)) / (2 * hs)
    gamma_fd = (bs_price(S + hs, K, sigma, T, r, q, cp) - 2 * bs_price(S, K, sigma, T, r, q, cp)
                + bs_price(S - hs, K, sigma, T, r, q, cp)) / hs**2
    vega_fd = (bs_price(S, K, sigma + h, T, r, q, cp) - bs_price(S, K, sigma - h, T, r, q, cp)) / (2 * h)
    # theta is the derivative w.r.t. time passing, i.e. -dP/dT
    theta_fd = -(bs_price(S, K, sigma, T + h, r, q, cp) - bs_price(S, K, sigma, T - h, r, q, cp)) / (2 * h)

    assert g["delta"] == pytest.approx(delta_fd, rel=1e-7)
    assert g["gamma"] == pytest.approx(gamma_fd, rel=1e-5)
    assert g["vega"] == pytest.approx(vega_fd, rel=1e-7)
    assert g["theta"] == pytest.approx(theta_fd, rel=1e-6)

    # second order: volga = d(vega)/d(sigma), vanna = d(vega)/d(S)
    volga_fd = (bs_greeks(S, K, sigma + h, T, r, q, cp)["vega"]
                - bs_greeks(S, K, sigma - h, T, r, q, cp)["vega"]) / (2 * h)
    vanna_fd = (bs_greeks(S + hs, K, sigma, T, r, q, cp)["vega"]
                - bs_greeks(S - hs, K, sigma, T, r, q, cp)["vega"]) / (2 * hs)
    assert g["volga"] == pytest.approx(volga_fd, rel=1e-5)
    assert g["vanna"] == pytest.approx(vanna_fd, rel=1e-5)


def test_second_order_greeks_vanish_at_d2_zero_strike():
    # volga (∝ d1·d2) and vanna (∝ d2) are both zero at the strike where d2 = 0,
    # and grow as a straddle ages away from it -- that drift is the convexity risk.
    S, sigma, T, r, q = 100.0, 0.25, 0.5, 0.03, 0.01
    K0 = S * np.exp((r - q - 0.5 * sigma**2) * T)      # d2 = 0
    g0 = bs_greeks(S, K0, sigma, T, r, q, "C")
    assert abs(g0["vanna"]) < 1e-9 and abs(g0["volga"]) < 1e-9
    # spot drifted down 15%: both are clearly non-zero
    g1 = bs_greeks(S * 0.85, K0, sigma, T, r, q, "C")
    assert abs(g1["vanna"]) > 1e-3 and abs(g1["volga"]) > 1e-2


# ---------------------------------------------------------------------------- #
# 3. Relative vs per-contract hedge ratios
# ---------------------------------------------------------------------------- #
def test_relative_vega_hedge_is_wealth_neutral_and_per_contract_is_not():
    T, r, q = 0.25, 0.04, 0.0
    # expensive index-like straddle vs cheap single-name-like straddle
```

```
    idx = straddle_greeks(S=5000.0, K_call=5000.0, K_put=5000.0,
                          sigma_call=0.12, sigma_put=0.12, T=T, r=r, q=q)
    comp = straddle_greeks(S=50.0, K_call=50.0, K_put=50.0,
                           sigma_call=0.30, sigma_put=0.30, T=T, r=r, q=q)
    idx_rel, comp_rel = relative_greeks(idx), relative_greeks(comp)

    # hedge ratio from the relative vegas
    y = idx_rel["vega"] / comp_rel["vega"]
    # short $1 of index, long $y of component -> zero wealth-vega
    wealth_vega = -1.0 * idx_rel["vega"] + y * comp_rel["vega"]
    assert wealth_vega == pytest.approx(0.0, abs=1e-12)

    # using the per-contract vega ratio instead leaves a large residual
    y_trap = idx["vega"] / comp["vega"]
    wealth_vega_trap = -1.0 * idx_rel["vega"] + y_trap * comp_rel["vega"]
    assert abs(wealth_vega_trap) > 0.1 * idx_rel["vega"]  # off by a lot, not rounding

    # the two ratios differ by exactly the price ratio
    assert y_trap / y == pytest.approx(idx["price"] / comp["price"], rel=1e-12)


# ---------------------------------------------------------------------------- #
# 4. Real data: SPX standardised 91d ±50δ, 2024-06-28
# ---------------------------------------------------------------------------- #
def test_reproduces_spx_impl_premium_2024():
    S, T, r = 5460.48, 91 / 365, 0.0550  # secprd close, zerocd 91d
    q = 0.0104                           # implied dividend yield, same for both legs
    call = bs_price(S, 5530.623, 0.121323, T, r, q, "C")
    put = bs_price(S, 5531.409, 0.113183, T, r, q, "P")
    assert call == pytest.approx(127.2951, rel=5e-3)
    assert put == pytest.approx(127.7991, rel=5e-3)
```

Listing 19: tests/test_marking.py

```
"""
Tests for dispersion.backtest.marking: the mark-to-surface rules.
"""
import numpy as np
import pandas as pd
import pytest

from dispersion.backtest.marking import RateCurve, adjust_strike, interp_sigma


# ---------------------------------------------------------------------------- #
# interp_sigma -- total-variance interpolation + frozen short end
# ---------------------------------------------------------------------------- #
def test_exact_at_pillars():
    s30, s60, s91 = 0.30, 0.25, 0.22
    assert interp_sigma(s30, s60, s91, 30) == pytest.approx(s30, rel=1e-12)
    assert interp_sigma(s30, s60, s91, 60) == pytest.approx(s60, rel=1e-12)
```

```python
    assert interp_sigma(s30, s60, s91, 91) == pytest.approx(s91, rel=1e-12)


def test_linear_in_total_variance_between_pillars():
    s30, s60, s91 = 0.30, 0.25, 0.22
    # tau = 45 sits halfway between the 30 and 60 pillars in total variance
    w30, w60 = s30**2 * 30, s60**2 * 60
    expected = np.sqrt(((w30 + w60) / 2) / 45)
    assert interp_sigma(s30, s60, s91, 45) == pytest.approx(expected, rel=1e-12)
    # tau = 75.5, midpoint of [60, 91]
    w91 = s91**2 * 91
    expected = np.sqrt(((w60 + w91) / 2) / 75.5)
    assert interp_sigma(s30, s60, s91, 75.5) == pytest.approx(expected, rel=1e-12)


def test_frozen_below_30d():
    s30, s60, s91 = 0.30, 0.25, 0.22
    for tau in (29.9, 15, 5, 1):
        assert interp_sigma(s30, s60, s91, tau) == pytest.approx(s30, rel=1e-12)


def test_vectorised_and_nan_propagation():
    s30 = np.array([0.30, np.nan, 0.40])
    s60 = np.array([0.25, 0.25, 0.35])
    s91 = np.array([0.22, 0.22, 0.33])
    out = interp_sigma(s30, s60, s91, np.array([45.0, 45.0, 20.0]))
    assert out.shape == (3,)
    assert np.isnan(out[1])            # a NaN pillar propagates to the output
    assert out[2] == pytest.approx(0.40)  # short end frozen element by element


# ---------------------------------------------------------------------------- #
# RateCurve -- maturity interpolation + bounded date-ffill
# ---------------------------------------------------------------------------- #
def _toy_rates():
    return pd.DataFrame({
        "date": pd.to_datetime(["2020-01-02"] * 3 + ["2020-01-06"] * 3),
        "days": [30, 91, 182] * 2,
        "rate": [1.0, 2.0, 3.0, 1.5, 2.5, 3.5],   # percent
    })


def test_rate_exact_and_interpolated():
    rc = RateCurve(_toy_rates())
    assert rc.rate("2020-01-02", 91) == pytest.approx(0.02, abs=1e-12)
    # midpoint of [30, 91]
    assert rc.rate("2020-01-02", 60.5) == pytest.approx(0.015, abs=1e-12)
    # off the ends of the grid -> clamped
    assert rc.rate("2020-01-02", 10) == pytest.approx(0.01, abs=1e-12)
    assert rc.rate("2020-01-02", 400) == pytest.approx(0.03, abs=1e-12)

```

```

def test_rate_bounded_ffill_and_loud_failure():
    rc = RateCurve(_toy_rates(), max_stale_days=7)
    # 01-03 has no curve, so reuse 01-02 (1 day stale, within bound)
    assert rc.rate("2020-01-03", 91) == pytest.approx(0.02, abs=1e-12)
    # past the staleness bound -> raise rather than use a stale curve
    with pytest.raises(KeyError):
        rc.rate("2020-02-01", 91)
    # before the first curve -> raise
    with pytest.raises(KeyError):
        rc.rate("2019-12-31", 91)


# ---------------------------------------------------------------------------- #
# adjust_strike -- split handling via cfadj
# ---------------------------------------------------------------------------- #
def test_adjust_strike_split():
    # 2:1 split: cfadj goes 1 -> 2, price halves, so the strike halves too
    assert adjust_strike(100.0, 1.0, 2.0) == pytest.approx(50.0)
    # no split leaves the strike alone
    assert adjust_strike(100.0, 1.0, 1.0) == pytest.approx(100.0)
    # once both are rebased, |S - K| is unchanged by the split
    S_pre, K_pre = 120.0, 100.0
    S_post, cf_e, cf_t = 60.0, 1.0, 2.0
    K_post = adjust_strike(K_pre, cf_e, cf_t)
    qty_scale = cf_t / cf_e
    assert qty_scale * abs(S_post - K_post) == pytest.approx(abs(S_pre - K_pre))
```

Listing 20: tests/test_engine.py

```
"""
Tests for dispersion.backtest.engine on a synthetic quarter with a closed-form
answer. With r = 0, flat sigmas, constant spots, ATM strikes and q = 0, the
settlement payoffs are 0 and daily hedging is cash-neutral, so the quarterly
return is exactly 1 - sum(y_i). Sizing identities are checked the same way.
"""
import numpy as np
import pandas as pd
import pytest

from dispersion.backtest.engine import (SPX_SECID, exante_quantile_threshold,
                                        implied_q, run_backtest)
from dispersion.utils.greeks import bs_greeks, bs_price

REB = pd.Timestamp("2020-03-31")
NXT = pd.Timestamp("2020-06-30")           # 91 calendar days
T0 = (NXT - REB).days / 365.0
NAMES = {1: dict(S=100.0, w=0.6), 2: dict(S=50.0, w=0.4)}
SPX_S = 1000.0


```

```python
def _make_data(sig_by_secid):
    dates = pd.DatetimeIndex([REB, "2020-04-30", "2020-05-29", "2020-06-29"])
    surf, spot, rate, wgt = [], [], [], []
    all_ids = {SPX_SECID: SPX_S, **{s: NAMES[s]["S"] for s in NAMES}}

    for secid, S in all_ids.items():
        sig = sig_by_secid[secid]
        prem = {cp: bs_price(S, S, sig, T0, 0.0, 0.0, cp) for cp in ("C", "P")}
        for d in dates:
            for days in (30, 60, 91):
                for cp in ("C", "P"):
                    surf.append((REB, d, secid, days, cp, sig, prem[cp], S))
            spot.append((REB, d, secid, S, 1.0))
    # settlement rows live in the next quarter's partition (date == NXT)
    for secid, S in all_ids.items():
        spot.append((NXT, NXT, secid, S, 1.0))

    for d in list(dates) + [NXT]:
        for days in (10, 30, 60, 91, 122):
            rate.append((d, days, 0.0))

    for rnk, (secid, info) in enumerate(NAMES.items(), start=1):
        wgt.append((REB, secid, info["w"], rnk))
        wgt.append((NXT, secid, info["w"], rnk))   # gives NXT a rebalance so
    settlement fires

    return {
        "surface": pd.DataFrame(surf, columns=["rebalance_date", "date", "secid",
                                               "days", "cp_flag", "iv", "premium",
                                               "strike"]),
        "spots": pd.DataFrame(spot, columns=["rebalance_date", "date", "secid",
                                             "close", "cfadj"]),
        "rates": pd.DataFrame(rate, columns=["date", "days", "rate"]),
        "weights": pd.DataFrame(wgt, columns=["rebalance_date", "secid", "weight", "
    rnk"]),
        "signal": pd.DataFrame({"date": [REB], "signal": [0.10]}),
    }


def test_implied_q_roundtrip():
    S, k_c, k_p, sc, sp_, T, r, q_true = 100.0, 101.0, 99.0, 0.25, 0.27, 0.25, 0.03,
    0.017
    c = bs_price(S, k_c, sc, T, r, q_true, "C")
    p = bs_price(S, k_p, sp_, T, r, q_true, "P")
    assert implied_q(S, k_c, k_p, sc, sp_, T, r, c, p) == pytest.approx(q_true, abs=1e
    -8)


def test_flat_quarter_closed_form():
    # same sigma everywhere -> nu*sigma equal across names -> lam = 1, y_sum = 1
    data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
```

```
    res = run_backtest(data=data, min_names=2)
    q = res["quarterly"].iloc[0]

    assert q["traded"] and q["n_names"] == 2
    assert q["lam"] == pytest.approx(1.0, rel=1e-10)
    assert q["y_sum"] == pytest.approx(1.0, rel=1e-10)
    # zero settlement, cash-neutral hedge, r = 0: quarterly return is 1 - y_sum
    assert q["ret_q"] == pytest.approx(1.0 - q["y_sum"], abs=1e-12)
    # compounded daily returns match the quarterly return
    d = res["daily"]
    assert (1 + d["ret"]).prod() - 1 == pytest.approx(q["ret_q"], abs=1e-12)
    assert q["avg_frozen_days"] == 0 and q["n_frozen_settle"] == 0


def test_sizing_identity_heterogeneous_vols():
    sig = {SPX_SECID: 0.18, 1: 0.35, 2: 0.22}
    data = _make_data(sig)
    res = run_backtest(data=data, min_names=2)
    q = res["quarterly"].iloc[0]

    # closed-form lam: nu = straddle vega / straddle price at entry
    def nu(S, s):
        g_c = bs_greeks(S, S, s, T0, 0.0, 0.0, "C")
        g_p = bs_greeks(S, S, s, T0, 0.0, 0.0, "P")
        return (g_c["vega"] + g_p["vega"]) / (g_c["price"] + g_p["price"])

    nu_i = nu(SPX_S, sig[SPX_SECID])
    denom = sum(NAMES[s]["w"] * nu(NAMES[s]["S"], sig[s]) * sig[s] for s in NAMES)
    lam_expected = nu_i * sig[SPX_SECID] / denom
    assert q["lam"] == pytest.approx(lam_expected, rel=1e-9)

    # wealth-vega neutrality under a proportional shock holds by construction
    lhs = sum(q["lam"] * NAMES[s]["w"] * nu(NAMES[s]["S"], sig[s]) * sig[s] for s in
    NAMES)
    assert lhs == pytest.approx(nu_i * sig[SPX_SECID], rel=1e-9)


def test_settlement_intrinsic_payoff():
    data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
    # move name 1's settlement spot so its straddle pays |120 - 100| = 20 per unit
    sp = data["spots"]
    sp.loc[(sp["rebalance_date"] == NXT) & (sp["secid"] == 1), "close"] = 120.0
    res = run_backtest(data=data, min_names=2)
    q = res["quarterly"].iloc[0]

    # flat-case return (1 - y_sum) plus name 1's settlement payoff
    price_1 = bs_price(100.0, 100.0, 0.20, T0, 0.0, 0.0, "C") + \
              bs_price(100.0, 100.0, 0.20, T0, 0.0, 0.0, "P")
    y_1 = q["lam"] * NAMES[1]["w"]
    expected = (1.0 - q["y_sum"]) + y_1 / price_1 * 20.0
    assert q["ret_q"] == pytest.approx(expected, abs=1e-12)
```

```
119
120
121 def test_parsimonious_leg_keeps_vega_neutrality():
122     # n_leg=1 keeps only the top-weight name but rescales it, so wealth-vega
123     # neutrality (y_sum ~ 1 in the flat case) and ret_q = 1 - y_sum still hold.
124     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
125     res = run_backtest(data=data, min_names=2, n_leg=1)
126     q = res["quarterly"].iloc[0]
127     assert q["n_names"] == 1                          # only the top-weight component
128     assert q["y_sum"] == pytest.approx(1.0, rel=1e-10)   # still wealth-vega neutral
129     assert q["ret_q"] == pytest.approx(1.0 - q["y_sum"], abs=1e-12)
130
131
132 def test_v1_gate_skips_quarter():
133     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
134     res = run_backtest(data=data, min_names=2, threshold=0.50)  # signal = 0.10 < 0.50
135     q = res["quarterly"].iloc[0]
136     assert not q["traded"]
137     assert q["ret_q"] == pytest.approx(0.0, abs=1e-12)          # nothing traded, r =
    0 -> flat
138
139
140 def test_costs_closed_form():
141     from dispersion.backtest.engine import parametric_spread
142     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
143     res_g = run_backtest(data=data, min_names=2)
144     res_n = run_backtest(data=data, min_names=2, costs={"hedge_bps": 0.0})
145     qg, qn = res_g["quarterly"].iloc[0], res_n["quarterly"].iloc[0]
146     # entry cost = half-spread on the index leg (100% of W) plus each component leg
147     expected = 0.5 * parametric_spread(REB.year, "spx") \
148         + 0.5 * parametric_spread(REB.year, "large") * qg["y_sum"]  # rnk 1-2 map to "
    large"
149     assert qn["cost_entry"] == pytest.approx(expected, rel=1e-10)
150     assert qn["cost_hedge"] == 0.0
151     assert qn["ret_q"] == pytest.approx(qg["ret_q"] - expected, abs=1e-12)
152
153
154 def test_exante_quantile_threshold_values():
155     sig = pd.DataFrame({"date": pd.date_range("2020-01-01", periods=100, freq="D"),
156                         "signal": np.linspace(0.0, 1.0, 100)})
157     rebs = [pd.Timestamp("2020-02-01"), pd.Timestamp("2020-04-01")]
158     thr = exante_quantile_threshold(sig, rebs, q=0.5, warmup=1)
159     assert np.isnan(thr.iloc[0])                                  # still warming up
160     expected = sig.set_index("date")["signal"].loc[:"2020-04-01"].median()
161     assert thr.iloc[1] == pytest.approx(expected, rel=1e-12)    # median of history so
     far
162
163
164 def test_v1_series_gate_warmup_trades():
165     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
166     # NaN threshold at REB (warm-up) trades like v0
```

```
    thr = pd.Series({REB: np.nan})
    res = run_backtest(data=data, min_names=2, threshold=thr)
    assert bool(res["quarterly"].iloc[0]["traded"])
    # threshold above the signal (0.10) skips
    thr = pd.Series({REB: 0.50})
    res = run_backtest(data=data, min_names=2, threshold=thr)
    assert not bool(res["quarterly"].iloc[0]["traded"])
```

Listing 21: tests/test_rmt.py

```
"""
Tests for dispersion.rmt.cleaning: the Marchenko-Pastur + Laloux cleaning.
"""
import numpy as np
import pandas as pd
import pytest

from dispersion.rmt.cleaning import (corr_window, devolatilise, laloux_clip,
                                     spectral_features)


def _corr_df(A):
    n = A.shape[0]
    idx = list(range(n))
    return pd.DataFrame(A, index=idx, columns=idx)


# ---------------------------------------------------------------------------- #
# pure noise in -> filter finds nothing, comes out near-identity
# ---------------------------------------------------------------------------- #
def test_iid_noise_cleans_to_near_identity():
    rng = np.random.default_rng(7)
    X = rng.standard_normal((252, 100))
    C = _corr_df(np.corrcoef(X, rowvar=False))
    C2, d = laloux_clip(C, t_win=252)

    off_raw = np.abs(C.values[np.triu_indices(100, 1)]).mean()
    off_clean = np.abs(C2.values[np.triu_indices(100, 1)]).mean()
    assert d["k_signal"] == 0                       # no signal invented from noise
    assert off_clean < 0.05 * off_raw               # off-diagonals collapse to ~
    identity
    assert d["trace_prenorm"] == pytest.approx(100.0, rel=1e-10)


def test_diag_psd_trace_on_correlated_data():
    rng = np.random.default_rng(11)
    # one-factor returns: beta * market + idiosyncratic
    mkt = rng.standard_normal(252)
    X = 0.6 * mkt[:, None] + rng.standard_normal((252, 80))
    C = _corr_df(np.corrcoef(X, rowvar=False))
    C2, d = laloux_clip(C, t_win=252)
```

```
41
42      assert np.allclose(np.diag(C2.values), 1.0, atol=1e-12)          # diagonal stays
        1
43      assert np.linalg.eigvalsh(C2.values).min() > -1e-9               # PSD
44      assert d["trace_prenorm"] == pytest.approx(80.0, rel=1e-10)      # trace preserved
45      assert d["k_signal"] >= 1                                        # market mode
        kept
46      assert d["edge_laloux"] < d["edge_naive"]                        # Laloux edge
        below the naive one
47
48
49  def test_equicorrelation_structure_survives_cleaning():
50      # exact one-factor matrix C = (1-rho) I + rho J: cleaning should leave it alone
51      n, rho = 100, 0.3
52      C = _corr_df((1 - rho) * np.eye(n) + rho * np.ones((n, n)))
53      C2, d = laloux_clip(C, t_win=252)
54      assert np.abs(C2.values - C.values).max() < 1e-10
55      assert d["k_signal"] == 1                                        # one mode: the
        market
56
57
58  def test_devolatilise_scale_invariance():
59      # rescaling each name in log-return space must not change the z-correlations
60      # (log1p is nonlinear in simple returns, so the rescale is done in log space)
61      rng = np.random.default_rng(3)
62      dates = pd.date_range("2020-01-01", periods=300, freq="B")
63      r1 = pd.DataFrame(rng.standard_normal((300, 4)) * 0.01, index=dates)
64      r2 = np.expm1(np.log1p(r1) * [1.0, 5.0, 0.2, 3.0])   # per-name log-scale
        rescaling
65      c1 = devolatilise(r1).tail(200).dropna().corr().values
66      c2 = devolatilise(r2).tail(200).dropna().corr().values
67      assert np.abs(c1 - c2).max() < 1e-10                 # correlations unchanged by
        scale
68
69
70  def test_corr_window_thin_panel_returns_none():
71      rng = np.random.default_rng(5)
72      dates = pd.date_range("2020-01-01", periods=300, freq="B")
73      z = pd.DataFrame(rng.standard_normal((300, 50)), index=dates)
74      assert corr_window(z, dates[-1], list(range(50)), min_names=60) is None
75
76
77  def test_corr_window_returns_effective_t():
78      rng = np.random.default_rng(9)
79      dates = pd.date_range("2019-01-01", periods=400, freq="B")
80      z = pd.DataFrame(rng.standard_normal((400, 70)), index=dates)
81      z.iloc[-30:-20, 0] = np.nan                          # 10 missing days on one name
82      C, t_eff = corr_window(z, dates[-1], list(range(70)))
83      assert t_eff == 242 and C.shape == (70, 70)          # complete-case: 252 - 10
84
85
```

```python
def test_effective_t_kills_spurious_signal_on_noise():
    # with T_eff = 201, the MP edge must use q_mp = N/201, not N/252
    rng = np.random.default_rng(21)
    X = rng.standard_normal((201, 100))
    C = _corr_df(np.corrcoef(X, rowvar=False))
    _, d_wrong = laloux_clip(C, t_win=252)            # nominal T (the old bug)
    _, d_right = laloux_clip(C, t_win=201)            # effective T
    assert d_right["k_signal"] == 0                   # nothing in pure noise
    assert d_wrong["k_signal"] > 0                    # wrong T would invent signal


def test_rotation_metric():
    from dispersion.rmt.daily import _rotation
    idx = list(range(20))
    v = pd.Series(np.ones(20) / np.sqrt(20), index=idx)
    assert _rotation(v, v) == pytest.approx(0.0, abs=1e-12)          # same vector -> 0
    assert _rotation(v, -v) == pytest.approx(0.0, abs=1e-12)         # sign ignored
    w = pd.Series(np.zeros(20), index=idx); w.iloc[0] = 1.0
    v2 = pd.Series(np.zeros(20), index=idx); v2.iloc[1] = 1.0
    assert _rotation(v2, w) == pytest.approx(1.0, abs=1e-12)         # orthogonal -> 1
    assert np.isnan(_rotation(v, None))                              # no prior vector -> NaN
    u = pd.Series(np.ones(5), index=list(range(100, 105)))
    assert np.isnan(_rotation(v, u))                                 # overlap under 10 names -> NaN


def test_spectral_features_consistency():
    rng = np.random.default_rng(13)
    mkt = rng.standard_normal(252)
    X = 0.7 * mkt[:, None] + rng.standard_normal((252, 90))
    C = _corr_df(np.corrcoef(X, rowvar=False))
    feats, v1 = spectral_features(C, t_win=252, top=5)
    _, d = laloux_clip(C, t_win=252)
    assert feats["k_signal"] == d["k_signal"]
    assert 0 < feats["lam1_share"] < 1
    assert feats["absorption_top"] > feats["lam1_share"]             # top-5 includes top-1
    assert v1.sum() > 0                                              # sign convention
    assert np.isclose((v1**2).sum(), 1.0)                            # unit norm
```

Listing 22: tests/test_regime.py

```python
"""
Causality tests for dispersion.ml.regime: the HMM filtered posterior must use
only past and present observations, never the future.
"""
import numpy as np
import pytest
from hmmlearn.hmm import GaussianHMM
```

```python

from dispersion.ml.regime import hmm_filtered_posterior


def _fit_hmm(X, seed=0):
    m = GaussianHMM(3, covariance_type="full", random_state=seed, n_iter=50, tol=1e-3)
    m.fit(X)
    return m


def test_filtered_posterior_is_causal():
    # perturbing the future must leave every past filtered posterior unchanged
    rng = np.random.default_rng(1)
    X = rng.standard_normal((400, 4))
    m = _fit_hmm(X)
    post = hmm_filtered_posterior(m, X)

    t0 = 250
    X2 = X.copy()
    X2[t0 + 1:] += 5.0 * rng.standard_normal((len(X) - t0 - 1, 4))   # scramble the
    future
    post2 = hmm_filtered_posterior(m, X2)

    assert np.allclose(post[:t0 + 1], post2[:t0 + 1], atol=1e-12)     # past unchanged
    assert not np.allclose(post[t0 + 1:], post2[t0 + 1:])             # future did
    change


def test_filtered_rows_are_probabilities():
    rng = np.random.default_rng(2)
    X = rng.standard_normal((300, 4))
    post = hmm_filtered_posterior(_fit_hmm(X), X)
    assert np.allclose(post.sum(axis=1), 1.0, atol=1e-10)
    assert (post >= 0).all() and (post <= 1).all()


def test_filtered_equals_truncated_smoothed_at_last_step():
    # filtered posterior at t equals the smoothed posterior of the sequence cut off
    # at t (no future left to smooth over) -- cross-checked against hmmlearn
    rng = np.random.default_rng(3)
    X = rng.standard_normal((120, 4))
    m = _fit_hmm(X)
    filt = hmm_filtered_posterior(m, X)
    for t in (30, 60, 119):
        smoothed_trunc = m.predict_proba(X[:t + 1])[-1]   # last row, no future
        assert np.allclose(filt[t], smoothed_trunc, atol=1e-8)
```